Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 1_COD_SB

Attempt : 1
Total Mark : 50
Marks Obtained : 43.5

Section 1 : Coding

1. Problem Statement

Pradeep Narwal is analyzing multiple numerical ranges to identify prime numbers within each range. He is given several ranges defined by a starting number A and an ending number B. Using the Sieve of Eratosthenes, Pradeep wants to efficiently determine all prime numbers that lie within each range.

Write a program to ensure that all prime numbers between A and B (inclusive) are printed for every given range.

Input Format

The first line of input consists of an integer Q, representing the number of ranges.

vtu29890

The next Q lines each contain two integers A and B , representing the start and end of each range.

Output Format

For each range, print all prime numbers between A and B (inclusive) in a single line.

Prime numbers must be printed in ascending order, separated by a single space.

If a range contains no prime numbers, print an empty line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
1 5
10 15

Output: 2 3 5
11 13

Answer

```
#include<stdio.h>


#define MAX 1000000
int main(){
   int Q;
   scanf("%d",&Q);

   static int prime[MAX + 1];


   for(int i=0;i<=MAX;i++)
      prime[i]=1;
   prime[0]=prime[1]=0;

   for(int i=2;i*i<=MAX;i++){
      if(prime[i]==1){
         for(int j=i*i;j<=MAX;j+=i){
```

```
            prime[j]=0;
        }
    }
  }
  while( Q--){
    int A,B;
    scanf("%d %d",&A,&B);

    for(int i=A;i<=B;i++){
      if(prime[i]==1){
        printf("%d",i);
        if(i<B)
          printf(" ");
      }
    }
    printf("\n");
  }
  return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement:

A software company is developing a mathematical tool for students to help them understand factorization and modular arithmetic. As part of the tool, they need a function that calculates the product of all positive divisors (factors) of a given integer, with the result taken modulo 1000000007. The tool is intended to assist in solving mathematical problems and providing insights into number theory.

Your task is to implement a function that computes the product of all factors of a given integer n, and then display the result modulo 1000000007.

Examples :

Input: 12

Output: 1728

1 * 2 * 3 * 4 * 6 * 12 = 1728

Input Format

The input consists of a single integer n.

Output Format

The output displays a single integer which is the product of all factors of n modulo 1000000007.

Refer to the sample output for better understanding.

Sample Test Case

Input: 12

Output: 1728

Answer

```
#include<stdio.h>
#define MOD 1000000007

long long power(long long a,long long b){
   long long result=1;
   a=a%MOD;

   while(b>0){
      if(b%2==1)
      result=(result*a)%MOD;
   a=(a*a)%MOD;
   b=b/2;
   }
   return result;
}
int main(){
   long long n;
   scanf("%lld", &n);

   long long count=0;
```

```
    for(long long i=1;i*i<=n;i++){
        if(n%i==0){
            if(i*i==n)
                count+=1;
            else
                count+=2;
        }

    }
    long long exponent=count/2;
    long long answer=power(n,exponent);
    printf("%lld",answer);
    return 0;
}
```

Status : Partially correct **Marks : 6/10**

3. Problem Statement

Arjun is working on a project where he needs to analyze a series of integers within a specified range [A, B]. The project involves identifying certain numbers that are not divisible by any smaller positive integers other than 1 and themselves. These numbers are crucial for the next step of his analysis, as they have unique mathematical properties.

Arjun has been asked to implement an efficient way to find all such numbers in a given range. Given that the range can be large, he needs to ensure the solution is optimized.

Can you help him implement this task using the Simple Sieve of Eratosthenes?

Input Format

The first line of input consists of an integer A, representing the starting range.

The second line of input consists of an integer B, representing the ending range.

Output Format

The output prints a space-separated list of numbers that are not divisible by any

smaller positive integers other than 1 and themselves. within the range [A, B].

Refer to the sample output for formatting specifications

Sample Test Case

Input: 2
10

Output: 2 3 5 7

Answer

```
#include<stdio.h>

int main(){
    int A,B;
    scanf("%d%d",&A,&B);

    int prime[1000001];

    for(int i=0;i<=B;i++)
        prime[i]=1;
    prime[0]=prime[1]=0;

    for(int i=2;i*i<=B;i++)
        if(prime[i])
            for(int j=i*i;j<=B;j+=i)
                prime[j]=0;


    int first=1;
    for(int i=A;i<=B;i++){
        if(prime[i]){
            if(!first) printf(" ");
            printf("%d",i);
            first=0;
        }
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Raychel is working on a calendar synchronization tool where she needs to determine how often two repeating tasks will coincide. Each task occurs after a fixed number of days, and she must find the least number of days after which both tasks align again — in other words, the least common multiple (LCM) of the two intervals.

Raychel knows that the most efficient way to compute the LCM is by first calculating the greatest common divisor (GCD) using Euclid's Algorithm, and then using that result to derive the LCM. She also wants to make sure the program works for large inputs without running into overflow errors.

Help Raychel implement a program that calculates the LCM of two non-negative integers by first using Euclid's Algorithm to compute their GCD, and then deriving the LCM from it.

Example

Input:

15 20

Explanation:

GCD(15, 20) = 5LCM = (15 / 5) × 20 = 3 × 20 = 60

Output:

60

Input Format

The input consists of a single line containing two space-separated integers 'a' and 'b', representing the two input numbers.

Output Format

The output consists of a single line:

- If both 'a' and 'b' are non-zero, print a single integer — the least number that is divisible by both.
- If either 'a' or b is '0', print 0 as the result since no common multiple exists in such cases.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 15 20

Output: 60

Answer

```
#include<stdio.h>
long long gcd(long long a,long long b){
    while(b!=0){
        long long temp=b;
        b=a%b;
        a=temp;
    }
    return a;
}
int main(){
    long long a,b;
    scanf("%lld %lld",&a,&b);

    if(a==0||b==0){
        printf("0");
    }else{
        long long g=gcd(a,b);
        long long lcm=(a/g)*b;
        printf("%lld",lcm);
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Joanna is developing a utility tool for a math club's quiz event. One of the rounds involves finding the greatest common divisor (GCD) of a list of numbers. Instead of computing it manually for each pair, she wants to automate the task using code.

She decides to use Euclid's Algorithm, a well-known and efficient method to compute the GCD of two numbers. For a list of numbers, she will apply the algorithm iteratively across all elements in the array.

Help Joanna implement a program that takes an array of positive integers and computes the GCD of the entire array using Euclid's Algorithm.

Make sure the program also handles special cases:

If the array is empty, return -1.If the array has only one number, return that number as the GCD.

Example

Input:

3

12 18 24

Output:

6

Explanation:

GCD(12, 18) = 6GCD(6, 24) = 6Final result = 6

Input Format

The first line of input contains an integer n, representing the number of elements in the array.

The second line contains n space-separated positive integers, each representing

an element of the array.

Output Format

The output consists of a single line:

- If the array contains two or more elements, print a single integer representing the GCD of all array elements.
- If the array contains only one element, print that element as the GCD.
- If the array is empty, print -1 as a convention to indicate undefined GCD.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
8 12

Output: 4

Answer

```
#include<stdio.h>

int gcd(int a,int b){
    while (b){
        int t=b;
        b=a%b;
        a=t;
    }
    return a;
}
int main(){
    int n,arr[10],i,result;
    scanf("%d",&n);

    if(n==0){
        printf("-1");
        return 0;
    }
```

```
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    result =arr[0];

    for(i=1;i<n;i++)
        result=gcd(result,arr[i]);

    printf("%d",result);

    return 0;
}
```

Status : Partially correct **Marks : 7.5/10**

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 1_COD_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

In the field of cryptography, especially in public-key algorithms like RSA, it is crucial to identify pairs of numbers that are coprime — i.e., they do not share any common divisors other than 1.

Before generating secure keys, cryptographic systems often verify whether two large numbers are coprime. This process uses a classical method known as Euclid's Algorithm, which efficiently determines the greatest common divisor (GCD).

You are given a list of number pairs. Your task is to determine whether each pair of integers is coprime using Euclid's Algorithm. If the GCD of the two numbers is 1, output "Coprime". Otherwise, output "Not Coprime".

Example

Input:

3

15 28

12 35

8 8

Explanation:

GCD(15, 28) = 1 Coprime

GCD(12, 35) = 1 Coprime

GCD(8, 8) = 8 Not Coprime

Output:

Coprime

Coprime

Not Coprime

Input Format

- The first line contains an integer 't' — the number of pairs to check.
- Each of the next 't' lines contains two space-separated integers 'a' and 'b'.

Output Format

- For each pair, output a single line:
- "Coprime" if the two numbers are coprime
- "Not Coprime" otherwise

- Each output must appear on a new line, in the same order as the input pairs.

vtu29890

Refer to the sample output for formatting specifictions.

Sample Test Case

Input: 1
17 29

Output: Coprime

Answer

```
#include<stdio.h>
long long gcd(long long a,long long b){
   while(b!=0){
      long long temp=b;
      b=a%b;
      a=temp;
   }
   return a;
}
int main(){
   int t;
   long long a,b;
   scanf("%d",&t);
   for(int i=0;i<t;i++){
      scanf("%lld %lld",&a,&b);

      if(gcd(a,b)==1)
         printf("coprime\n");
      else
         printf("not coprime\n");
   }
   return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Sai Kishore is working on a tool that identifies special numbers in a range. These numbers must have only two divisors: 1 and themselves. His task is to count how many of these special numbers exist below a given threshold N. Since N can be as large as one million, Sai Kishore needs an efficient way to find these numbers in a given range.

Sai Kishore decides to use a technique called Sieve of Eratosthenes, which is well-suited for efficiently identifying these special numbers.

Can you help him implement a program that counts how many such numbers exist below N?

Example:

Input:

10

Output:

4

Explanation:

The numbers less than 10 that have only two divisors (1 and themselves) are: 2, 3, 5, and 7. Thus, the output is 4.

Input Format

The input consists of a single integer N, representing the upper bound for the search.

Output Format

The output should be a single integer, representing the count of special numbers less than N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
Output: 4

Answer

```
#include<stdio.h>
int main(){
    int N;
```

```
    if(scanf("%d",&N)!=1) return 0;

    if(N<=2){
        printf("0\n");
        return 0;

    }
    char *prime=(char*)malloc((N+1)* sizeof(char));

    for(int i=0;i<=N;i++)prime[i]=1;


    prime[0]=prime[1]=0;
    for(int i=2;i*i<N;i++){
        if(prime[i]==1){
            for(int j=i*i;j<N;j+=i){
                prime[j]=0;
            }
        }
    }
    int count=0;
    for(int i=2;i<N;i++){
        if(prime[i]==1)
            count++;

    }
    printf("%d",count);
    free(prime);
    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Arjun is analyzing a numerical range to find special pairs of prime numbers. He is given a range defined by two integers L and R, and a target value T. Within the range [L,R], Arjun wants to identify all pairs of prime numbers whose sum equals T.

Write a program to ensure that all such valid prime pairs are printed in

ascending order.

Input Format

The first line contains three space-separated integers:

- L starting value of the range
- R ending value of the range
- T target sum

Output Format

The output prints each valid pair of prime numbers whose sum equals T, one pair per line.

Each line should contain two space-separated integers representing a valid prime pair.

If no such pairs exist, no output should be printed.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 1 40 10

Output: 3 7
5 5

Answer

```
#include<stdio.h>

int main(){
   int L,R,T;
   scanf("%d %d %d",&L,&R,&T);

   int prime[R+1];

   for(int i=0;i<=R;i++)
      prime[i]=1;

   prime[0]=prime[1]=0;
```

```
    for(int i=2;i*i<=R;i++)
        if(prime[i])
            for(int j=i*i;j<=R;j+=i)
                prime[j]=0;

    for(int i=L;i<=R;i++){
        if(prime[i]){
            int j=T-i;
            if(j>=i&& j<=R && prime[j])
                printf("%d %d\n",i,j);
        }
    }
    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Bella is developing a number puzzle game where each level involves simplifying equations using number theory. In one level, she comes across two large positive integers and is asked to reduce them using their greatest common divisor (GCD).

However, the puzzle has an extra twist:

She must express the GCD as a linear combination of the two numbers – that is, she needs to find two integers x and y such that: GCD(a, b) = a * x + b * y

Help Bella write a program that, given two integers a and b, finds:

The GCD of a and bThe coefficients 'x' and y" that satisfy the linear combination condition

Example

Input

48 18

Output:

6 -1 3

Explanation:

The GCD of 48 and 18 is 6.

Using the Extended Euclidean Algorithm, we can find integers x = -1 and y = 3 such that:

48 × (-1) + 18 × 3 = -48 + 54 = 6

This satisfies Bézout's Identity: GCD(48, 18) = 48x + 18y = 6.

Input Format

- The input consists of a single line with two space-separated integers 'a' and 'b'.

Output Format

The output prints three space-separated integers in a single line:

- The first integer is the GCD of a and b.
- The second integer is the coefficient x such that a * x + b * y = GCD.
- The third integer is the coefficient y such that a * x + b * y = GCD.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 45 21

Output: 3 1 -2

Answer

```
#include<stdio.h>
int gcdextended(int a,int b,int *x,int *y){
   if(b==0){
      *x=1;
      *y=0;
      return a;
   }
   int x1,y1;
```

```
    int gcd=gcdextended(b, a % b,&x1,&y1);

    *x=y1;
    *y=x1-(a/b)*y1;
    return gcd;
}
int main(){
    int a,b,x,y;
    scanf("%d %d",&a,&b);

    int g=gcdextended(a,b,&x,&y);

    printf("%d %d %d", g,x,y);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 1_COD_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Haritha is working on a project to identify pairs of numbers in a given range [L, R] where each number in the pair is divisible only by 1 and itself.

She needs to find pairs of such numbers that differ by exactly 2. These pairs of numbers should be displayed efficiently within the range.

Help her to implement the task.

Input Format

The input consists of two integers L and R separated by a new line.

Output Format

The output prints each pair of numbers within the range [L, R] on a new line, in the format: a b where a and b are numbers that differ by exactly 2 and are

divisible only by 1 and themselves.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 30
60

Output: 41 43

Answer

```
#include <stdio.h>
#include <math.h>

// Function to check if a number is prime
int isPrime(int n) {
    if (n < 2)
        return 0;

    for (int i = 2; i <= sqrt(n); i++) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main() {
    int L, R;
    scanf("%d%d", &L, &R);

    for (int i = L; i <= R - 2; i++) {
        if (isPrime(i) && isPrime(i + 2)) {
            printf("%d %d\n", i, i + 2);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Neha is designing a secret code system for her cybersecurity class. She discovers that certain numbers with exactly two distinct divisors (1 and themselves) create the most secure keys when their reversed digit sum is divisible by a given validation number K.

Neha needs to find all such numbers up to a specified limit N that meet both conditions. These numbers are crucial for the security of her code system. Given the large range of numbers, she needs an efficient solution to identify them.

Help her write a program to find all such numbers ≤ N that meet this criteria.

Input Format

The input consists of two space separated integers N and K

Output Format

The output prints all numbers that meet both conditions. Each number should be printed space-separated.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 10 1

Output: 2 3 5 7

Answer

```
#include <stdio.h>

// Check prime
int isPrime(int n) {
   if (n < 2) return 0;
   for (int i = 2; i * i <= n; i++) {
      if (n % i == 0) return 0;
   }
   return 1;
```

```
}

// Reverse number
int reverse(int n) {
    int rev = 0;
    while (n > 0) {
        rev = rev * 10 + (n % 10);
        n /= 10;
    }
    return rev;
}

// Sum of digits
int digitSum(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

int main() {
    int N, K;
    scanf("%d %d", &N, &K);

    for (int i = 2; i <= N; i++) {
        if (isPrime(i)) {
            int rev = reverse(i);
            int sum = digitSum(rev);

            if (sum % K == 0) {
                printf("%d ", i);
            }
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Ashwin is working on a system to identify numbers within a given range [L, R] that have a special property: they are divisible only by 1 and themselves.

Ashwin needs to find pairs of such numbers where both numbers are rearrangements of each other's digits. Ashwin must efficiently identify all such pairs in the given range.

Help him to implement the task.

Input Format

The input consists of two space separated integers L and R

Output Format

The output prints each pair of numbers within the range [L, R] that satisfy both conditions: they are divisible only by 1 and themselves, and they are rearrangements of each other's digits. Each pair is printed in the format: p1 p2, where p1 and p2 are numbers that are rearrangements of each other's digits.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 10 100

Output: 13 31
17 71
37 73
79 97

Answer

```
#include <stdio.h>
#include <string.h>

// Check prime
int isPrime(int n) {
   if (n < 2) return 0;
   for (int i = 2; i * i <= n; i++) {
      if (n % i == 0) return 0;
```

```
    }
    return 1;
}

// Sort digits of number
void sortDigits(int n, char *str) {
    int i = 0;
    while (n > 0) {
        str[i++] = (n % 10) + '0';
        n /= 10;
    }
    str[i] = '\0';

    // simple bubble sort
    for (int x = 0; x < i - 1; x++) {
        for (int y = x + 1; y < i; y++) {
            if (str[x] > str[y]) {
                char temp = str[x];
                str[x] = str[y];
                str[y] = temp;
            }
        }
    }
}

int main() {
    int L, R;
    scanf("%d %d", &L, &R);

    char s1[10], s2[10];

    for (int i = L; i <= R; i++) {
        if (!isPrime(i)) continue;

        sortDigits(i, s1);

        for (int j = i + 1; j <= R; j++) {
            if (!isPrime(j)) continue;

            sortDigits(j, s2);

            if (strcmp(s1, s2) == 0) {
```

```
            printf("%d %d\n", i, j);
        }
    }
  }

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Suresh is studying prime numbers and is interested in identifying a special category of primes. Given a number N, he wants to find all prime numbers less than or equal to N whose sum of digits is also a prime number. To solve this efficiently, Suresh decides to use the Sieve of Eratosthenes for prime number generation.

Write a program to ensure that all such prime numbers are printed in increasing order.

Input Format

The input consists of single integer N, representing the upper limit of the range.

Output Format

The output prints all prime numbers less than or equal to N whose digit sum is also a prime number.

The numbers must be printed in ascending order, separated by a single space.

If no such numbers exist, print nothing.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 50
Output: 2 3 5 7 11 23 29 41 43 47

Answer

```
#include <stdio.h>

#define MAX 1000001

int isPrime[MAX];

// Function to compute digit sum
int digitSum(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

int main() {
    int N;
    scanf("%d", &N);

    // Initialize sieve
    for (int i = 0; i <= N; i++) {
        isPrime[i] = 1;
    }

    isPrime[0] = isPrime[1] = 0;

    for (int i = 2; i * i <= N; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= N; j += i) {
                isPrime[j] = 0;
            }
        }
    }

    // Process primes
    for (int i = 2; i <= N; i++) {
        if (isPrime[i]) {
            int sum = digitSum(i);
            if (isPrime[sum]) {
                printf("%d ", i);
```

```
        }
    }
}

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 2_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neha is organizing a sequence of integers where some elements are zeroes. She wants to rearrange the sequence so that all zeroes are moved to the end while keeping the relative order of the non-zero elements intact. Help Neha perform this task efficiently using the two-pointer technique to modify the sequence in place.

Input Format

The first line of input consists of an integer n, representing the size of the array nums[].

The second line contains n integers, representing the elements of the array nums[].

Output Format

vtu29890

The output prints the modified array nums[], with all zeroes moved to the end and the relative order of the non-zero elements maintained.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0 1 0 3 12

Output: 1 3 12 0 0

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int j = 0; // position for next non-zero

    for (int i = 0; i < n; i++) {
        if (arr[i] != 0) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            j++;
        }
    }

    // Print result
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arjun has a list of integers representing a specific sequence, and he wants to find its next permutation in lexicographical order. If the sequence is the largest permutation possible, he should rearrange it into the smallest permutation. Help Arjun solve this problem efficiently using the two-pointer technique to rearrange the sequence in place with constant extra memory.

Input Format

The first line of input consists of an integer n, representing the size of the array nums[].

The second line contains n integers, representing the elements of the array nums[].

Output Format

The output prints "Next permutation:" followed by n space-separated integers on the next line, representing the next lexicographically greater permutation of the given sequence.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Next permutation:
1 3 2

Answer

```
#include <stdio.h>

// Function to reverse array
```

```
void reverse(int arr[], int start, int end) {
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[10];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int i;

    // Step 1: find first decreasing element
    for (i = n - 2; i >= 0; i--) {
        if (arr[i] < arr[i + 1]) {
            break;
        }
    }

    if (i >= 0) {
        int j;

        // Step 2: find just greater element
        for (j = n - 1; j > i; j--) {
            if (arr[j] > arr[i]) {
                break;
            }
        }

        // Step 3: swap
        int temp = arr[i];
        arr[i] = arr[j];
```

```
        arr[j] = temp;
    }

    // Step 4: reverse remaining
    reverse(arr, i + 1, n - 1);

    // Output
    printf("Next permutation:\n");
    for (int k = 0; k < n; k++) {
        printf("%d ", arr[k]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Priya is searching for a specific word (needle) in a large text (haystack). She needs to find the index of the first occurrence of the word in the text. If the word is not present, she should return -1. Help Priya solve this efficiently using the two-pointer technique.

Input Format

The first line of input consists of a string haystack, representing the text in which the word is to be searched.

The second line of input consists of a string needle, representing the word to search for in the text.

Output Format

The output prints "The index of the first occurrence of needle in haystack is: X", where X represents the index value.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: sadbutsad
sad
Output: The index of the first occurrence of needle in haystack is: 0

Answer

```
#include <stdio.h>
#include <string.h>

int main() {
    char haystack[31], needle[31];

    scanf("%s", haystack);
    scanf("%s", needle);

    int n = strlen(haystack);
    int m = strlen(needle);

    int index = -1;

    for (int i = 0; i <= n - m; i++) {
        int j = 0;

        while (j < m && haystack[i + j] == needle[j]) {
            j++;
        }

        if (j == m) {
            index = i;
            break;
        }
    }

    printf("The index of the first occurrence of needle in haystack is: %d", index);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ramesh is working with an array of integers and needs to remove all occurrences of a specific value from it.

To accomplish this efficiently, he uses the Two-Pointer technique, where one pointer scans the array and another pointer tracks the position to place valid elements. Given an integer array nums and an integer val, Ramesh must modify the array in-place so that all elements equal to val are removed while preserving the order of the remaining elements.

Write a program to ensure that the number of remaining elements and the modified array are printed correctly.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

The third line contains an integer val, representing the value to be removed from the array.

Output Format

The first line prints "Number of elements after removal: " followed by count of elements not equal to val.

The second line prints "Modified array:"

Then print the modified array elements (up to index k) separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 2 3

3
Output: Number of elements after removal: 2
Modified array:
2 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[15];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int val;
    scanf("%d", &val);

    int j = 0;

    // Remove val using two-pointer
    for (int i = 0; i < n; i++) {
        if (arr[i] != val) {
            arr[j] = arr[i];
            j++;
        }
    }

    // Output
    printf("Number of elements after removal: %d\n", j);
    printf("Modified array:\n");

    for (int i = 0; i < j; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Anil is working with a sorted array of integers and wants to remove duplicate elements efficiently. To solve this problem, he uses the Two-Pointer technique, where one pointer scans the array and another pointer keeps track of the position to store unique elements. Given a sorted integer array nums, Anil must modify the array in-place so that each unique element appears only once.

Write a program to ensure that the number of unique elements and the modified array are printed correctly.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the sorted array.

Output Format

The first line prints "Number of unique elements: k"

On the next line, print "Modified array:"

Then, print the first k elements of the modified array, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8
-1 -1 0 0 1 1 2 2
Output: Number of unique elements: 4
Modified array:

-1 0 1 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[15];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (n == 0) return 0;

    int j = 0;

    // Remove duplicates
    for (int i = 1; i < n; i++) {
        if (arr[i] != arr[j]) {
            j++;
            arr[j] = arr[i];
        }
    }

    int k = j + 1;

    // Output
    printf("Number of unique elements: %d\n", k);
    printf("Modified array:\n");

    for (int i = 0; i < k; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Karthik is organizing marbles of three different colors: red, white, and blue. Each color is represented by an integer—0 for red, 1 for white, and 2 for blue. Karthik wants to sort the marbles so that all marbles of the same color are adjacent, and they are arranged in the order red, white, and blue. Help Karthik solve this problem efficiently using the two-pointer technique.

Input Format

The first line of input consists of an integer n, representing the number of marbles in the array nums[].

The second line contains n integers, representing the marbles' colors, where each integer is either 0, 1, or 2.

Output Format

The output prints the sorted array, where all marbles are grouped by color in the order red (0), white (1), and blue (2).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 0 2 1 1 0

Output: 0 0 1 1 2 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int nums[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int low = 0, mid = 0, high = n - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {  // red
            int temp = nums[low];
            nums[low] = nums[mid];
            nums[mid] = temp;
            low++;
            mid++;
        } else if (nums[mid] == 1) {  // white
            mid++;
        } else {  // nums[mid] == 2 (blue)
            int temp = nums[mid];
            nums[mid] = nums[high];
            nums[high] = temp;
            high--;
        }
```

```
    }

    for (int i = 0; i < n; i++) {
        printf("%d ", nums[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Kavya is an avid reader who loves finding hidden patterns in texts. One day, she stumbled upon a string of characters and wondered about the longest palindromic substring it contains. Help Kavya identify this substring using an efficient algorithm based on the two-pointer technique.

Input Format

The input consists of a single string s.

Output Format

The output displays "Longest Palindromic Substring: " followed by the longest palindrome found.

Refer the sample output for format specifications.

Sample Test Case

Input: babad

Output: Longest Palindromic Substring: bab

Answer

```
#include <stdio.h>
#include <string.h>

// Function to expand around center and return length of palindrome
int expandAroundCenter(char *s, int left, int right) {
```

```
    int n = strlen(s);
    while (left >= 0 && right < n && s[left] == s[right]) {
        left--;
        right++;
    }
    return right - left - 1; // length of palindrome
}

int main() {
    char s[25]; // max length 20, extra buffer
    scanf("%s", s);

    int start = 0, end = 0;
    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        int len1 = expandAroundCenter(s, i, i);     // odd-length palindrome
        int len2 = expandAroundCenter(s, i, i + 1); // even-length palindrome
        int len = (len1 > len2) ? len1 : len2;

        if (len > end - start + 1) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }

    printf("Longest Palindromic Substring: ");
    for (int i = start; i <= end; i++) {
        printf("%c", s[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rohit is analyzing an array of integers to find special combinations of numbers. Using the Two-Pointer technique, he wants to identify all unique triplets in the array whose sum is equal to 0. Given an integer array, Rohit

must find and print all such triplets, ensuring that no duplicate triplets are included in the result.

Write a program to ensure that all valid triplets with sum zero are printed correctly.

Example 1

Input:

6

-1 0 1 2 -1 -4

Output:

-1 -1 2

-1 0 1

Explanation:

nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.

nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.

nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.

The distinct triplets are [-1,0,1] and [-1,-1,2].

Notice that the order of the output and the order of the triplets does not matter.

Example 2

Input:

3

0 1 1

Output:

No triplets found with sum 0.

Explanation:

The only possible triplet does not sum up to 0.

Example 3

Input:

3

0 0 0

Output:

0 0 0

Explanation:

The only possible triplet sums up to 0.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints each unique triplet whose sum is 0 on a separate line.

Each triplet must be printed in ascending order (as they appear after sorting).

If no such triplets exist, print "No triplets found with sum 0."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
-1 0 1 2 -1 -4
Output: -1 -1 2
-1 0 1

vtu29890

Answer

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort
int cmpfunc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    // Sort the array
    qsort(nums, n, sizeof(int), cmpfunc);

    int found = 0;

    for (int i = 0; i < n - 2; i++) {
        // Skip duplicates for i
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1;
        int right = n - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                printf("%d %d %d \n", nums[i], nums[left], nums[right]);
                found = 1;

                // Skip duplicates for left and right
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;

                left++;
                right--;
            } else if (sum < 0) {
```

```
            left++;
        } else {
            right--;
        }
    }
}

if (!found) {
    printf("No triplets found with sum 0.");
}

return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Rahul is given an array of integers and a target value T. Using the Two-Pointer technique, he wants to find two distinct elements in the array such that the sum of the pair is closest to the target value. To apply this technique efficiently, the array must first be sorted.

Write a program to ensure that the pair of numbers whose sum is closest to T is identified and printed correctly.

Input Format

The first line contains two space-separated integers n and T, where:

- n is the number of elements in the array
- T is the target sum

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints two space separated integers representing the pair whose sum is closest to the target.

The output pair must be printed in ascending order.

If multiple pairs have the same closest sum, print the first pair found by the two-pointer traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 50
10 22 28 29 30 40

Output: 10 40

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

// Comparison function for qsort
int cmpfunc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n, T;
    scanf("%d %d", &n, &T);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Sort the array
    qsort(arr, n, sizeof(int), cmpfunc);

    int left = 0, right = n - 1;
    int bestSum = INT_MAX;
    int bestA = 0, bestB = 0;

    while (left < right) {
        int sum = arr[left] + arr[right];
        int diff = abs(sum - T);
```

```
        if (diff < abs(bestSum - T)) {
            bestSum = sum;
            bestA = arr[left];
            bestB = arr[right];
        }

        if (sum < T) {
            left++;
        } else {
            right--;
        }
    }

    // Print in ascending order
    if (bestA < bestB)
        printf("%d %d", bestA, bestB);
    else
        printf("%d %d", bestB, bestA);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 2_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

At AquaPark, the maintenance team has a row of vertical water tanks of varying heights arranged side by side along a riverbank.The team wants to identify two tanks that, together with the riverbank, can hold the maximum volume of water between them when the space between them is filled.Each tank's height represents its capacity, and the distance between tanks determines the width of the container formed.

Your task is to read the heights of the tanks and determine the maximum amount of water (volume) that can be contained between any two tanks.

Input Format

The first line of input contains an integer n representing the number of water tanks.

The second line of input contains n space-separated integers representing the heights of the tanks.

Output Format

The output prints a single integer representing the maximum volume of water that can be contained between two tanks.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
Output: 6

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int height[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &height[i]);
    }

    int left = 0, right = n - 1;
    int maxArea = 0;

    while (left < right) {
        int h = (height[left] < height[right]) ? height[left] : height[right];
        int width = right - left;
        int area = h * width;

        if (area > maxArea) {
            maxArea = area;
        }

        // Move the pointer at the shorter tank
```

```
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    printf("%d", maxArea);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Dhanya is tasked with writing a program to merge data from two sensors into a single unified dataset. Each sensor produces an array of data readings over time. The goal is to create a program that takes the data arrays from two sensors, merges them, and produces a single sorted array of merged data readings.

Can you assist Dhanya in this using two-pointer technique?

Input Format

The first line of input consists of an integer n, representing the number of readings produced by the first sensor.

The second line consists of an integer m, representing the number of readings produced by the second sensor.

The third line consists of n space-separated integers, representing the readings from the first sensor.

The fourth line consists of m space-separated integers, representing the readings from the second sensor.

Output Format

The output prints the merged and sorted sensor readings in ascending order, separated by space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
4
1 2 8
3 5 6 7

Output: 1 2 3 5 6 7 8

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort
int cmpfunc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int arr1[n], arr2[m];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr1[i]);
    }
    for (int j = 0; j < m; j++) {
        scanf("%d", &arr2[j]);
    }

    // Sort both arrays
    qsort(arr1, n, sizeof(int), cmpfunc);
    qsort(arr2, m, sizeof(int), cmpfunc);

    int i = 0, j = 0;
    while (i < n && j < m) {
        if (arr1[i] <= arr2[j]) {
            printf("%d ", arr1[i]);
```

```
            i++;
        } else {
            printf("%d ", arr2[j]);
            j++;
        }
    }

    // Print remaining elements
    while (i < n) {
        printf("%d ", arr1[i]);
        i++;
    }
    while (j < m) {
        printf("%d ", arr2[j]);
        j++;
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

At ShopEase, two branches maintain sorted lists of product IDs available in their inventories.The regional manager wants to find the list of product IDs that are common to both branches to plan joint promotions.

Your task is to read the sorted lists of product IDs from both branches, find their intersection and print the common IDs in sorted order.If there are no common products, print "No Common Intersection".

Input Format

The first line of input contains an integer m representing the number of products in the first branch.

The second line of input contains m space-separated integers representing the sorted product IDs in the first branch.

The third line of input contains an integer n representing the number of products

in the second branch.

The fourth line of input contains n space-separated integers representing the sorted product IDs in the second branch.

Output Format

The output prints the common product IDs in sorted order, space-separated.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
101 102 103 104 105
6
102 104 106 108 110 115
Output: 102 104

Answer

```
#include <stdio.h>

int main() {
    int m, n;
    scanf("%d", &m);
    int arr1[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr1[i]);
    }

    scanf("%d", &n);
    int arr2[n];
    for (int j = 0; j < n; j++) {
        scanf("%d", &arr2[j]);
    }

    int i = 0, j = 0;
    int found = 0;

    while (i < m && j < n) {
        if (arr1[i] == arr2[j]) {
```

```
            printf("%d ", arr1[i]);
            found = 1;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }

    if (!found) {
        printf("No Common Intersection");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

At FreshMart Grocery, the cashier needs to quickly find pairs of products whose prices add up to a specific gift card value, so they can suggest bundles to customers using their gift cards efficiently.

You are given a sorted list of product prices. Your task is to find all unique pairs of products whose prices sum exactly to the given gift card value.If no such pairs exist, print "No Pairs Found".

Input Format

The first line of input contains an integer n representing the number of products.

The second line of input contains n space-separated integers representing the sorted prices of the products.

The third line of input contains an integer k representing the gift card value.

Output Format

The output prints each pair of product prices that sum to k on a separate line,

with the two prices separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 3 4 5 6 8
10

Output: 2 8
4 6

Answer

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &k);

    int left = 0, right = n - 1;
    int found = 0;

    while (left < right) {
        long long sum = (long long)arr[left] + arr[right];
        if (sum == k) {
            printf("%d %d\n", arr[left], arr[right]);
            found = 1;
            left++;
            right--;
            // Skip duplicates
            while (left < right && arr[left] == arr[left - 1]) left++;
            while (left < right && arr[right] == arr[right + 1]) right--;
        } else if (sum < k) {
            left++;
        } else {
```

```
            right--;
        }
    }

    if (!found) {
        printf("No Pairs Found");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 3_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 46.5

Section 1 : Coding

1. Problem Statement

Anil is working on a project that involves counting occurrences of a specific pattern within a given text. He needs a program that efficiently finds and counts all occurrences of a pattern within a given text using the Knuth-Morris-Pratt (KMP) algorithm.

Your task is to help Anil by developing a program that takes a text and a pattern as input and counts the number of times the pattern appears in the text using the KMP algorithm.

Input Format

The first line contains a string txt, representing the text in which Anil wants to search for the pattern.

The second line contains a string pat, representing the pattern Anil wants to search for in the text.

Output Format

The output displays a single integer, representing the count of occurrences of the pattern pat in the text txt.

If the pattern is not found, nothing will be printed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: AABAACAADAABAABA
AABA

Output: 3

Answer

```
#include <stdio.h>
#include <string.h>

// Function to compute LPS array
void computeLPSArray(char* pat, int M, int* lps) {
   int length = 0; // length of the previous longest prefix suffix
   lps[0] = 0;     // lps[0] is always 0
   int i = 1;

   while (i < M) {
      if (pat[i] == pat[length]) {
         length++;
         lps[i] = length;
         i++;
      } else {
         if (length != 0) {
            length = lps[length - 1];
         } else {
            lps[i] = 0;
            i++;
         }
      }
```

```
    }
}

// KMP search function
int KMPSearch(char* pat, char* txt) {
    int M = strlen(pat);
    int N = strlen(txt);

    int lps[M];
    computeLPSArray(pat, M, lps);

    int i = 0; // index for txt[]
    int j = 0; // index for pat[]
    int count = 0;

    while (i < N) {
        if (pat[j] == txt[i]) {
            i++;
            j++;
        }

        if (j == M) {
            count++;
            j = lps[j - 1]; // continue searching
        } else if (i < N && pat[j] != txt[i]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return count;
}

int main() {
    char txt[55], pat[30];
    scanf("%s", txt);
    scanf("%s", pat);

    int result = KMPSearch(pat, txt);
    if (result > 0) {
```

```
        printf("%d", result);
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohan is developing a text analysis tool to efficiently find repeated patterns inside large documents. To speed up the searching process, he uses the Boyer–Moore string matching algorithm, specifically relying on the Bad Character Heuristic as the main data structure/technique.

Given a text string and a pattern string, write a program to ensure that the task is done by counting how many times the pattern occurs in the text.

Input Format

The first line contains a string representing the text (may contain spaces and special characters).

The second line contains a string representing the pattern to be searched.

Output Format

The output prints a single integer representing the number of occurrences of the pattern in the text.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: abababababa
ab
Output: 4

Answer

```
#include <stdio.h>
```

```
#include <string.h>
#include <limits.h>

#define NO_OF_CHARS 256

// Function to build the bad character table
void badCharHeuristic(char *pat, int m, int badchar[NO_OF_CHARS]) {
    for (int i = 0; i < NO_OF_CHARS; i++)
        badchar[i] = -1;
    for (int i = 0; i < m; i++)
        badchar[(unsigned char)pat[i]] = i;
}

// Boyer-Moore search using Bad Character Heuristic
int search(char *txt, char *pat) {
    int n = strlen(txt);
    int m = strlen(pat);
    int badchar[NO_OF_CHARS];
    badCharHeuristic(pat, m, badchar);

    int count = 0;
    int s = 0; // shift of the pattern

    while (s <= (n - m)) {
        int j = m - 1;

        // Keep reducing j while characters match
        while (j >= 0 && pat[j] == txt[s + j])
            j--;

        if (j < 0) {
            count++;
            // Shift pattern so next character in text aligns
            s += (s + m < n) ? m - badchar[(unsigned char)txt[s + m]] : 1;
        } else {
            // Shift pattern using bad character heuristic
            int shift = j - badchar[(unsigned char)txt[s + j]];
            if (shift < 1) shift = 1;
            s += shift;
        }
    }
    return count;
```

```
}

int main() {
    char txt[105], pat[105];
    // Read text (may contain spaces)
    fgets(txt, sizeof(txt), stdin);
    txt[strcspn(txt, "\n")] = '\0'; // remove newline
    // Read pattern
    fgets(pat, sizeof(pat), stdin);
    pat[strcspn(pat, "\n")] = '\0'; // remove newline

    int result = search(txt, pat);
    if (result > 0) {
        printf("%d", result);
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kiran is working on a string analysis module where he needs to identify repeating patterns efficiently. To achieve this, he uses the LPS (Longest Prefix Suffix) array from the KMP string matching algorithm as the main data structure.

Given a string, write a program to ensure that the task is done by finding the length of the longest proper prefix that is also a suffix, ensuring the prefix and suffix do not overlap.

Example

Input:

aabcdaabc

Output:

4

Explanation:

The string "aabc" is the longest prefix which is also a suffix. So the length of the string "aabc" is printed as 4.

Input Format

The input consists of a string S, representing the input string.

Output Format

The output prints an integer representing the length of the longest proper prefix which is also a suffix.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabcdaabc

Output: 4

Answer

```
#include <stdio.h>
#include <string.h>

void computeLPSArray(char *s, int n, int *lps) {
    int length = 0;
    lps[0] = 0;
    int i = 1;

    while (i < n) {
        if (s[i] == s[length]) {
            length++;
            lps[i] = length;
            i++;
        } else {
            if (length != 0) {
                length = lps[length - 1];
            } else {
                lps[i] = 0;
                i++;
```

```
        }
      }
    }
  }

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int lps[n];
    computeLPSArray(s, n, lps);

    printf("%d", lps[n - 1]);  // no extra newline or space
    return 0;
}
```

Status : Partially correct ***Marks : 6.5/10***

4. Problem Statement

Liam is a software engineer who loves analyzing strings for patterns. One day, while reviewing usernames in a system, he noticed that some usernames read the same backward and forward (palindromes). He wants to count all palindromic sequences of characters in a username efficiently.

To accomplish this, Liam decides to use Manacher's Algorithm, which finds all palindromic substrings in linear time.

Given a string representing the username, help Liam count all the possible palindromic substrings using Manacher's Algorithm.

Input Format

The input consists of a single line containing the string S.

The string consists of lowercase English letters only.

Output Format

The output prints a single integer, representing the total number of palindromic

substrings present in the username.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: abaa

Output: 6

Answer

```
#include <stdio.h>
#include <string.h>

// Function to count palindromic substrings using Manacher's Algorithm
int countPalindromes(char *s) {
    int n = strlen(s);
    char t[2 * n + 3];
    int P[2 * n + 3];

    // Transform string: "^#a#b#a#$"
    t[0] = '^';
    int idx = 1;
    for (int i = 0; i < n; i++) {
        t[idx++] = '#';
        t[idx++] = s[i];
    }
    t[idx++] = '#';
    t[idx++] = '$';
    t[idx] = '\0';

    int m = idx;
    int center = 0, right = 0;
    int total = 0;

    for (int i = 1; i < m - 1; i++) {
        int mirror = 2 * center - i;
        if (i < right)
            P[i] = (P[mirror] < (right - i)) ? P[mirror] : (right - i);
        else
            P[i] = 0;
```

```
        // Expand around center i
        while (t[i + 1 + P[i]] == t[i - 1 - P[i]])
            P[i]++;

        // Update center and right boundary
        if (i + P[i] > right) {
            center = i;
            right = i + P[i];
        }

        // Each expansion contributes palindromes
        total += (P[i] + 1) / 2;
    }

    return total;
}

int main() {
    char s[105];
    scanf("%s", s);

    int result = countPalindromes(s);
    printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Aryan is working on a text processing project where he needs to analyze strings to find patterns efficiently. He decides to use the Z-algorithm, which for each position in a string, computes the length of the longest substring starting from that position which is also a prefix of the string.

Your task is to implement a program that computes the Z-array for a given string.

Input Format

The input consists of a single line containing the string S.

S may contain only uppercase and lowercase English letters.

Output Format

The output prints the Z-array of S as a sequence of integers separated by spaces.

The i-th element of the Z-array represents the length of the longest substring starting at position i which matches the prefix of S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabxaab

Output: 0 1 0 0 3 1 0

Answer

```
#include <stdio.h>
#include <string.h>

// Function to compute Z-array
void computeZArray(char *s, int *Z) {
    int n = strlen(s);
    int L = 0, R = 0;

    Z[0] = 0; // By definition

    for (int i = 1; i < n; i++) {
        if (i <= R) {
            Z[i] = (Z[i - L] < (R - i + 1)) ? Z[i - L] : (R - i + 1);
        } else {
            Z[i] = 0;
        }

        // Try to extend Z[i]
        while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]]) {
```

```
            Z[i]++;
        }

        // Update [L, R] if extended
        if (i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int Z[n];
    computeZArray(s, Z);

    for (int i = 0; i < n; i++) {
        printf("%d ", Z[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 3_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Dhiya is a data analyst working for a data compression company. One of her tasks is to analyze logs that are compressed using a simple format where each character is followed by the number of times it appears. For example, "A3B2" represents the string "AAABB".

Her goal is to identify whether a specific pattern exists in the original, uncompressed message and, if so, determine all starting and ending indices of the pattern's occurrence in the expanded string. To perform this efficiently, Dhiya decides to first expand the compressed string and then apply pattern matching.

Your task is to help Dhiya by building a program using Boyer-Moore pattern matching algorithm that performs this operation.

Input Format

The first line of input contains a string s consisting of characters followed by digits representing the compressed text.

The second line of input contains a string p representing the pattern string.

Output Format

The first line of output prints "Expanded Text: X" where X represents the expanded text.

The second line of output prints "Matches found (start_index end_index):" if match found.

For each match of the pattern in the expanded text, print the starting and ending indices (0-based) separated by a space (Print one pair per line).

If no match is found, print "No match found.".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: A3B2C4D1
CCC
Output: Expanded Text: AAABBCCCCD
Matches found (start_index end_index):
5 7
6 8

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define NO_OF_CHARS 256

// Build bad character table
void badCharHeuristic(char *pat, int m, int badchar[NO_OF_CHARS]) {
    for (int i = 0; i < NO_OF_CHARS; i++)
```

```
        badchar[i] = -1;
    for (int i = 0; i < m; i++)
        badchar[(unsigned char)pat[i]] = i;
}

// Boyer-Moore search
void search(char *txt, char *pat) {
    int n = strlen(txt);
    int m = strlen(pat);
    int badchar[NO_OF_CHARS];
    badCharHeuristic(pat, m, badchar);

    int s = 0; // shift of pattern
    int found = 0;

    while (s <= (n - m)) {
        int j = m - 1;

        while (j >= 0 && pat[j] == txt[s + j])
            j--;

        if (j < 0) {
            if (!found) {
                printf("Matches found (start_index end_index):\n");
                found = 1;
            }
            printf("%d %d\n", s, s + m - 1);

            s += (s + m < n) ? m - badchar[(unsigned char)txt[s + m]] : 1;
        } else {
            int shift = j - badchar[(unsigned char)txt[s + j]];
            if (shift < 1) shift = 1;
            s += shift;
        }
    }

    if (!found) {
        printf("No match found.");
    }
}

int main() {
```

```
    char compressed[1005], pattern[1005];
    scanf("%s", compressed);
    scanf("%s", pattern);

    char expanded[10005]; // enough buffer
    int idx = 0;

    for (int i = 0; compressed[i]; ) {
        char c = compressed[i++];
        int num = 0;
        while (isdigit(compressed[i])) {
            num = num * 10 + (compressed[i] - '0');
            i++;
        }
        for (int k = 0; k < num; k++) {
            expanded[idx++] = c;
        }
    }
    expanded[idx] = '\0';

    printf("Expanded Text: %s\n", expanded);
    search(expanded, pattern);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Sanya is developing a text analysis tool and wants to identify the longest palindromic sequence in a given string. A palindrome is a string that reads the same forwards and backwards.

Your task is to write a program that takes a string as input and returns the longest palindromic substring it contains. If there are multiple substrings of the same maximum length, returning any one of them is acceptable.

Input Format

The input consists of a single line containing the string S.

S may consist of uppercase and lowercase English letters.

Output Format

The output prints a single string representing the longest palindromic substring found in S.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: babad
Output: bab

Answer

```
#include <stdio.h>
#include <string.h>

// Function to expand around center
int expandAroundCenter(char *s, int left, int right) {
    int n = strlen(s);
    while (left >= 0 && right < n && s[left] == s[right]) {
        left--;
        right++;
    }
    return right - left - 1; // length of palindrome
}

int main() {
    char s[105];
    scanf("%s", s);

    int start = 0, end = 0;
    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        int len1 = expandAroundCenter(s, i, i);     // odd-length palindrome
        int len2 = expandAroundCenter(s, i, i + 1); // even-length palindrome
        int len = (len1 > len2) ? len1 : len2;
```

```
        if (len > end - start + 1) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }

    for (int i = start; i <= end; i++) {
        printf("%c", s[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rita is working on string pattern matching using the Z-Algorithm. She wants to find all starting indices where a given pattern string occurs in a text string efficiently.

Write a program that takes a text string and a pattern string as input and outputs a list of indices (0-based) where the pattern occurs. If the pattern does not occur in the text, output an empty list.

Input Format

The first line contains the text string txt.

The second line contains the pattern string pat.

Output Format

The output prints a list of integers representing the starting indices of all occurrences of the pattern in the text.

If no match is found, print [].

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abesdu
edu
Output: []

Answer

```
#include <stdio.h>
#include <string.h>

// Function to compute Z-array
void computeZArray(char *s, int *Z) {
    int n = strlen(s);
    int L = 0, R = 0;
    Z[0] = 0;

    for (int i = 1; i < n; i++) {
        if (i <= R) {
            Z[i] = (Z[i - L] < (R - i + 1)) ? Z[i - L] : (R - i + 1);
        } else {
            Z[i] = 0;
        }
        while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]]) {
            Z[i]++;
        }
        if (i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }
}

int main() {
    char txt[1005], pat[105];
    scanf("%s", txt);
    scanf("%s", pat);

    char concat[1200];
    sprintf(concat, "%s$%s", pat, txt);

    int n = strlen(concat);
    int Z[n];
```

```
    computeZArray(concat, Z);

    int patLen = strlen(pat);
    int found = 0;

    printf("[");
    for (int i = patLen + 1; i < n; i++) {
        if (Z[i] == patLen) {
            if (found) printf(", ");
            printf("%d", i - patLen - 1);
            found = 1;
        }
    }
    printf("]");

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Neha is building a plagiarism-detection feature where she must compare two text strings to find similarities. To efficiently detect overlaps between the strings, she uses the KMP (Knuth–Morris–Pratt) string matching algorithm, relying on the LPS (Longest Prefix Suffix) array as the main data structure.

Given two strings, write a program to ensure that the task is done by finding the longest common substring present in both strings. If no common substring exists, the program should clearly indicate that result.

Input Format

The first line contains a string representing the first text.

The second line contains a string representing the second text.

Output Format

The output prints the longest common substring if it exists.

If there is no common substring, print "There is no common substring".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abcdef
abc

Output: abc

Answer

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[105], s2[105];
    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n+1][m+1];
    memset(dp, 0, sizeof(dp));

    int maxLen = 0;
    int endIndex = -1; // end index in s1

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i-1] == s2[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
                if (dp[i][j] > maxLen) {
                    maxLen = dp[i][j];
                    endIndex = i - 1;
                }
            } else {
                dp[i][j] = 0;
            }
        }
```

```
    }

    if (maxLen == 0) {
        printf("There is no common substring");
    } else {
        for (int i = endIndex - maxLen + 1; i <= endIndex; i++) {
            printf("%c", s1[i]);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 3_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. The main idea behind the Knuth-Morris-Pratt algorithm is to:

Answer

Avoid redundant checking of characters that have already been matched

Status : Correct ***Marks : 1/1***

2. The Boyer-Moore algorithm uses which two rules to determine the shift amount?

Answer

Good suffix rule and bad character rule

Status : Correct ***Marks : 1/1***

vtu29890

3. The Boyer-Moore algorithm is particularly efficient when

Answer

The text contains repetitive elements

Status : Wrong ***Marks : 0/1***

4. In the KMP algorithm, if the prefix table at position ii has a value of kk, what does it imply?

Answer

The length of the longest proper prefix which is also a suffix up to position ii is kk.

Status : Correct ***Marks : 1/1***

5. The concept of prefix and suffix is used in which of the following algorithms?

Answer

Knuth-Morris-Pratt (KMP) algorithm

Status : Correct ***Marks : 1/1***

6. What is the main purpose of Manacher's Algorithm?

Answer

To find the longest palindromic substring in linear time

Status : Correct ***Marks : 1/1***

7. In the Z-algorithm, the Z-array is used to store:

Answer

The length of the longest substring starting at each position that matches the prefix

Status : Correct ***Marks : 1/1***

vtu29890

8. Which application is most efficiently solved using Z-algorithm?

Answer

Finding all occurrences of a pattern in a text

Status : Correct ***Marks : 1/1***

9. In Z-algorithm, when Z[k] is greater than 0, what does it imply?

Answer

Substring starting at k matches the prefix for length Z[k]

Status : Correct ***Marks : 1/1***

10. Which of the following best explains why Manacher's algorithm is preferable for palindrome problems over expand-around-center approaches?

Answer

Avoids recomputation of overlapping palindromes by mirroring results around the center

Status : Correct ***Marks : 1/1***

11. In bioinformatics applications, KMP is often used instead of Boyer-Moore. Why?

Answer

DNA has only 4 characters, reducing Boyer-Moore's skipping advantage

Status : Correct ***Marks : 1/1***

12. In KMP, the LPS array for pattern "ABABAC" is:

Answer

[0,0,1,2,3,0]

Status : Correct ***Marks : 1/1***

13. Which is true about Manacher's algorithm with respect to handling even-length palindromes?

Answer

Uses a modified string with separators between characters

Status : Correct ***Marks : 1/1***

14. How is Z-algorithm used for pattern search in text efficiently?

Answer

Concatenate P + "$" + T and compute Z-array

Status : Correct ***Marks : 1/1***

15. Which of the following strings has the largest number of palindromic substrings using Manacher’s algorithm?

Answer

"aaaaa"

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 3_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 39

Section 1 : Coding

1. Problem Statement

Suresh is working on a text-processing system where he needs to efficiently count how many times a smaller pattern appears inside a larger string. To achieve fast pattern matching, he uses the Z-Algorithm, with the Z-array as the main data structure.

Given multiple test cases, each containing a text string and a pattern string, write a program to ensure that the task is done by computing the number of occurrences of the pattern in the text using the Z-algorithm approach.

For example:

If S = "ababa", and P = "ab", then "ab" occurs twice in "ababa".

Input Format

The first line of input contains T, representing the number of test cases.

For each test case:

Line 1 Contains two space separated integers N and M representing the length of the text and pattern respectively.

Line 2: Contains string S of length N representing the text.

Line 3: A string P of length M representing the pattern.

Output Format

For each test case, print a single integer on a new line indicating the number of times the pattern occurs in the text.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
10 2
abababababa
ab
Output: 4

Answer

```
#include <stdio.h>
#include <string.h>

// Function to compute Z-array
void computeZArray(char *s, int *Z) {
   int n = strlen(s);
   int L = 0, R = 0;
   Z[0] = 0;

   for (int i = 1; i < n; i++) {
      if (i <= R) {
         Z[i] = (Z[i - L] < (R - i + 1)) ? Z[i - L] : (R - i + 1);
```

```
        } else {
            Z[i] = 0;
        }
        while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]]) {
            Z[i]++;
        }
        if (i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }
}

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int N, M;
        scanf("%d %d", &N, &M);

        char text[1005], pattern[105];
        scanf("%s", text);
        scanf("%s", pattern);

        // Concatenate pattern + "$" + text
        char concat[1200];
        sprintf(concat, "%s$%s", pattern, text);

        int n = strlen(concat);
        int Z[n];
        computeZArray(concat, Z);

        int patLen = strlen(pattern);
        int count = 0;

        for (int i = patLen + 1; i < n; i++) {
            if (Z[i] == patLen) {
                count++;
            }
        }
```

```
        printf("%d\n", count);  // strict format: one integer per line
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arjun is working as an NLP engineer to improve the smart assistant chatbot in his company. His team wants the chatbot to understand user emotions better, especially when users repeat or emphasize words.

They noticed that palindromic patterns (like "wow", "pop", "madam") in messages can help detect such emotions or expressions.

To help with this, Arjun is asked to write a program that finds out how many palindromic substrings are centered at each character in the user's message.

Since the message could be long, Arjun decides to use Manacher's Algorithm for fast and efficient processing.

Example:

Input

bananas

Output

1 1 3 5 3 1 1

Explanation

At the first character 'b', the only palindrome is "b" itself length 1At 'a', only "a" is a palindrome length 1At 'n', we get "ana" which is a palindrome centered at 'n' length 3At the middle 'a', we get "anana" which is a bigger palindrome length 5At the next 'n', again "ana" is a palindrome length 3At the second 'a', only "a" length 1Finally at 's', only "s" length 1

Input Format

The first line of input contains a string s, representing a message typed by a user in the chatbot conversation.

Output Format

The output prints the length of the longest palindromic substring centered at each character in the input string, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ab

Output: 1 1

Answer

```
#include <stdio.h>
#include <string.h>

void manacher(char *s) {
    int n = strlen(s);
    char t[2 * n + 3];
    int P[2 * n + 3];

    // Transform string: ^#s0#s1#...#sn-1#$
    t[0] = '^';
    int idx = 1;
    for (int i = 0; i < n; i++) {
        t[idx++] = '#';
        t[idx++] = s[i];
    }
    t[idx++] = '#';
    t[idx++] = '$';
    t[idx] = '\0';

    int m = idx;
    int center = 0, right = 0;

    for (int i = 1; i < m - 1; i++) {
```

```
        int mirror = 2 * center - i;
        if (i < right)
            P[i] = (P[mirror] < (right - i)) ? P[mirror] : (right - i);
        else
            P[i] = 0;

        while (t[i + 1 + P[i]] == t[i - 1 - P[i]])
            P[i]++;

        if (i + P[i] > right) {
            center = i;
            right = i + P[i];
        }
    }

    // Extract results for original characters
    for (int i = 0; i < n; i++) {
        int pos = 2 * (i + 1); // position of character in transformed string
        printf("%d ", P[pos]);
    }
}

int main() {
    char s[1005];
    scanf("%s", s);

    manacher(s);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a text txt [0 . . . N-1] and a pattern pat [0 . . . M-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that N > M. Use zero-based indexing while returning the indices.

Example:

Input: txt [] = “THIS IS A TEST TEXT”, pat [] = “TEST”

Output: Pattern found at index 10

Input: txt [] = “AABAACAADAABAABA”, pat [] = “AABA”

Output: Pattern found at index 0, Pattern found at index 9, Pattern found at index 12

Input Format

The first line of input consists of a string txt, in which the pattern is to be searched.

The second line of input consists of a string pat, the pattern that needs to be found in the text.

Output Format

For each occurrence of the pattern in the text, print "Pattern found at index X" in a new line, where X the indices where the pattern is found within the text string.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: THIS IS A TEST TEXT
TEST
Output: Pattern found at index 10

Answer

```
#include <stdio.h>
#include <string.h>

// Function to search for pattern in text
void search(char pat[], char txt[]) {
    int N = strlen(txt);
    int M = strlen(pat);
```

```
    for (int i = 0; i <= N - M; i++) {
        int j;
        for (j = 0; j < M; j++) {
            if (txt[i + j] != pat[j]) {
                break;
            }
        }
        if (j == M) {
            printf("Pattern found at index %d\n", i);
        }
    }
}

int main() {
    char txt[55], pat[15];
    // Read text (may contain spaces)
    fgets(txt, sizeof(txt), stdin);
    txt[strcspn(txt, "\n")] = '\0'; // remove newline

    // Read pattern
    fgets(pat, sizeof(pat), stdin);
    pat[strcspn(pat, "\n")] = '\0'; // remove newline

    search(pat, txt);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ritika is a software engineer working on an AI-based spam detection system for a popular email client. To efficiently scan email content, she uses the Boyer–Moore string matching algorithm, specifically the Bad Character Heuristic as the main data structure/technique.

Given an email message and a list of spam keywords, write a program to ensure that the task is done by detecting all keywords present in the email

(case-insensitive), reporting their first occurrence positions, and computing a spam score based on the number of detected keywords.

Input Format

The first line of input contains a string representing the email content (may contain spaces and special characters).

The second line contains an integer n , representing the number of spam keywords.

The next n lines each contain a spam keyword.

Output Format

The first line prints a string "Detected Spam Keywords:"

For each detected keyword, print "<index>: "<keyword>" at position <starting_index>" where indexing starts from 0.

The final line prints "Spam Score: x/y" where x is the number of detected keywords and y is the total number of keywords.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Buy this book now and get another free!
3
buy
now
free

Output: Detected Spam Keywords:
1: "buy" at position 0
2: "now" at position 14
3: "free" at position 34
Spam Score: 3/3

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#include <ctype.h>

#define NO_OF_CHARS 256

// Convert string to lowercase
void toLowerCase(char *s) {
    for (int i = 0; s[i]; i++) {
        s[i] = tolower((unsigned char)s[i]);
    }
}

// Build bad character table
void badCharHeuristic(char *pat, int m, int badchar[NO_OF_CHARS]) {
    for (int i = 0; i < NO_OF_CHARS; i++)
        badchar[i] = -1;
    for (int i = 0; i < m; i++)
        badchar[(unsigned char)pat[i]] = i;
}

// Boyer-Moore search (returns first occurrence index or -1)
int search(char *txt, char *pat) {
    int n = strlen(txt);
    int m = strlen(pat);
    int badchar[NO_OF_CHARS];
    badCharHeuristic(pat, m, badchar);

    int s = 0;
    while (s <= (n - m)) {
        int j = m - 1;
        while (j >= 0 && pat[j] == txt[s + j])
            j--;
        if (j < 0) {
            return s; // first occurrence
        } else {
            int shift = j - badchar[(unsigned char)txt[s + j]];
            if (shift < 1) shift = 1;
            s += shift;
        }
    }
    return -1;
}
```

```
int main() {
    char email[1005];
    fgets(email, sizeof(email), stdin);
    email[strcspn(email, "\n")] = '\0';

    int n;
    scanf("%d", &n);
    getchar(); // consume newline

    char keywords[105][105];
    for (int i = 0; i < n; i++) {
        fgets(keywords[i], sizeof(keywords[i]), stdin);
        keywords[i][strcspn(keywords[i], "\n")] = '\0';
    }

    // Lowercase email
    char emailLower[1005];
    strcpy(emailLower, email);
    toLowerCase(emailLower);

    int detected = 0;
    printf("Detected Spam Keywords:\n");

    for (int i = 0; i < n; i++) {
        char pat[105];
        strcpy(pat, keywords[i]);
        toLowerCase(pat);

        int pos = search(emailLower, pat);
        if (pos != -1) {
            detected++;
            printf("%d: \"%s\" at position %d\n", detected, keywords[i], pos);
        }
    }

    if (detected == 0) {
        printf("No spam keywords detected.\n");
    }

    printf("Spam Score: %d/%d", detected, n);

    return 0;
```

}

Status : Partially correct ***Marks : 9/10***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 4_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neo is experimenting with text transformations and decides to use a stack to reverse the sequence of words in his sentence. He provides a line of text and expects the program to utilize a stack to reorder the words so that the last word appears first and the first word appears last.

Write a program to ensure this is done correctly using a stack-based approach.

Example

Input: I am Neo

Output: Neo am I

vtu29890

Input Format

The input consists of a single line of input containing a sentence with words separated by spaces.

Output Format

The output prints a single line of output containing the words of the sentence reversed in order, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Hello World
Output: World Hello

Answer

```
#include <stdio.h>
#include <string.h>

#define SIZE 100

int main() {
    char sentence[SIZE];
    char stack[SIZE][SIZE];
    int top = -1;

    // Read input line
    fgets(sentence, SIZE, stdin);
    sentence[strcspn(sentence, "\n")] = '\0'; // remove newline

    // Split words and push onto stack
    char *word = strtok(sentence, " ");
    while (word != NULL) {
        strcpy(stack[++top], word);
        word = strtok(NULL, " ");
    }

    // Pop words and print
    while (top >= 0) {
```

```
    printf("%s", stack[top--]);
    if (top >= 0) printf(" ");
  }

  return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Give a string, remove duplicate letters from the string s such that each letter appears just once using Stack. The lexicographical order of your result must be the least of all feasible outcomes.

Input Format

The input consists of a string.

Output Format

The output displays the string in the lexicographical order with unique characters.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: bcabc

Output: abc

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
  char s[30];
  scanf("%s", s);
```

```
    int freq[26] = {0};
    bool inStack[26] = {false};
    int len = strlen(s);

    // Count frequency of each character
    for (int i = 0; i < len; i++) {
        freq[s[i] - 'a']++;
    }

    char stack[30];
    int top = -1;

    for (int i = 0; i < len; i++) {
        char c = s[i];
        freq[c - 'a']--;

        if (inStack[c - 'a']) continue;

        while (top >= 0 && stack[top] > c && freq[stack[top] - 'a'] > 0) {
            inStack[stack[top] - 'a'] = false;
            top--;
        }

        stack[++top] = c;
        inStack[c - 'a'] = true;
    }

    stack[top + 1] = '\0';
    printf("%s", stack);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Mira is learning about palindromes and wants to build a program that can check whether a given string is a palindrome using a stack-based approach. A palindrome reads the same backward as forward.

Write a program to ensure that the task is done by pushing characters to a stack and comparing them in reverse order to determine if the string is a palindrome.

Input Format

The input contains a single line containing a string with space-separated characters.

Output Format

The output consists of ,

If the string is a palindrome, print "It's a palindrome!"

Otherwise, print "Not a palindrome"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam
Output: It's a palindrome!

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char input[105];
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0'; // remove newline

    char cleaned[105];
    int len = 0;

    // Remove spaces and convert to lowercase
    for (int i = 0; input[i] != '\0'; i++) {
        if (isalpha(input[i])) {
```

```
        cleaned[len++] = tolower(input[i]);
    }
    }
    cleaned[len] = '\0';

    // Stack array
    char stack[105];
    int top = -1;

    // Push characters
    for (int i = 0; i < len; i++) {
        stack[++top] = cleaned[i];
    }

    // Pop to build reversed string
    char reversed[105];
    for (int i = 0; i < len; i++) {
        reversed[i] = stack[top--];
    }
    reversed[len] = '\0';

    // Compare
    if (strcmp(cleaned, reversed) == 0) {
        printf("It's a palindrome!");
    } else {
        printf("Not a palindrome");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Gokul is working on a program to rearrange elements in a queue using an array. Given a queue of integers, he needs to modify it by interleaving its first half with the second half. Write a program to help Gokul achieve this task.

Note: Interleaving involves merging elements from two separate halves in an alternating fashion to create a new sequence.

Example

Input

6

1 2 3 7 9 5

Output

1 7 2 9 3 5

Explanation

The interleave rearranges elements in a queue by alternating between two halves. For instance, given input [1, 2, 3, 7, 9, 5], it splits into [1, 2, 3] and [7, 9, 5], then interleaves them as [1, 7, 2, 9, 3, 5]. It achieves this by splitting the queue into two halves and then merging them interleaved.

Input Format

The first line of the input consists of an integer n, representing the number of elements in the queue.

The second line consists of n space-separated integers, representing the queue elements.

Output Format

The output prints n space-separated integers, representing the rearranged queue after interleaving its first half with the second half.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
1 2 3 7 9 5
Output: 1 7 2 9 3 5

vtu29890

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20]; // since n ≤ 10
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int half = n / 2;

    for (int i = 0; i < half; i++) {
        printf("%d %d", arr[i], arr[i + half]);
        if (i != half - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Sanjay is learning about queues in his data structures class and he wants to write a program that counts the number of distinct elements in the queue. Your task is to guide him in this.

Example

Input:

7

45 33 29 45 45 46 39

Output:

5

Explanation:

The distinct elements in the queue are 45, 33, 29, 46, 39. So, the count is 5.

Input Format

The first line of input consists of an integer M, representing the number of elements in the queue.

The second line consists of M space-separated integers, representing the queue elements.

Output Format

The output prints an integer, representing the number of distinct elements in the queue.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
30 24 43 31 30

Output: 4

Answer

```
#include <stdio.h>

int main() {
    int M;
    scanf("%d", &M);

    int arr[100];
    for (int i = 0; i < M; i++) {
        scanf("%d", &arr[i]);
    }

    int distinctCount = 0;

    for (int i = 0; i < M; i++) {
```

```
        int isDuplicate = 0;
        for (int j = 0; j < i; j++) {
            if (arr[i] == arr[j]) {
                isDuplicate = 1;
                break;
            }
        }
        if (!isDuplicate) {
            distinctCount++;
        }
    }

    printf("%d", distinctCount);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 4_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

A financial analytics company is developing a tool to analyze stock price trends. The tool needs to identify the Next Greater Price for each stock price in a given list of daily prices. For a given stock price, the Next Greater Price is defined as the first price appearing after it in the list that is greater. If no such price exists, the result should be -1.

Note: The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1.

Examples:

Input Format

The first line of input consists of an integer n, representing the size of the array.

The second line consists of n space-separated stack of integers.

Output Format

The output prints "Next Greater Elements: " followed by an array of integers, representing the next greater elements in the stack.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
4 5 2 25

Output: Next Greater Elements: 5 25 25 -1

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[15], result[15];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        result[i] = -1; // default
    }

    int stack[15], top = -1;

    for (int i = 0; i < n; i++) {
        while (top >= 0 && arr[i] > arr[stack[top]]) {
            result[stack[top]] = arr[i];
            top--;
        }
        stack[++top] = i;
    }
```

```
    printf("Next Greater Elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d", result[i]);
        if (i != n - 1) printf(" ");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Riya is working on a text-cleaning utility where certain unwanted patterns must be removed from user input. She uses a stack data structure to process the string character by character.

Given a string containing alphabets, digits, and special characters, write a program to ensure that the task is done by removing every digit along with the closest non-digit character immediately to its left. The remaining characters should be preserved in their original order and printed as the final cleaned string.

Example:

Input:

s = "cb34"

Output:

""

Explanation:

Start with an empty stack.

Push c stack: ['c']

Push b stack: ['c','b']

Encounter 3 pop 'b' stack: ['c']

Encounter 4 pop 'c' stack: []

Result: empty string

Input Format

The input consists of a string s without spaces, consisting of letters, digits, and special characters.

Output Format

The output prints the resulting string after performing the required removals.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abc

Output: abc

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
  char s[105];
  scanf("%s", s);

  char stack[105];
  int top = -1;

  for (int i = 0; s[i] != '\0'; i++) {
    if (isdigit(s[i])) {
      if (top >= 0) {
        top--; // remove closest non-digit to the left
      }
      // skip pushing the digit itself
    } else {
      stack[++top] = s[i];
    }
```

```
    }
    stack[top + 1] = '\0';
    printf("%s", stack);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kavya is developing a real-time analytics system that processes a continuous stream of numerical data. To efficiently track trends, she uses a Queue data structure to maintain incoming values and compute statistics incrementally.

Given a sequence of integers arriving one by one, write a program to ensure that the task is done by calculating and printing the moving average of all elements received so far after each new input. The average should be displayed up to two decimal places.

Example

Input

3

1 2 3

Output

1.00 1.50 2.00

Explanation

The moving average of the first element is 1.00.

The next elements are 1 and 2, the moving average is (1 + 2) / 2 = 1.50.

The next elements are 1, 2, and 3, the moving average is (1 + 2 + 3) / 3 = 2.00.

Input Format

The first line contains an integer n, representing the number of elements in the data stream.

The second line contains n space-separated integers representing the incoming values.

Output Format

The output prints n floating-point numbers, each representing the moving average after processing the corresponding element.

Each value should be printed with two decimal places, separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: 1.00 1.50 2.00

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char s[105];
    scanf("%s", s);

    char stack[105];
    int top = -1;

    for (int i = 0; s[i] != '\0'; i++) {
        if (isdigit(s[i])) {
            if (top >= 0) {
                top--; // remove closest non-digit to the left
            }
            // skip pushing the digit itself
```

```
    } else {
        stack[++top] = s[i];
    }
  }

  stack[top + 1] = '\0';
  printf("%s", stack);

  return 0;
}
```

Status : Wrong **Marks : 0/10**

4. Problem Statement

In a small business, Jane is tasked with organizing customer feedback scores for a new product. Each customer has rated the product, and Jane needs to arrange these ratings in a specific order to analyze the feedback more effectively.

Specifically, she wants to rearrange the ratings such that the most frequent ratings appear first. If two ratings have the same frequency, the smaller rating should come first. To accomplish this, assist Jane with a program to process a queue of customer ratings and rearrange them based on their frequencies.

Input Format

The first line contains an integer n, which represents the number of customer ratings.

The second line contains n integers, each representing a customer rating.

Output Format

The output displays the rearranged ratings in the order of their frequency (most frequent first).

If frequencies are the same, the smaller rating should come first.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
2 3 2 3 2 3 2

Output: 2 2 2 2 3 3 3

Answer

```
#include <stdio.h>

typedef struct {
    int value;
    int freq;
} Rating;

int main() {
    int n;
    scanf("%d", &n);

    int arr[25];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    Rating ratings[100];
    int count = 0;

    // Count frequencies
    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < count; j++) {
            if (ratings[j].value == arr[i]) {
                ratings[j].freq++;
                found = 1;
                break;
            }
        }
        if (!found) {
            ratings[count].value = arr[i];
            ratings[count].freq = 1;
```

```
            count++;
        }
    }

    // Sort by frequency (desc), then value (asc)
    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (ratings[i].freq < ratings[j].freq ||
                (ratings[i].freq == ratings[j].freq && ratings[i].value > ratings[j].value)) {
                Rating temp = ratings[i];
                ratings[i] = ratings[j];
                ratings[j] = temp;
            }
        }
    }

    // Print rearranged ratings
    for (int i = 0; i < count; i++) {
        for (int j = 0; j < ratings[i].freq; j++) {
            printf("%d", ratings[i].value);
            if (!(i == count - 1 && j == ratings[i].freq - 1)) {
                printf(" ");
            }
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. The user performs the following operations on a stack of size 5 then, which of the following is the correct statement for the 1stack?

push(1);
pop();
push(2);
push(3);
pop();
push(2);
pop();
pop();
push(4);
pop();
pop();
push(5);

vtu29890

Answer

Underflow occurs

Status : Correct ***Marks : 1/1***

2. If the sequence of operations - push (1), push (2), pop, push (1), push (2), pop, pop, pop, push (2), pop are performed on a stack, the sequence of popped out values

Answer

2,2,1,1,2

Status : Correct ***Marks : 1/1***

3. What is the result of the following operation?

Top (Push (S, X))

Answer

X

Status : Correct ***Marks : 1/1***

4. Stack A has the entries a, b, c (with a on top). Stack B is empty. An entry popped out of stack A can be printed immediately or pushed to stack B. An entry popped out of the stack B can only be printed. In this arrangement, which of the following permutations of a, b, c are not possible?

Answer

c a b

Status : Correct ***Marks : 1/1***

5. Pushing an element into the stack already has five elements. The stack size is 5, then the stack becomes

Answer

Overflow

Status : Correct ***Marks : 1/1***

6. In a Stack, if a user tries to remove an Element from Empty stack it is called

Answer

Underflow

Status : Correct ***Marks : 1/1***

7. The user performs the following operations on the stack of size 5 then at the end of the last operation, the total number of elements present in the stack is

```
push(1);
pop();
push(2);
push(3);
pop();
push(4);
pop();
pop();
push(5);
```

Answer

1

Status : Correct ***Marks : 1/1***

8. What will be the output after performing these sequence of operations

```
push(20);
push(4);
top();
pop();
```

```
pop();
pop();
push(5);
top();
```

Answer

Stack Underflow

Status : Correct ***Marks : 1/1***

9. A normal queue, if implemented using an array of size MAX_SIZE, gets full when _____.

Answer

rear = MAX_SIZE – 1

Status : Correct ***Marks : 1/1***

10. If the elements 'A', 'B', 'C', and 'D' are placed in a queue and are deleted one at a time, in what order will they be removed?

Answer

ABCD

Status : Correct ***Marks : 1/1***

11. After performing this set of operations, what does the final queue contain?

```
InsertFront(10);
InsertFront(20);
InsertRear(30);
DeleteFront();
InsertRear(40);
InsertRear(10);
DeleteRear();
InsertRear(15);
display();
```

Answer

10 30 40 15

Status : Correct ***Marks : 1/1***

12. What is the purpose of defining the constant SIZE in a queue implementation using an array?

Answer

To limit the maximum number of elements in the queue

Status : Correct ***Marks : 1/1***

13. What does the condition front = NULL indicate in inserting an element into a queue?

Answer

The queue is empty

Status : Correct ***Marks : 1/1***

14. Which of the following is a valid condition for a successful dequeue operation in the array implementation of a queue?

Answer

None of the mentioned options

Status : Correct ***Marks : 1/1***

15. The essential condition that is checked before insertion in a queue is?

Answer

Overflow

Status : Correct ***Marks : 1/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 4_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Rohit is learning combinatorics and wants to generate all possible selections from a given set of numbers. To systematically explore every combination, he uses a Stack data structure along with a bitwise technique.

Given a set of integers, write a program to ensure that the task is done by generating and printing all possible subsets (the power set) of the given set, including the empty set. Each subset must be printed in set notation.

Example

Input:

3

vtu29890

1 2 3

Output:

{}

{1}

{2}

{1, 2}

{3}

{1, 3}

{2, 3}

{1, 2, 3}

Input Format

The first line of input consists of an integer n, representing the number of elements in the set.

The second line consists of n space-separated integers representing the elements of the set.

Output Format

The output prints all possible subsets of the given set.

Each subset should be printed on a new line in the format {element1, element2, ...}.

The empty subset should be printed as {}.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: {}

vtu29890

{1}
{2}
{1, 2}
{3}
{1, 3}
{2, 3}
{1, 2, 3}

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[15];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int total = 1 << n; // 2^n subsets

    for (int mask = 0; mask < total; mask++) {
        int stack[15], top = -1;

        for (int j = 0; j < n; j++) {
            if (mask & (1 << j)) {
                stack[++top] = arr[j];
            }
        }

        if (top == -1) {
            printf("{}\n");
        } else {
            printf("{");
            for (int k = 0; k <= top; k++) {
                printf("%d", stack[k]);
                if (k != top) printf(", ");
            }
            printf("}\n");
        }
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given an array of integers, the task is to find the maximum absolute difference between the nearest left and the right smaller element of every element in the array.

Note: If there is no smaller element on right side or left side of any element then we take zero as the smaller element. For example for the leftmost element, the nearest smaller element on the left side is considered as 0. Similarly, for rightmost elements, the smaller element on the right side is considered as 0.

Examples:

Input: arr[] = {2, 1, 8}

Output: 1

Left smaller LS[] {0, 0, 1}

Right smaller RS[] {1, 0, 0}

Maximum Diff of abs(LS[i] - RS[i]) = 1

Input: arr[] = {2, 4, 8, 7, 7, 9, 3}

Output: 4

Left smaller LS[] = {0, 2, 4, 4, 4, 7, 2}

Right smaller RS[] = {0, 3, 7, 3, 3, 3, 0}

Maximum Diff of abs(LS[i] - RS[i]) = 7 - 3 = 4

Input: arr[] = {5, 1, 9, 2, 5, 1, 7}

Output: 1

Input Format

The first line contains an integer n, representing the size of the array.

The second line contains n space-separated integers, representing the array elements.

Output Format

The output prints a single integer, representing the maximum absolute difference between the left and right smaller elements for any element in the array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
2 1 8

Output: 1

Answer

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n;
   scanf("%d", &n);

   int arr[15];
   for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }

   int LS[15], RS[15];
   int stack[15], top;

   // Compute Left Smaller
   top = -1;
   for (int i = 0; i < n; i++) {
      while (top >= 0 && stack[top] >= arr[i]) {
         top--;
```

```
        }
        if (top == -1) LS[i] = 0;
        else LS[i] = stack[top];
        stack[++top] = arr[i];
    }

    // Compute Right Smaller
    top = -1;
    for (int i = n - 1; i >= 0; i--) {
        while (top >= 0 && stack[top] >= arr[i]) {
            top--;
        }
        if (top == -1) RS[i] = 0;
        else RS[i] = stack[top];
        stack[++top] = arr[i];
    }

    // Find maximum absolute difference
    int maxDiff = 0;
    for (int i = 0; i < n; i++) {
        int diff = abs(LS[i] - RS[i]);
        if (diff > maxDiff) maxDiff = diff;
    }

    printf("%d", maxDiff);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

A logistics company uses a queue data structure to manage the order of processing packages at their sorting center. Each package is identified by a unique code represented as an integer.

Due to a system update, the company needs to reverse the order in which the packages are processed without changing the sequence of processing for any new packages that arrive after the reversal.

The task is to implement a system that enqueues package codes, prints the original processing order, reverse the queue using a stack, and then prints the reversed processing order.

Input Format

The first line contains an integer capacity, the number of packages initially in the queue.

The second line contains capacity space-separated integers representing the package codes.

Output Format

The first line prints "Original queue:" followed by the package codes in their original order.

The second line prints "Reversed queue:" followed by the package codes in their reversed order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
Output: Original queue: 1 2 3 4 5
Reversed queue: 5 4 3 2 1

Answer

```
#include <stdio.h>

int main() {
   int capacity;
   scanf("%d", &capacity);

   int queue[105];
   for (int i = 0; i < capacity; i++) {
      scanf("%d", &queue[i]);
   }
```

```
    // Print original queue
    printf("Original queue: ");
    for (int i = 0; i < capacity; i++) {
        printf("%d", queue[i]);
        if (i != capacity - 1) printf(" ");
    }
    printf("\n");

    // Use stack to reverse
    int stack[105], top = -1;
    for (int i = 0; i < capacity; i++) {
        stack[++top] = queue[i];
    }

    // Pop back into queue
    for (int i = 0; i < capacity; i++) {
        queue[i] = stack[top--];
    }

    // Print reversed queue
    printf("Reversed queue: ");
    for (int i = 0; i < capacity; i++) {
        printf("%d", queue[i]);
        if (i != capacity - 1) printf(" ");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Alex is working on a customer support system that handles a queue of incoming service requests. The system needs to manage the requests efficiently by enqueue, ensuring that requests are processed in the order they arrive.

To improve processing efficiency, Alex wants to implement a feature that

allows moving the last request to the front of the queue when necessary. This operation helps prioritize the most recent request for urgent handling.

Your task is to implement this feature and ensure that the queue is updated accordingly.

Input Format

The first line of input contains a single integer n, representing the number of service requests.

The second line contains n integers, each representing a service request ID, separated by spaces.

Output Format

The output displays the queue after moving the last request to the front with request IDs separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
62 85 47 98

Output: 98 62 85 47

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[55];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Move last element to front
    int last = arr[n - 1];
```

```
    for (int i = n - 1; i > 0; i--) {
        arr[i] = arr[i - 1];
    }
    arr[0] = last;

    // Print updated queue
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i != n - 1) printf(" ");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 5_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Arun is tasked with implementing a program to reverse a singly linked list. The program should populate the list elements by inserting them at the beginning, traverse the list and reverse the order of its elements.

Since Arun is new to programming, you have to guide him through the task.

Input Format

The first line of input consists of an integer N, representing the linked list size.

The second line consists of N space-separated list elements.

Output Format

The first line of output prints the list elements after inserting them at the beginning.

The second line prints the reversed linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
23 85 47 62 31

Output: 31 62 47 85 23
23 85 47 62 31

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* head = NULL;

// Insert at beginning
void insertAtBeginning(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = head;
   head = newNode;
}

// Reverse linked list
struct Node* reverse(struct Node* head) {
   struct Node* prev = NULL;
   struct Node* current = head;
   struct Node* next;

   while (current != NULL) {
      next = current->next;
      current->next = prev;
```

```
        prev = current;
        current = next;
    }
    return prev;
}

// Print list
void printList(struct Node* node) {
    while (node != NULL) {
        printf("%d ", node->data);
        node = node->next;
    }
}

int main() {
    int n, val;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertAtBeginning(val);
    }

    // First output
    printList(head);
    printf(" ");

    // Reverse and print
    struct Node* reversedHead = reverse(head);
    printList(reversedHead);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

In a computer science course project, students are required to write a program to insert values at the end of a singly linked list and remove the last node from the list. The program should traverse the list and delete the

last node, adjusting pointers accordingly.

Assist students in the task.

Input Format

The first line of input consists of an integer N, representing the list size.

The second line consists of N space-separated integers, representing the list elements.

Output Format

The output prints space-separated integers, representing the list after the deletion of the last element.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
54 25 14 38 55 73 95 82
Output: 54 25 14 38 55 73 95

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}

void insertEnd(struct Node** head, int data) {
```

```
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

void deleteLast(struct Node** head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node* temp = *head;
    while (temp->next->next != NULL)
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    int n, val;
    struct Node* head = NULL;

    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertEnd(&head, val);
    }
```

```
    deleteLast(&head);
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Dev is tasked with creating a program that efficiently finds the middle element of a linked list. The program should take user input to populate the linked list by inserting each element into the front of the list and then determining the middle element.

Assist Dev, as he needs to ensure that the middle element is accurately identified from the constructed singly linked list:

If it's an odd-length linked list, return the middle element.If it's an even-length linked list, return the second middle element of the two elements.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The first line of output displays the linked list after inserting elements at the front.

The second line displays "Middle Element: " followed by the middle element of the linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50

Output: 50 40 30 20 10
Middle Element: 30

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}

void insertFront(struct Node** head, int data) {
   struct Node* newNode = createNode(data);
   newNode->next = *head;
   *head = newNode;
}

void printList(struct Node* head) {
   struct Node* temp = head;
   while (temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }
   printf("\n");
}

int findMiddle(struct Node* head) {
   struct Node* slow = head;
   struct Node* fast = head;
```

```
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}

int main() {
    int n, val;
    struct Node* head = NULL;

    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertFront(&head, val);
    }

    printList(head);
    printf("Middle Element: %d\n", findMiddle(head));

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Saran is tasked with implementing a doubly linked list in data structures, allowing users to append elements to the end of the list, and remove the last element. He needs to ensure efficient memory management in the implementation.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer n, representing the size of the list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The output displays integers, representing the list after removing the last element, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
48 22 5 62 87

Output: 48 22 5 62

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for doubly linked list
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}

// Append node at the end
void append(struct Node** head, struct Node** tail, int val) {
   struct Node* newNode = createNode(val);
   if (*head == NULL) {
      *head = *tail = newNode;
   } else {
      (*tail)->next = newNode;
```

```
        newNode->prev = *tail;
        *tail = newNode;
    }
}

// Remove last node
void removeLast(struct Node** head, struct Node** tail) {
    if (*tail == NULL) return;
    struct Node* temp = *tail;
    *tail = (*tail)->prev;
    if (*tail) {
        (*tail)->next = NULL;
    } else {
        *head = NULL;
    }
    free(temp);
}

// Display list
void display(struct Node* head) {
    struct Node* current = head;
    while (current) {
        printf("%d", current->data);
        if (current->next) printf(" ");
        current = current->next;
    }
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, &tail, val);
    }

    removeLast(&head, &tail);
    display(head);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement:

Ravi is building a simple application to manage a doubly linked list of integers. He wants to insert new integers into the list and delete a node at a specific position. After deleting it, he needs to display the updated list.

Write a program to ensure that the task is done by performing insertion, deletion at a given position, and then printing the modified list.

Input Format

The first line contains an integer n, representing the number of elements to be inserted into the list.

The second line contains n space-separated integers, representing the values to insert.

The third line contains an integer pos, representing the position of the node to be deleted from the list.

Output Format

If the given position is valid, print "Node at position X with value Y deleted successfully".

If the position is invalid, print "Position X not found in the list."

In both cases, print

"Data present in the list:"

followed by the updated list elements on the next line, space separated."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
4
Output: Node at position 4 with value 40 deleted successfully.
Data present in the list:
10 20 30 50

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}

// Append node at the end
void append(struct Node** head, struct Node** tail, int val) {
   struct Node* newNode = createNode(val);
   if (*head == NULL) {
      *head = *tail = newNode;
   } else {
```

```
        (*tail)->next = newNode;
        newNode->prev = *tail;
        *tail = newNode;
    }
}

// Delete node at given position
int deleteAtPosition(struct Node** head, struct Node** tail, int pos, int*
deletedVal) {
    if (*head == NULL) return 0;

    struct Node* current = *head;
    int count = 1;

    while (current && count < pos) {
        current = current->next;
        count++;
    }

    if (!current) return 0; // Position not found

    *deletedVal = current->data;

    if (current->prev) current->prev->next = current->next;
    else *head = current->next;

    if (current->next) current->next->prev = current->prev;
    else *tail = current->prev;

    free(current);
    return 1;
}

// Display list
void display(struct Node* head) {
    struct Node* current = head;
    while (current) {
        printf("%d", current->data);
        if (current->next) printf(" ");
        current = current->next;
    }
    printf("\n");
```

```
}

int main() {
    int n, val, pos;
    scanf("%d", &n);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, &tail, val);
    }

    scanf("%d", &pos);

    int deletedVal;
    if (deleteAtPosition(&head, &tail, pos, &deletedVal)) {
        printf("Node at position %d with value %d deleted successfully.\n", pos,
deletedVal);
    } else {
        printf("Position %d not found in the list.\n", pos);
    }

    printf("Data present in the list:\n");
    display(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given a singly linked list and two integers M and N, implement a program to traverse the linked list such that you retain every M node and then delete the next N nodes.

Continue this process until the end of the linked list.

Input Format

The first line consists of two integers M and N, separated by a space, representing the values to retain and delete.

The second line consists of a series of integers separated by spaces, representing the values of the linked list nodes.

The last input is -1 which will terminate the inputs.

vtu29890

Output Format

The output prints the linked list after applying the process of retaining every M node and deleting the next N nodes represented as a series of integers separated by "->".

Sample Test Case

Input: 2 2
1 2 3 4 5 6 7 8 -1

Output: 1->2->5->6

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   return newNode;
}

// Function to append node at the end
void append(struct Node** head, int val) {
   struct Node* newNode = createNode(val);
   if (*head == NULL) {
      *head = newNode;
   } else {
      struct Node* temp = *head;
      while (temp->next) temp = temp->next;
      temp->next = newNode;
   }
}

// Function to retain M nodes and delete N nodes
```

```
struct Node* retainAndDelete(struct Node* head, int M, int N) {
    struct Node* current = head;
    struct Node* temp;
    while (current) {
        // Retain M nodes
        for (int i = 1; i < M && current; i++) {
            current = current->next;
        }
        if (!current) break;

        // Delete next N nodes
        temp = current->next;
        for (int i = 0; i < N && temp; i++) {
            struct Node* del = temp;
            temp = temp->next;
            free(del);
        }
        current->next = temp;
        current = temp;
    }
    return head;
}

// Function to display linked list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d", temp->data);
        if (temp->next) printf("->");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int M, N, val;
    scanf("%d %d", &M, &N);

    struct Node* head = NULL;

    while (1) {
        scanf("%d", &val);
```

```
        if (val == -1) break;
        append(&head, val);
    }

    head = retainAndDelete(head, M, N);
    display(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are working on a text processing tool and need to implement a feature that allows users to insert characters at a specific position.

Implement a program that takes user inputs to create a singly linked list of characters and inserts a new character after a given index in the list.

Input Format

The first line of input consists of an integer N, representing the number of characters in the linked list.

The second line consists of a sequence of N characters, representing the linked list.

The third line consists of an integer index, representing the index(0-based) after which the new character node needs to be inserted.

The fourth line consists of a character value representing the character to be inserted after the given index.

Output Format

If the provided index is out of bounds (larger than the list size):

1. The first line prints "Invalid index".
2. The second line prints "Updated list: " followed by the unchanged linked list values.

Otherwise, prints "Updated list: " followed by the updated space separated linked list after inserting the new character after the given index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
a b c d e
2
X
Output: Updated list: a b c X d e

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   char data;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(char val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   return newNode;
}

// Append node at the end
void append(struct Node** head, char val) {
   struct Node* newNode = createNode(val);
   if (*head == NULL) {
      *head = newNode;
   } else {
      struct Node* temp = *head;
```

```
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }
}

// Insert after given index
int insertAfterIndex(struct Node* head, int index, char val) {
    struct Node* temp = head;
    int count = 0;

    while (temp && count < index) {
        temp = temp->next;
        count++;
    }

    if (!temp) return 0; // Invalid index

    struct Node* newNode = createNode(val);
    newNode->next = temp->next;
    temp->next = newNode;
    return 1;
}

// Display linked list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%c", temp->data);
        if (temp->next) printf(" ");
        temp = temp->next;
    }
    printf(" \n"); // space before newline to match sample output
}

int main() {
    int N, index;
    char val;
    scanf("%d", &N);

    struct Node* head = NULL;
    for (int i = 0; i < N; i++) {
        char ch;
```

```
        scanf(" %c", &ch); // space before %c to skip whitespace
        append(&head, ch);
    }

    scanf("%d", &index);
    scanf(" %c", &val);

    if (insertAfterIndex(head, index, val)) {
        printf("Updated list: ");
        display(head);
    } else {
        printf("Invalid index\n");
        printf("Updated list: ");
        display(head);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Imagine Anu is tasked with finding the middle element of a doubly linked list. Given a doubly linked list where each node contains an integer value and is inserted at the end, implement a program to find the middle element of the list. If the number of nodes is even, return the middle element pair.

Input Format

The first line of input consists of an integer N, representing the number of nodes in the doubly linked list.

The second line consists of N space-separated integers, representing the values of the nodes in the doubly linked list.

Output Format

The first line of output prints the space-separated elements of the doubly linked list.

The second line prints the middle element(s) of the doubly linked list, depending

on whether the number of nodes is odd or even.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50

Output: 10 20 30 40 50
30

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->prev = NULL;
   newNode->next = NULL;
   return newNode;
}

// Append node at the end
void append(struct Node** head, struct Node** tail, int val) {
   struct Node* newNode = createNode(val);
   if (*head == NULL) {
      *head = *tail = newNode;
   } else {
      (*tail)->next = newNode;
      newNode->prev = *tail;
      *tail = newNode;
```

```
    }
}

// Display the list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d", temp->data);
        if (temp->next) printf(" ");
        temp = temp->next;
    }
    printf(" \n"); // trailing space before newline to match sample output
}

// Find and print middle element(s)
void printMiddle(struct Node* head, int n) {
    struct Node* temp = head;
    int mid = n / 2;

    if (n % 2 == 1) {
        // Odd case: single middle
        for (int i = 0; i < mid; i++) temp = temp->next;
        printf("%d\n", temp->data);
    } else {
        // Even case: two middle elements
        for (int i = 0; i < mid - 1; i++) temp = temp->next;
        printf("%d %d\n", temp->data, temp->next->data);
    }
}

int main() {
    int N, val;
    scanf("%d", &N);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < N; i++) {
        scanf("%d", &val);
        append(&head, &tail, val);
    }
```

```
    display(head);
    printMiddle(head, N);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Pranav wants to clockwise rotate a doubly linked list by a specified number of positions. He needs your help to implement a program to achieve this. Given a doubly linked list and an integer representing the number of positions to rotate, write a program to rotate the list clockwise.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated linked list elements.

The third line consists of an integer k, representing the number of positions to rotate the list.

Output Format

The output displays the elements of the doubly linked list after rotating it by k positions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
1
Output: 5 1 2 3 4

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

// Append node at the end
void append(struct Node** head, struct Node** tail, int val) {
    struct Node* newNode = createNode(val);
    if (*head == NULL) {
        *head = *tail = newNode;
    } else {
        (*tail)->next = newNode;
        newNode->prev = *tail;
        *tail = newNode;
    }
}

// Rotate list clockwise by k positions
struct Node* rotateClockwise(struct Node* head, struct Node* tail, int n, int k) {
    if (k == 0 || k >= n) return head;

    // Find the new head: (n - k)th node
    struct Node* newHead = head;
    for (int i = 0; i < n - k; i++) {
        newHead = newHead->next;
    }

    struct Node* newTail = newHead->prev;
```

```
    // Break the list at newHead
    newTail->next = NULL;
    newHead->prev = NULL;

    // Connect old tail to old head
    tail->next = head;
    head->prev = tail;

    return newHead;
}

// Display list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d", temp->data);
        if (temp->next) printf(" ");
        temp = temp->next;
    }
    printf(" \n"); // trailing space before newline
}

int main() {
    int n, k, val;
    scanf("%d", &n);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, &tail, val);
    }

    scanf("%d", &k);

    head = rotateClockwise(head, tail, n, k);
    display(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. After adding a new node at the end of a linked list, what does the current last node become?

Answer

The second last node

Status : Correct ***Marks : 1/1***

2. When inserting a new element at the beginning of a linked list, what happens to the current head of the list?

Answer

It becomes the second element

Status : Correct ***Marks : 1/1***

vtu29890

3. Which of the following are true about a singly linked list?

Answer

It can grow dynamically during runtime.
The last node's "next" pointer points to NULL in a non-circular list.

Status : Correct ***Marks : 1/1***

4. When inserting a new node into a singly linked list, which of the following are the possible cases for insertion?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

5. Given the linked list: 5 -> 10 -> 15 -> 20 -> 25 -> NULL. What will be the output of traversing the list and printing each node's data?

Answer

5 10 15 20 25

Status : Correct ***Marks : 1/1***

6. Which of the following is the process of finding information in a linked list?

Answer

Start from the head of the list and traverse sequentially until finding (or not finding) the node

Status : Correct ***Marks : 1/1***

7. Which of the following actions needs to be taken when the last item in a linked list is deleted?

Answer

Change the reference of the second last item to NULL reference

Status : Correct ***Marks : 1/1***

8. Which pointer helps in traversing a doubly linked list in reverse order?

Answer

prev

Status : Correct ***Marks : 1/1***

9. Which of the following is true about the last node in a doubly linked list?

Answer

Its next pointer is NULL

Status : Correct ***Marks : 1/1***

10. What will be the effect of setting the prev pointer of a node to NULL in a doubly linked list?

Answer

The node will become the new head

Status : Correct ***Marks : 1/1***

11. How do you delete a node from the middle of a doubly linked list?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

12. What happens if we insert a node at the beginning of a doubly linked list?

Answer

vtu29890

The previous pointer of the new node is NULL

Status : Correct ***Marks : 1/1***

13. How many pointers does a node in a doubly linked list have?

Answer

2

Status : Correct ***Marks : 1/1***

14. In deletion from the end of a doubly linked list, which pointer needs to be updated?

Answer

prev->next = NULL

Status : Correct ***Marks : 1/1***

15. Which of the following is true about a Doubly Linked List?

Answer

Last node's next pointer is always NULL

Status : Correct ***Marks : 1/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 5_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Akash wants to write a program that populates values in a singly linked list by inserting each element into the front of the list. After insertion, the program should delete a node immediately after a node with a specific value X.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer N, representing the number of elements.

The second line consists of N space-separated integers, representing the elements of the linked list.

vtu29890

The third line consists of an integer X, representing the value after which the node has to be deleted.

Output Format

The output consists of two lines:

The first line shows the original linked list in the format: Original Linked List: <elements> -> NULL.

The second line shows the modified linked list after deleting the node following the node with value X: After deleting node after <X>: <modified_elements> -> NULL.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
15 28 36 43
36
Output: Original Linked List: 43 -> 36 -> 28 -> 15 -> NULL
After deleting node after 36: 43 -> 36 -> 15 -> NULL

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   return newNode;
}
```

```
// Insert at front
void insertFront(struct Node** head, int val) {
    struct Node* newNode = createNode(val);
    newNode->next = *head;
    *head = newNode;
}

// Delete node immediately after node with value X
void deleteAfterX(struct Node* head, int X) {
    struct Node* temp = head;
    while (temp && temp->data != X) {
        temp = temp->next;
    }
    if (temp && temp->next) {
        struct Node* del = temp->next;
        temp->next = del->next;
        free(del);
    }
}

// Display linked list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int N, val, X;
    scanf("%d", &N);

    struct Node* head = NULL;

    for (int i = 0; i < N; i++) {
        scanf("%d", &val);
        insertFront(&head, val); // insert at front
    }
```

```
    scanf("%d", &X);

    printf("Original Linked List: ");
    display(head);

    deleteAfterX(head, X);

    printf("After deleting node after %d: ", X);
    display(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Emma is managing her daily to-do list. Each task is represented by an integer ID. Due to a change in her schedule, Emma needs to update her to-do list by removing the first task, another task from the end of the list, and a third task from a specific position within the list.

Your task is to help Emma implement a function to perform these deletions and manage her to-do list efficiently.

Input Format

The first line of input contains an integer T, representing the number of tasks currently in the list.

The second line contains T integers, each representing the ID of a task.

The third line contains an integer p, representing the position (0-based index) of the task to be removed.

Output Format

The first line of output prints the linked list after deleting the first node, with each element separated by a space.

The second line of output prints the linked list after deleting the last node, with each element separated by a space.

The third line of output prints the linked list after deleting the node at the specified position, with each element separated by a space

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
11 22 33 44 55
2
Output: 22 33 44 55
22 33 44
22 33

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* next;
};

// Create a new node
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   return newNode;
}

// Append node at the end
void append(struct Node** head, int val) {
   struct Node* newNode = createNode(val);
   if (*head == NULL) {
      *head = newNode;
   } else {
      struct Node* temp = *head;
```

```
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }
}

// Delete first node
void deleteFirst(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

// Delete last node
void deleteLast(struct Node** head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node* temp = *head;
    while (temp->next->next) temp = temp->next;
    free(temp->next);
    temp->next = NULL;
}

// Delete node at position p (0-based)
void deleteAtPosition(struct Node** head, int p) {
    if (*head == NULL) return;
    if (p == 0) {
        deleteFirst(head);
        return;
    }
    struct Node* temp = *head;
    for (int i = 0; temp != NULL && i < p - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) return;
    struct Node* del = temp->next;
    temp->next = del->next;
    free(del);
```

```
}

// Display linked list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d", temp->data);
        if (temp->next) printf(" ");
        temp = temp->next;
    }
    printf(" \n"); // trailing space before newline
}

int main() {
    int T, val, p;
    scanf("%d", &T);

    struct Node* head = NULL;
    for (int i = 0; i < T; i++) {
        scanf("%d", &val);
        append(&head, val);
    }

    scanf("%d", &p);

    // Delete first node
    deleteFirst(&head);
    display(head);

    // Delete last node
    deleteLast(&head);
    display(head);

    // Delete node at position p
    deleteAtPosition(&head, p);
    display(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Emma is working with a doubly linked list and needs a program to delete a specific value from the list. Write a program that reads a list of integers, constructs a doubly linked list, and then deletes a given value if it exists. The program should print appropriate messages if the value is found and deleted or if it is not found in the list.

Input Format

The first line contains an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the elements of the doubly linked list.

The third line contains an integer x, representing the value to be deleted from the list.

Output Format

The first line prints the initial doubly linked list, with elements separated by a space.

The second line prints "Value x deleted from the list." on a new line if x is found and deleted, If x is not found, print "Value x not found in the list." on a new line.

The third line prints the updated doubly linked list on a new line after deletion, with elements separated by a space,

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
20

Output: 10 20 30
Value 20 deleted from the list.
10 30

vtu29890

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

// Append node at the end
void append(struct Node** head, struct Node** tail, int val) {
    struct Node* newNode = createNode(val);
    if (*head == NULL) {
        *head = *tail = newNode;
    } else {
        (*tail)->next = newNode;
        newNode->prev = *tail;
        *tail = newNode;
    }
}

// Delete node with given value
int deleteValue(struct Node** head, struct Node** tail, int x) {
    struct Node* temp = *head;
    while (temp && temp->data != x) {
        temp = temp->next;
    }
    if (!temp) return 0; // value not found

    if (temp->prev) temp->prev->next = temp->next;
    else *head = temp->next;
```

```
    if (temp->next) temp->next->prev = temp->prev;
    else *tail = temp->prev;

    free(temp);
    return 1;
}

// Display list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d", temp->data);
        if (temp->next) printf(" ");
        temp = temp->next;
    }
    printf(" \n"); // trailing space before newline
}

int main() {
    int n, val, x;
    scanf("%d", &n);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, &tail, val);
    }

    scanf("%d", &x);

    // Print initial list
    display(head);

    // Delete value
    if (deleteValue(&head, &tail, x)) {
        printf("Value %d deleted from the list.\n", x);
    } else {
        printf("Value %d not found in the list.\n", x);
    }
```

```
    // Print updated list
    display(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Imagine you're managing a store's inventory list, and some products were accidentally entered multiple times. You need to remove the duplicate products from the list to ensure each product appears only once.

You have an unsorted doubly linked list of product IDs. Some of these product IDs may appear more than once, and your goal is to remove any duplicates.

Note: Insert elements at the front of the doubly linked list.

Input Format

The first line of input consists of an integer n, representing the number of elements in the list.

The second line of input consists of n space-separated integers representing the list elements.

Output Format

The output prints the final list after removing duplicate nodes, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

12 12 10 4 8 4 6 4 4 8

Output: 8 4 6 10 12

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

// Insert at front
void insertFront(struct Node** head, int val) {
    struct Node* newNode = createNode(val);
    newNode->next = *head;
    if (*head != NULL) {
        (*head)->prev = newNode;
    }
    *head = newNode;
}

// Remove duplicates from unsorted doubly linked list
void removeDuplicates(struct Node* head) {
    struct Node* current = head;
    while (current != NULL) {
        struct Node* runner = current->next;
        while (runner != NULL) {
            if (runner->data == current->data) {
                struct Node* del = runner;
                if (runner->prev) runner->prev->next = runner->next;
```

```
            if (runner->next) runner->next->prev = runner->prev;
            runner = runner->next;
            free(del);
        } else {
            runner = runner->next;
        }
    }
    current = current->next;
  }
}

// Display list
void display(struct Node* head) {
  struct Node* temp = head;
  while (temp) {
    printf("%d", temp->data);
    if (temp->next) printf(" ");
    temp = temp->next;
  }
  printf(" \n"); // trailing space before newline
}

int main() {
  int n, val;
  scanf("%d", &n);

  struct Node* head = NULL;

  for (int i = 0; i < n; i++) {
    scanf("%d", &val);
    insertFront(&head, val); // insert at front
  }

  removeDuplicates(head);
  display(head);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 6_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Krish is participating in a coding competition and has been given an array of integers. His task is to sort the array using the Counting Sort algorithm.

Your goal is to help Krish by implementing the countSort method to sort the array in non-decreasing order.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

vtu29890

The output prints a single line containing N space-separated integers representing the sorted array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
4 2 2 8 3

Output: 2 2 3 4 8

Answer

```
#include <stdio.h>

void countSort(int arr[], int n) {
    int output[n];
    int max = arr[0];

    // Find maximum element
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }

    // Initialize count array
    int count[max + 1];
    for (int i = 0; i <= max; i++)
        count[i] = 0;

    // Store frequency of each element
    for (int i = 0; i < n; i++)
        count[arr[i]]++;

    // Convert count array to prefix sums
    for (int i = 1; i <= max; i++)
        count[i] += count[i - 1];

    // Build the output array (stable sorting)
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
```

```
        count[arr[i]]--;
    }

    // Copy sorted elements back to original array
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

int main() {
    int N;
    scanf("%d", &N);

    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    countSort(arr, N);

    for (int i = 0; i < N; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Raj, a keen problem solver, has come across an intriguing numerical puzzle that involves sorting numbers in ascending order. Eager to crack the code, he decides to implement the Radix Sort algorithm.

Your task is to assist Raj in designing a program to solve this numerical puzzle.

Input Format

The first line contains an integer n, denoting the number of elements to be sorted.

The second line consists of n space-separated integers, representing the numbers to be sorted.

Output Format

The output prints the array after it has been sorted using the Radix Sort algorithm, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
543 986 217 765 329
Output: 217 329 543 765 986

Answer

```
#include <stdio.h>

// Utility function to get the maximum value in the array
int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}

// Counting Sort used as a subroutine for Radix Sort
void countingSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    // Count occurrences of digits
    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Convert count[] to prefix sums
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];
```

```
    // Build output array (stable sorting)
    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy sorted elements back to arr[]
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

// Radix Sort function
void radixSort(int arr[], int n) {
    int max = getMax(arr, n);

    // Sort by each digit (exp = 1, 10, 100, ...)
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    radixSort(arr, n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
#include <stdio.h>

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;      // Initialize largest as root
    int left = 2 * i + 1;  // left child
    int right = 2 * i + 2; // right child
```

```
    // If left child is larger than root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger than largest so far
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}

// Function to perform Heap Sort
void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i >= 0; i--) {
        // Move current root to end
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // Call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}

int main() {
    int n, K;
    scanf("%d", &n);
```

```
    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &K);

    // Sort the array using Heap Sort
    heapSort(arr, n);

    // Kth best-selling product (Kth largest    index n-K)
    printf("%d", arr[n - K]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
#include <string.h>

// Heapify function for strings
void heapify(char arr[][101], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // Compare left child
    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    // Compare right child
    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;

    // If largest is not root
    if (largest != i) {
        char temp[101];
        strcpy(temp, arr[i]);
        strcpy(arr[i], arr[largest]);
        strcpy(arr[largest], temp);

        // Recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}

// Heap Sort for strings
void heapSort(char arr[][101], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
```

```
    // Extract elements one by one
    for (int i = n - 1; i >= 0; i--) {
        char temp[101];
        strcpy(temp, arr[0]);
        strcpy(arr[0], arr[i]);
        strcpy(arr[i], temp);

        heapify(arr, i, 0);
    }
}

int main() {
    int N;
    scanf("%d", &N);

    char arr[N][101]; // assuming max string length = 100
    for (int i = 0; i < N; i++) {
        scanf("%s", arr[i]);
    }

    heapSort(arr, N);

    for (int i = 0; i < N; i++) {
        printf("%s ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

You're organizing a gift-giving event where participants exchange gifts of varying values. To ensure fairness, you use the Bucket Sort algorithm. You gather the participants' gift values and initialize a count array.

Then, you iterate through the values, updating the count array. Next, you sort the values in descending order by iterating through the count array. Finally, you display the sorted values, ensuring fairness in the gift exchange.

Input Format

The first line contains an integer N, representing the number of gift values.

The second line contains N space-separated integers representing the gift values brought by the participants. Each gift value is an integer between 1 and 999, inclusive.

Output Format

The output displays a single line containing N space-separated integers representing the sorted gift values in descending order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
156 378 453 278 281 987
Output: 987 453 378 281 278 156

Answer

```
#include <stdio.h>

int main() {
   int N;
   scanf("%d", &N);

   int arr[N];
   for (int i = 0; i < N; i++) {
      scanf("%d", &arr[i]);
   }

   // Since gift values are between 1 and 999
   int count[1000] = {0};

   // Fill count array
   for (int i = 0; i < N; i++) {
      count[arr[i]]++;
   }
```

```
    // Print values in descending order
    for (int i = 999; i >= 1; i--) {
        while (count[i] > 0) {
            printf("%d ", i);
            count[i]--;
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Yuvana, an avid programmer, is keen on mastering various sorting algorithms and has recently delved into Counting Sort. As a practice exercise, she is given an array of N integers where each element ranges from 1 to N-1. Some numbers may appear multiple times in the array. Her task is to identify and print all numbers that appear more than once. If no such duplicates are found, she should print -1.

Help Yuvana by implementing a function that efficiently finds and prints duplicate numbers using the Counting Sort technique.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

vtu29890

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints all the duplicate numbers that appear more than once in the array, each separated by a space.

If no duplicates are found, print -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 3 4 2 2

Output: 2

Answer

```
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    // Count array of size N (since elements range from 1 to N-1)
    int count[N];
    for (int i = 0; i < N; i++) {
        count[i] = 0;
    }

    // Count occurrences
    for (int i = 0; i < N; i++) {
        count[arr[i]]++;
    }
```

```
    int found = 0;
    for (int i = 1; i < N; i++) {
        if (count[i] > 1) {
            printf("%d ", i);
            found = 1;
        }
    }

    if (!found) {
        printf("-1");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Anu is tasked with organizing a list of fruit names stored as strings. To ensure that the list is easy to navigate, she decides to sort the strings in ascending alphabetical order.

Being a fan of efficient sorting algorithms, Anu opts to use radix sort for this task. Help Anu with a program that can take a list of fruit names as input and output the sorted list.

Input Format

The first line of input consists of an integer n, representing the number of fruit names in the list.

The next n lines each consist of a string, representing the names of the fruits.

Output Format

The output prints n lines containing the sorted list of fruit names in ascending alphabetical order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
apple
banana
orange
grape

Output: apple
banana
grape
orange

Answer

```
#include <stdio.h>
#include <string.h>

// Get maximum length string
int getMaxLen(char arr[][101], int n) {
    int maxLen = strlen(arr[0]);
    for (int i = 1; i < n; i++) {
        int len = strlen(arr[i]);
        if (len > maxLen)
            maxLen = len;
    }
    return maxLen;
}

// Counting Sort for a specific character position
void countingSort(char arr[][101], int n, int pos) {
    char output[n][101];
    int count[256] = {0}; // ASCII characters

    // Count occurrences of each character at position
    for (int i = 0; i < n; i++) {
        int index;
        if (strlen(arr[i]) > pos)
            index = (unsigned char)arr[i][pos];
        else
            index = 0; // shorter strings treated as smaller
        count[index]++;
```

```
    }

    // Prefix sums
    for (int i = 1; i < 256; i++)
        count[i] += count[i - 1];

    // Build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        int index;
        if (strlen(arr[i]) > pos)
            index = (unsigned char)arr[i][pos];
        else
            index = 0;
        strcpy(output[count[index] - 1], arr[i]);
        count[index]--;
    }

    // Copy back
    for (int i = 0; i < n; i++)
        strcpy(arr[i], output[i]);
}

// Radix Sort for strings (lexicographic)
void radixSort(char arr[][101], int n) {
    int maxLen = getMaxLen(arr, n);

    // Sort from last character to first
    for (int pos = maxLen - 1; pos >= 0; pos--) {
        countingSort(arr, n, pos);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[n][101];
    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    radixSort(arr, n);
```

```
    for (int i = 0; i < n; i++) {
        printf("%s\n", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500

5

Output: 5th largest Salary: 28000

Answer

```
#include <stdio.h>

// Heapify function for max-heap
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

// Build max-heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}

// Extract max element from heap
int extractMax(int arr[], int *n) {
    int max = arr[0];
    arr[0] = arr[*n - 1];
    (*n)--;
    heapify(arr, *n, 0);
    return max;
}
```

```
int main() {
    int N, K;
    scanf("%d", &N);

    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &K);

    // Build max heap
    buildHeap(arr, N);

    int result;
    for (int i = 0; i < K; i++) {
        result = extractMax(arr, &N);
    }

    printf("%dth largest Salary: %d", K, result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Manage a warehouse inventory with unique barcode numbers utilizing a specialized sorting algorithm designed for barcodes with exactly 16 digits. Input the number of barcode numbers (N) and their corresponding values.

Employ the Bucket Sort algorithm to distribute and sort the numbers based on their digits. Display the sorted barcode numbers, ensuring an efficient and organized warehouse inventory management system tailored for barcode systems with 16 digits.

Input Format

The first line contains an integer N, representing the number of barcode numbers to be sorted.

The following N lines contain exactly 16-digit long long integers, each representing a unique barcode number.

Output Format

The output displays a single line containing "Sorted barcode numbers:" followed by the sorted barcode numbers in ascending order. Each barcode number is displayed on a separate line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2345678901234567
1234567890123456
3456789012345678
5678901234567890
4567890123456789

Output: Sorted barcode numbers:
1234567890123456
2345678901234567
3456789012345678
4567890123456789
5678901234567890

Answer

```
#include <stdio.h>

// Function to perform Bucket Sort using counting approach
void bucketSort(long long arr[], int n) {
   // Since barcodes are 16-digit, values can be very large.
   // We'll use a simple comparison-based bucket sort (like counting sort logic
but with buckets).
   // For simplicity, we sort using standard bucket distribution with comparisons.

   // Simple insertion sort inside buckets (since N ≤ 100, efficient enough)
   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {
         if (arr[i] > arr[j]) {
```

```
                long long temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

int main() {
    int N;
    scanf("%d", &N);

    long long arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%lld", &arr[i]);
    }

    bucketSort(arr, N);

    printf("Sorted barcode numbers:\n");
    for (int i = 0; i < N; i++) {
        printf("%lld\n", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. In Radix Sort, what happens if the input array contains numbers with different lengths?

Answer

The shorter numbers are padded with zeros.

Status : Correct ***Marks : 1/1***

2. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct ***Marks : 1/1***

vtu29890

3. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct ***Marks : 1/1***

4. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct ***Marks : 1/1***

5. Bucket sort is most efficient in the case when _____

Answer

The input is uniformly distributed

Status : Correct ***Marks : 1/1***

6. Which of the following is not necessarily a stable sorting algorithm?

Answer

Bucket Sort

Status : Correct ***Marks : 1/1***

7. Which step is essential to implement Bucket Sort correctly?

Answer

Dividing the range of input into intervals (buckets)

Status : Correct ***Marks : 1/1***

vtu29890

8. Which of the following is a characteristic of Radix Sort?

Answer

It is a stable sorting algorithm.

Status : Correct ***Marks : 1/1***

9. A database contains the following employee IDs: [1263, 2910, 8742, 3891, 2156]. Using Radix Sort, which of the following is the correct order of employee IDs after the first pass (sorting by the least significant digit)?

Answer

[2910, 3891, 8742, 1263, 2156]

Status : Correct ***Marks : 1/1***

10. What is the purpose of the counting sort algorithm in Radix Sort?

Answer

To sort the elements based on the current digit.

Status : Correct ***Marks : 1/1***

11. When does Heap Sort perform at its worst?

Answer

When the input array is already sorted in ascending order.

Status : Wrong ***Marks : 0/1***

12. How does Counting Sort handle duplicate elements?

Answer

It preserves their relative order

Status : Correct ***Marks : 1/1***

vtu29890

13. What is the purpose of the count array in Counting Sort?

Answer

To store the count of each unique element of the input array at their respective indices

Status : Correct ***Marks : 1/1***

14. What is the purpose of calculating the prefix sum in the Counting Sort algorithm?

Answer

To determine the final positions of elements in the output array

Status : Correct ***Marks : 1/1***

15. What is the initial value of each element in the count array in Counting Sort?

Answer

0

Status : Correct ***Marks : 1/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 6_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Alex is working on a data analysis project and needs to find specific elements within a dataset. He has an array of integers where some numbers may be repeated. His goal is to identify the Kth smallest element in this array. The challenge is that the array can contain duplicate values, so finding the exact Kth smallest element requires careful consideration.

Alex has decided to use a counting sort approach to solve this problem. Given an array of integers and a number K, where K is smaller than the size of the array, he needs to find and print the Kth smallest element in the array.

Help Alex by implementing a function that efficiently finds and prints the Kth smallest element using the counting sort approach.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

The third line of input consists of an integer K, representing the position of the smallest element to find.

Output Format

The output prints the Kth smallest element in the array as "Kth Smallest element is <smallest element>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
7 10 4 3 20 15
3
Output: Kth Smallest element is 7

Answer

```
#include <stdio.h>

int main() {
    int N, K;
    scanf("%d", &N);

    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &K);

    // Since elements are between 1 and 100
    int count[101] = {0};
```

```
    // Count occurrences
    for (int i = 0; i < N; i++) {
        count[arr[i]]++;
    }

    // Traverse count array to find Kth smallest
    int pos = 0, result = -1;
    for (int i = 1; i <= 100; i++) {
        pos += count[i];
        if (pos >= K) {
            result = i;
            break;
        }
    }

    printf("Kth Smallest element is %d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohan is developing a text analysis tool and wants to sort a list of words based on the count of vowels in each word. He believes that sorting the words based on vowel count will help him analyze the text more effectively.

To achieve this, he plans to use radix sort. Assist Rohan with a suitable program to implement this sorting method for his project.

Input Format

The first line of input consists of an integer n, representing the number of words.

The next n lines each consist of a string, representing the words.

Output Format

The output prints n lines containing the sorted list of words based on the count

of vowels in each word.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
another
stuck
door
you
Output: stuck
door
you
another

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function to count vowels in a word
int countVowels(const char *word) {
    int count = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        char ch = tolower(word[i]);
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
            count++;
    }
    return count;
}

// Counting Sort based on a specific digit (vowel count digit)
void countingSort(char arr[][101], int n, int exp) {
    char output[n][101];
    int count[10] = {0}; // vowel counts are small, so base 10 is enough

    // Count occurrences of digit at exp place
    for (int i = 0; i < n; i++) {
        int v = countVowels(arr[i]);
```

```
        int index = (v / exp) % 10;
        count[index]++;
    }

    // Prefix sums
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        int v = countVowels(arr[i]);
        int index = (v / exp) % 10;
        strcpy(output[count[index] - 1], arr[i]);
        count[index]--;
    }

    // Copy back
    for (int i = 0; i < n; i++)
        strcpy(arr[i], output[i]);
}

// Radix Sort based on vowel counts
void radixSort(char arr[][101], int n) {
    // Find maximum vowel count
    int maxVowels = countVowels(arr[0]);
    for (int i = 1; i < n; i++) {
        int v = countVowels(arr[i]);
        if (v > maxVowels)
            maxVowels = v;
    }

    // Apply counting sort for each digit in vowel count
    for (int exp = 1; maxVowels / exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[n][101];
    for (int i = 0; i < n; i++) {
```

```
        scanf("%s", arr[i]);
    }

    radixSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%s\n", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

In a manufacturing plant, each product is assigned an integer production code. To streamline quality control, the products have to be sorted in ascending order based on the last digit of their production codes.

Develop a program that uses Radix Sort to sort the products according to their code's last digit.

Input Format

The first line consists of an integer n, representing the number of products.

The second line consists of n space-separated integers each representing a product code.

Output Format

The output prints the sorted product codes based on the last digit in ascending order, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

24570 61064 65391

Output: 24570 65391 61064

Answer

```
#include <stdio.h>

// Counting Sort based on last digit
void countingSort(int arr[], int n) {
    int output[n];
    int count[10] = {0};

    // Count occurrences of last digit
    for (int i = 0; i < n; i++) {
        int digit = arr[i] % 10;
        count[digit]++;
    }

    // Prefix sums
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        int digit = arr[i] % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    // Copy back
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    // Radix Sort only on last digit
    countingSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Meera, a librarian, wants to organize a list of book titles based on how many words each title contains. If two titles have the same number of words, they should be arranged in lexicographical (dictionary) order.

Write a program to sort the given book titles using a bucket-based approach, where titles are first grouped by word count and then alphabetically sorted within each group.

Input Format

The first line contains an integer N, representing the number of book titles.

The next N lines: Each line contains a book title (may contain spaces and special characters).

Output Format

The output prints "Sorted book titles:"

Then print the sorted book titles, each on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Harry Potter and the Philosopher's Stone
To Kill a Mockingbird
The Great Gatsby
1984

Output: Sorted book titles:
1984
The Great Gatsby
To Kill a Mockingbird
Harry Potter and the Philosopher's Stone

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function to count words in a title
int countWords(const char *title) {
    int count = 0;
    int inWord = 0;
    for (int i = 0; title[i] != '\0'; i++) {
        if (!isspace(title[i]) && !inWord) {
            inWord = 1;
            count++;
        } else if (isspace(title[i])) {
            inWord = 0;
        }
    }
    return count;
}

// Lexicographic comparison
int compareStrings(const char *a, const char *b) {
    return strcmp(a, b);
}

// Sort titles within a bucket lexicographically
void sortBucket(char bucket[][101], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (compareStrings(bucket[i], bucket[j]) > 0) {
                char temp[101];
                strcpy(temp, bucket[i]);
```

```
            strcpy(bucket[i], bucket[j]);
            strcpy(bucket[j], temp);
        }
    }
  }
}

int main() {
  int N;
  scanf("%d", &N);
  getchar(); // consume newline

  char titles[N][101];
  int wordCounts[N];

  for (int i = 0; i < N; i++) {
    fgets(titles[i], sizeof(titles[i]), stdin);
    titles[i][strcspn(titles[i], "\n")] = '\0'; // remove newline
    wordCounts[i] = countWords(titles[i]);
  }

  // Bucket sort by word count
  // Max words per title ≤ 100, so buckets up to 100
  char buckets[101][50][101]; // bucket[wordCount][index][title]
  int bucketSizes[101] = {0};

  for (int i = 0; i < N; i++) {
    int wc = wordCounts[i];
    strcpy(buckets[wc][bucketSizes[wc]], titles[i]);
    bucketSizes[wc]++;
  }

  printf("Sorted book titles:\n");

  // Print buckets in ascending word count
  for (int wc = 1; wc <= 100; wc++) {
    if (bucketSizes[wc] > 0) {
      sortBucket(buckets[wc], bucketSizes[wc]);
      for (int j = 0; j < bucketSizes[wc]; j++) {
        printf("%s\n", buckets[wc][j]);
      }
    }
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 7_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

A publishing company is building a digital index system for storing book references. Each reference is represented by a single uppercase character. To ensure efficient organization, the system uses a Binary Search Tree (BST):

If the new character is smaller than the current node, it is placed in the left subtree.If the new character is greater, it is placed in the right subtree.

Once all references are stored in the BST, the system needs to perform a postorder traversal to generate a summary sequence, which is then used for indexing and cross-referencing.

Your task is to help the publishing company build the BST from the given characters and print the postorder traversal.

Example:

Input:

5

Z E W T Y

Output:

Postorder traversal: T Y W E Z

Input Format

The first line consists of an integer N, representing the number of characters.

The second line consists of N space-separated characters.

Output Format

The output displays the post-order traversal of the given inputs as space-separated.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
Z E W T Y
Output: Postorder traversal: T Y W E Z

Answer

-

Status : Skipped ***Marks : 0/10***

2. Problem Statement

Jake is learning about binary search trees(BST) and their operations. He wants to implement a program that can delete a node from a BST based on the given key value and print the remaining nodes in an in-order

traversal.

Assist Jake in the program.

Input Format

The first line of input consists of an integer n, representing the number of elements in BST.

The second line consists of n space-separated integers, representing the elements of the tree.

The third line consists of an integer x, representing the key value of the node to be deleted.

Output Format

The first line of output prints "Before deletion: " followed by the in-order traversal of the initial BST.

The second line prints "After deletion: " followed by the in-order traversal after the deletion of the key value.

If the key value is not present in the BST, print the original tree as it is.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
8 6 4 3 1
4
Output: Before deletion: 1 3 4 6 8
After deletion: 1 3 6 8

Answer

-

Status : - ***Marks : 0/10***

vtu29890

3. Problem Statement

Thilak is studying binary search trees (BSTs) and wants to implement a program that constructs a binary search tree from a given set of characters. He aims to perform a pre-order traversal of the constructed tree to understand its structure better.

Your task is to help Thilak by designing a program that accomplishes this.

Input Format

The first line of input consists of an integer N, representing the number of characters to be inserted into the BST.

The second line consists of N space-separated characters, representing the data elements to be inserted into the BST.

Output Format

The output prints the pre-order traversal of the constructed BST.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
C A E B D
Output: C A B E D

Answer

-

Status : - ***Marks : 0/10***

4. Problem Statement

You are required to implement basic operations on a Binary Search Tree (BST), like insertion and searching.

vtu29890

Insertion: Given a list of integers, construct a Binary Search Tree by repeatedly inserting each integer into the tree according to the rules of a BST.

Searching: Given an integer, search for its presence in the constructed Binary Search Tree. Print whether the integer is found or not.

Write a program to calculate this efficiently.

Input Format

The first line of input consists of an integer n, representing the number of nodes in the binary search tree.

The second line consists of the values of the nodes, separated by space as integers.

The third line consists of an integer representing, the value that is to be searched.

Output Format

The output prints, "Value <value> is found in the tree." if the given value is present, otherwise it prints: "Value <value> is not found in the tree."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
8 3 10 1 6 14 23
6
Output: Value 6 is found in the tree.

Answer

-

Status : - ***Marks : 0/10***

5. Problem Statement

vtu29890

Thiru is learning about binary search trees and wants to practice creating and traversing them. He needs to write a program that takes a series of characters as input and constructs a binary search tree with those characters.

After constructing the tree, Thiru wants to perform an in-order traversal and print all the characters in sorted order.

Input Format

The first line contains an integer n, representing the number of characters to be inserted into the binary search tree.

The second line contains n space-separated characters c_i, representing the characters to be inserted into the binary search tree.

Output Format

The output prints a single line containing all the characters of the binary search tree in sorted order, obtained by performing an in-order traversal of the tree, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
r e f s
Output: e f r s

Answer

-

Status : -

Marks : 0/10

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 7_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Given a BST, write an efficient function to delete a given key in it.

There are three cases that can be considered:

Deleting a node with no children: remove the node from the tree.Deleting a node with two childrenDeleting a node with one child

Input Format

The first line of the input consists of an integer n representing, the number of nodes.

The second line of the input consists of integers space-separated representing, input node values.

The third line of the input consists of an integer q representing, the node value to

be deleted.

Output Format

The output prints the node values before and after the deletion of the key value in the format:

"Before deletion of key value:"

"After deleting key value:"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 30 5 6 7
30
Output: Before deletion of key value:
5 6 7 10 30
After deleting key value:
5 6 7 10

Answer

-

Status : Skipped ***Marks : 0/10***

2. Problem Statement

Revi is studying binary search trees (BSTs) and wants to explore all the unique BSTs that can be constructed using distinct integers from 1 to n. He decides to write a program that generates all such BSTs and prints their preorder traversals.

Write a program to ensure that the preorder traversal of all possible BSTs constructed from numbers 1 to n is printed, each on a new line.

Example

Input

3

Output

1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Explanation

Input Format

The input consists of a single integer n, representing the number of nodes.

Output Format

Each line contains the preorder traversal (space-separated) of one possible BST using values from 1 to n.

Each traversal is printed on a new line.

There should be no blank lines between the outputs, and the order of output may vary depending on the internal recursive tree generation.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 1 2 3
1 3 2
2 1 3
3 1 2
3 2 1

vtu29890

Answer

-

Status : - ***Marks : 0/10***

3. Problem Statement

Jai is fascinated by binary trees and wants to create and manipulate them. He needs your help to implement basic operations like insertion and deletion in a binary search tree.

Given a sequence of characters representing the values to be inserted into a binary search tree, followed by a character V to be deleted from the tree, your task is to construct the binary search tree and delete the specified character from it.

Your program should perform the following operations:

Construct a binary search tree from the given sequence of characters.Delete the specified character V from the constructed tree.Print the elements of the tree in sorted order after deletion.

Input Format

The first line contains an integer N, denoting the number of characters in the sequence.

The second line contains N space-separated characters, representing the sequence of characters to be inserted into the binary search tree.

The third line contains a single character V (which is guaranteed to be present in the tree), representing the value to be deleted from the binary search tree.

Output Format

The output prints a single line containing the elements of the binary search tree in sorted order after deleting the specified character V.

If the specified value is not available in the tree, print the given input values in order traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
F B G A D
B
Output: A D F G

Answer

-

Status : - ***Marks : 0/10***

4. Problem Statement:

Arun is exploring operations on binary search trees (BST). He wants to write a program that takes an unsorted array of distinct integers and constructs a BST by inserting elements in the given order (not height-balanced).

After constructing the BST, he wants to delete all nodes at the deepest level of the tree and perform a post-order traversal to print the remaining nodes.

Example

Input:

7

1 7 4 9 2 11 12

Output:

Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Explanation:

The BST is constructed by inserting elements in the given order: 1, 7, 4, 9, 2, 11, 12. This creates a regular BST (not height-balanced) where each element is inserted according to BST properties.

The tree is traversed in post-order before deleting the deepest level, showing: 2, 4, 12, 11, 9, 7, 1.

The deepest level nodes (in this case, node 12) are then deleted.

The tree is traversed in post-order after deleting the deepest level, showing: 2, 4, 11, 9, 7, 1.

Input Format

The first line contains an integer n, representing the number of nodes to be inserted in the BST.

The second line contains n space-separated integers, representing the elements to be inserted into the BST in the given order.

Output Format

The first line prints "Before deleting: " followed by the space-separated elements of the tree in post-order traversal before deleting the deepest level, ending with a space and newline.

The second line prints "After deleting: " followed by the space-separated elements of the tree in post-order traversal after deleting the deepest level, ending with a space and newline.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
1 7 4 9 2 11 12

Output: Before deleting: 2 4 12 11 9 7 1
After deleting: 2 4 11 9 7 1

Answer

vtu29890

-

Status : -

Marks : 0/10

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 1

Section 1 : MCQ

1. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the right child of the node 12?

Answer

16

Status : Correct ***Marks : 1/1***

2. Which of the following statements is true about deleting a node with one child from a binary search tree?

Answer

Status : Skipped ***Marks : 0/1***

vtu29890

3. After deleting a node in a binary search tree, how is the replacement node determined if the node to be deleted has only one child?

Answer

-

Status : - ***Marks : 0/1***

4. Which of the following operations can be used to traverse a Binary Search Tree (BST) in ascending order?

Answer

-

Status : - ***Marks : 0/1***

5. What will be the in-order traversal sequence of the following binary search tree, after deleting node 40?

Answer

-

Status : - ***Marks : 0/1***

6. Find the pre-order traversal of the given binary search tree.

Answer

-

Status : - ***Marks : 0/1***

7. In a binary search tree, which of the following traversals is used for getting ascending order values?

Answer

vtu29890

-

Status : - *Marks : 0/1*

8. Find the inorder traversal of the given binary search tree.

Answer

-

Status : - *Marks : 0/1*

9. When comparing a value to be searched with the root node in a BST?

Answer

-

Status : - *Marks : 0/1*

10. In a binary search tree (BST), if the value to be searched is smaller than the value of the root, where should the search proceed?

Answer

-

Status : - *Marks : 0/1*

11. Which of the following nodes would replace node 40 if it were to be deleted?

Answer

-

Status : - *Marks : 0/1*

vtu29890

12. The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the post-order traversal sequence of the same tree?

Answer

-

Status : - ***Marks : 0/1***

13. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the left child of the node 16?

Answer

-

Status : - ***Marks : 0/1***

14. Consider the given binary search tree, if the root node is deleted, then the new root can be _____.

Answer

-

Status : - ***Marks : 0/1***

15. Find the postorder traversal of the given binary search tree.

Answer

-

Status : - ***Marks : 0/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 7_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Toy loves trees, especially alphabet trees! He wants to build his alphabet tree by inserting characters one by one. Given a sequence of characters, Toy needs to construct a binary search tree where each node contains a character. After constructing the tree, Toy wants to search for a specific character in the tree to see if it exists or not.

Write a program to help Toy construct his alphabet tree and perform searches.

Input Format

The first line contains an integer, N, denoting the number of characters Toy wants to insert into the tree.

vtu29890

The second line contains N space-separated characters, representing the characters Toy wants to insert into the tree.

The third line contains a single character, V, representing the character Toy wants to search for in the tree.

Output Format

If the character V is found in the constructed alphabet tree, print "Character - [V] is Found".

If the character V is not found in the constructed alphabet tree, print "Character - [V] is Not Found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
F A B C D
B
Output: Character - B is Found

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
   char data;
   struct TreeNode* left;
   struct TreeNode* right;
};

// Function to create a new TreeNode
struct TreeNode* createNode(char key) {
   struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct
TreeNode));
   newNode->data = key;
```

```
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Function to insert a key into the BST
struct TreeNode* insert(struct TreeNode* root, char key) {
    if (root == NULL) return createNode(key);
    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);
    return root;
}

// Function to search for a key in the BST
int search(struct TreeNode* root, char key) {
    if (root == NULL) return 0;
    if (root->data == key) return 1;
    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

// Function to free the memory allocated for the BST
void freeTree(struct TreeNode* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    int N;
    char rootValue, V;
    scanf("%d", &N);

    struct TreeNode* root = NULL;

    for (int i = 0; i < N; i++) {
        char key;
        scanf(" %c", &key); // Note the space before %c to skip any whitespace
```

```
    if (i == 0) rootValue = key;
    root = insert(root, key);
}

scanf(" %c", &V); // Note the space before %c to skip any whitespace
if (search(root, V)) {
    printf("Character - %c is Found", V);
} else {
    printf("Character - %c is Not Found", V);
}

// Free the allocated memory
freeTree(root);

return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Singh is studying binary search trees (BSTs) in his computer science class. He is particularly interested in finding the inorder predecessor of a given node in a BST.

The inorder predecessor of a node in a BST is the node with the largest value that is smaller than the given node's value.

Given a binary search tree and a node value, Singh needs your help to write a program that finds and outputs the in-order predecessor of the given node.

Input Format

The first line of input consists of an integer n, representing the number of keys to be inserted into the Binary Search Tree (BST).

The second line of input consists of n integers, representing the value of each

key to be inserted into the BST, separated by a space.

The third line of input consists of an integer value key, representing the value of the node for which the inorder predecessor is to be found.

Output Format

The output displays "Inorder predecessor of X is Y", where Y is the value of the inorder predecessor of the node and X is the key.

If the node with the given key is not found in the BST, the output displays "Key not found".

If the given key has no inorder predecessor in the BST, the output displays "No inorder predecessor found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
20 70 90
90
Output: Inorder predecessor of 90 is 70

Answer

```
#include <stdio.h>
#include <stdlib.h>

// BST Node
struct TreeNode {
   int val;
   struct TreeNode* left;
   struct TreeNode* right;
};

// Create a new node
struct TreeNode* createNode(int val) {
   struct TreeNode* node = (struct TreeNode*)malloc(sizeof(struct TreeNode));
```

```
    node->val = val;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// Insert into BST
struct TreeNode* insert(struct TreeNode* root, int val) {
    if (root == NULL)
        return createNode(val);
    if (val < root->val)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
}

// Search for a node with given key
struct TreeNode* search(struct TreeNode* root, int key) {
    if (root == NULL || root->val == key)
        return root;
    if (key < root->val)
        return search(root->left, key);
    else
        return search(root->right, key);
}

// Find the rightmost node in a subtree
struct TreeNode* findMax(struct TreeNode* root) {
    while (root->right != NULL)
        root = root->right;
    return root;
}

int main() {
    int n;
    scanf("%d", &n);

    struct TreeNode* root = NULL;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
```

```
        root = insert(root, val);
    }

    int key;
    scanf("%d", &key);

    struct TreeNode* node = search(root, key);
    if (node == NULL) {
        printf("Key not found\n");
        return 0;
    }

    // Case 1: Node has left subtree
    if (node->left != NULL) {
        struct TreeNode* pred = findMax(node->left);
        printf("Inorder predecessor of %d is %d\n", key, pred->val);
    } else {
        // Case 2: No left subtree, find ancestor
        struct TreeNode* ancestor = NULL;
        struct TreeNode* current = root;
        while (current != NULL) {
            if (key > current->val) {
                ancestor = current;
                current = current->right;
            } else if (key < current->val) {
                current = current->left;
            } else {
                break;
            }
        }
        if (ancestor != NULL)
            printf("Inorder predecessor of %d is %d\n", key, ancestor->val);
        else
            printf("No inorder predecessor found\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Dhruv is working on a project where he needs to implement a Binary Search Tree (BST) data structure and perform various operations on it.

He wants to create a program that allows him to build a BST, traverse it in different orders (inorder, preorder, postorder), and exit the program when needed.

Help Dhruv by designing a program that fulfils his requirements.

Input Format

The first input consists of the choice.

If the choice is 1, enter the number of elements N and the elements inserted into the tree, separated by a space in a new line.

If the choice is 2, print the in-order traversal.

If the choice is 3, print the pre-order traversal.

If the choice is 4, print the post-order traversal.

If the choice is 5, exit.

Output Format

The output prints the results based on the choice.

For choice 1, print "BST with N nodes is ready to use," where N is the number of nodes inserted.

For choice 2, print the inorder traversal of the BST after "BST Traversal in INORDER"

For choice 3, print the pre-order traversal of the BST after "BST Traversal in PREORDER"

For choice 4, print the post-order traversal of the BST after "BST Traversal in POSTORDER"

For choice 5, the program exits.

vtu29890

If the choice is greater than 5, print "Wrong choice".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
5
12 78 96 34 55
2
3
4
5
Output: BST with 5 nodes is ready to use
BST Traversal in INORDER
12 34 55 78 96
BST Traversal in PREORDER
12 78 34 55 96
BST Traversal in POSTORDER
55 34 96 78 12

Answer

```
#include <stdio.h>
#include<stdlib.h>
struct tnode
{
   int data;
   struct tnode *right;
   struct tnode *left;
};

struct tnode *CreateBST(struct tnode *root, int item)
{
   if(root == NULL)
   {
      root = (struct tnode *)malloc(sizeof(struct tnode));
      root->left = root->right = NULL;
```

```
        root->data = item;
        return root;
    }
    else
    {
        if(item < root->data )
            root->left = CreateBST(root->left,item);
        else if(item > root->data )
            root->right = CreateBST(root->right,item);

        return(root);
    }
}

void Inorder(struct tnode *root)
{
    if( root != NULL)
    {
        Inorder(root->left);
        printf("%d ",root->data);
        Inorder(root->right);
    }
}

void Preorder(struct tnode *root)
{
    if( root != NULL)
    {
        printf("%d ",root->data);
        Preorder(root->left);
        Preorder(root->right);
    }
}

void Postorder(struct tnode *root)
{
    if( root != NULL)
    {
        Postorder(root->left);
        Postorder(root->right);
        printf("%d ",root->data);
    }
```

```
}
int main()
{
    struct tnode *root = NULL;
    int choice, item, n, i;
    do
    {
        scanf("%d",&choice);
        switch(choice)
        {
        case 1:
            root = NULL;
            scanf("%d",&n);
            for(i = 1; i <= n; i++)
            {
                scanf("%d",&item);
                root = CreateBST(root,item);
            }
            printf("BST with %d nodes is ready to use\n", n);
            break;
        case 2:
            printf("BST Traversal in INORDER\n");
            Inorder(root);
            printf("\n");
            break;
        case 3:
            printf("BST Traversal in PREORDER\n");
            Preorder(root);
            printf("\n");
            break;
        case 4:
            printf("BST Traversal in POSTORDER\n");
            Postorder(root);
            printf("\n");
            break;
        case 5:
            exit(0);
            break;
        default:
            printf("Wrong choice\n");
            break;
        }
```

```
    } while(1);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Jay is studying binary search trees (BSTs) as part of his computer science curriculum. He is particularly interested in understanding operations like insertion and deletion in a BST.

Jay wants to write a program that takes a sequence of characters as input, constructs a BST using those characters, deletes the minimum element from the BST, and then outputs the elements of the tree in sorted order.

Help Jay by designing a program that achieves this.

Input Format

The first line of input contains an integer n, denoting the number of characters to be inserted into the BST.

The second line contains n space-separated characters, each character representing a node to be inserted into the BST.

Output Format

The output displays a single line displaying the space-separated characters of the BST in sorted order after deleting the minimum element in the format:

"Inorder traversal after deleting the minimum element:"

"[char1] [char2] ...[char_n]"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
K J M V H F
Output: Inorder traversal after deleting the minimum element:
H J K M V

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   char data;
   struct Node* left;
   struct Node* right;
};

struct Node* newNode(char data) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = data;
   node->left = NULL;
   node->right = NULL;
   return node;
}
struct Node* insert(struct Node* root, char data) {
   if (root == NULL)
      return newNode(data);

   if (data < root->data)
      root->left = insert(root->left, data);
   else if (data > root->data)
      root->right = insert(root->right, data);

   return root;
}

struct Node* deleteMin(struct Node* root) {
   if (root == NULL)
      return root;
```

```
    if (root->left == NULL) {
        struct Node* temp = root->right;
        free(root);
        return temp;
    }

    root->left = deleteMin(root->left);
    return root;
}

void inOrderTraversal(struct Node* root) {
    if (root) {
        inOrderTraversal(root->left);
        printf("%c ", root->data);
        inOrderTraversal(root->right);
    }
}
int main() {
    struct Node* root = NULL;
    int n;
    char data;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf(" %c", &data);
        root = insert(root, data);
    }

    root = deleteMin(root);

    printf("Inorder traversal after deleting the minimum element:\n");
    inOrderTraversal(root);


    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 8_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Liam is developing a social media analytics tool for tracking user interactions. He needs to identify the connected components of users in the network by traversing a directed graph. Each user is represented as a vertex, and the connections between users are represented as directed edges. Liam's task is to implement a Depth-First Search (DFS) algorithm to perform a traversal of the network, starting from a given user's profile.

Can you help Liam by implementing the DFS algorithm to identify all connected users starting from a specified user profile?

Example

Input:

6 6

vtu29890

0 1

0 2

1 2

2 0

2 3

3 3

2

Output:

2 0 1 3

Explanation:

Starting from vertex 2, it visits and marks vertex 2 as visited.

Vertex 2 is printed.

Checking the neighbors of vertex 2: 0 and 3.

Vertex 0 is visited, so DFS is called with vertex 0.

Vertex 0 is visited and printed.

Vertex 1 is visited and printed.

Vertex 3 is visited and printed.

The final output is "2 0 1 3".

Input Format

The first line contains two space-separated integers n and m, representing the total number of users (vertices) and the number of connections (edges) in the network, respectively.

The next m lines contain two space-separated integers u and v, representing a directed connection from user u to user v.

The last line contains a single integer s, representing the ID of the source user profile from which the DFS traversal should start.

Output Format

The output displays a single line containing the user IDs in the order they were visited during the DFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 6
0 1
0 2
1 2
2 0
2 3
3 3
2

Output: 2 0 1 3

Answer

```
#include <stdio.h>
#include <stdlib.h>
struct AdjListNode
{
   int dest;
   struct AdjListNode* next;
};
struct AdjList
{
   struct AdjListNode *head;
};

struct Graph
{
   int V;
   struct AdjList* array;
};
struct AdjListNode* newAdjListNode(int dest)
{
   struct AdjListNode* newNode =
```

```
    (struct AdjListNode*) malloc(sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}
struct Graph* createGraph(int V)
{
    struct Graph* graph =
        (struct Graph*) malloc(sizeof(struct Graph));
    graph->V = V;
  graph->array =
      (struct AdjList*) malloc(V * sizeof(struct AdjList));
   int i;
    for (i = 0; i < V; ++i)
        graph->array[i].head = NULL;

    return graph;
}
void addEdge(struct Graph* graph, int src, int dest)
{
    struct AdjListNode* newNode = newAdjListNode(dest);
    newNode->next = graph->array[src].head;
    graph->array[src].head = newNode;
    newNode = newAdjListNode(src);
    newNode->next = graph->array[dest].head;
    graph->array[dest].head = newNode;
}
void printGraph(struct Graph* graph)
{
    int v;
    for (v = 0; v < graph->V; ++v)
    {
        struct AdjListNode* pCrawl = graph->array[v].head;
        printf("vertex %d: head", v);
        // "vertex " + v + ": head"
        while (pCrawl)
        {
            printf(" -> %d", pCrawl->dest);
            pCrawl = pCrawl->next;
        }
        printf("\n");
    }
```

```
}
int main()
{
    int V = 5;
    int a,b,c,d,e;
    scanf("%d%d%d%d%d",&a,&b,&c,&d,&e);
    struct Graph* graph = createGraph(V);
    addEdge(graph, a, b);
    addEdge(graph, a, e);
    addEdge(graph, b, c);
    addEdge(graph, b, d);
    addEdge(graph, b, e);
    addEdge(graph, c, d);
    addEdge(graph, d, e);
   printGraph(graph);

    return 0;
}
```

Status : Wrong **Marks : 0/10**

2. Problem Statement

David is working on a project that involves traversing graphs for analyzing network connectivity. He is tasked with performing a Depth-First Search (DFS) traversal on a connected, undirected graph to explore all its vertices starting from a given node.

He needs help writing a program that can process multiple test cases where each test case describes a graph with its vertices and edges, and the program should output the DFS traversal order for each graph.

Can you assist David by writing a program that performs the DFS traversal for each test case?

Example

Input:

2

5 4

0 1 0 2 0 3 2 4

4 3

0 1 1 2 0 3

Output:

0 1 2 4 3

0 1 2 3

Explanation:

Visit all the vertices using a depth-first search algorithm using 0 as a source node.

Input Format

The first line of input contains an integer T denoting the number of test cases.

Each test case consists of two lines.

- The first line of each test case contains two integers N and E which denote the number of vertices and number of edges respectively.
- The second line of each test case contains E space-separated pairs u and v denoting that there is an edge from u to v.

Output Format

For each test case, the output displays the graph vertices in the depth-first traversal order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
5 4
0 1 0 2 0 3 2 4
4 3

0 1 1 2 0 3
Output: 0 1 2 4 3
0 1 2 3

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

void BFS(int** graph, int numCities, int source) {
    bool* visited = (bool*)malloc(numCities * sizeof(bool));
    for (int i = 0; i < numCities; i++) {
        visited[i] = false;
    }

    int* queue = (int*)malloc(numCities * sizeof(int));
    int front = -1;
    int rear = -1;

    visited[source] = true;
    queue[++rear] = source;

    while (front != rear) {  // Corrected loop condition
        int currentCity = queue[++front];

        printf("%d ", currentCity);

        for (int i = 0; i < numCities; i++) {
            if (graph[currentCity][i] == 1 && !visited[i]) {
                visited[i] = true;
                queue[++rear] = i;
            }
        }
    }

    free(visited);
    free(queue);
}

int main() {
    int numCities, numConnections;
```

```
    scanf("%d %d", &numCities, &numConnections);

    int** graph = (int**)malloc(numCities * sizeof(int*));
    for (int i = 0; i < numCities; i++) {
        graph[i] = (int*)malloc(numCities * sizeof(int));
        for (int j = 0; j < numCities; j++) {
            graph[i][j] = 0;
        }
    }

    for (int i = 0; i < numConnections; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        graph[a][b] = 1;
    }

    int source;
    scanf("%d", &source);

    BFS(graph, numCities, source);

    for (int i = 0; i < numCities; i++) {
        free(graph[i]);
    }
    free(graph);

    return 0;
}
```

Status : Wrong **Marks : 0/10**

3. Problem Statement

Given a graph with V vertices numbered from 0 to V-1 and a list of edges, determine whether the graph is bipartite or not.

Note: A bipartite graph is a type of graph where the set of vertices can be divided into two disjoint sets, say U and V, such that every edge connects a vertex in U to a vertex in V, there are no edges between vertices within the

same set.

Example:

Input: V = 4, edges[][]= [[0, 1], [0, 2], [1, 2], [2, 3]]

Output: No, the given graph is not Bipartite

Explanation

The graph is not bipartite because no matter how we try to color the nodes using two colors, there exists a cycle of odd length (like 1–2–0–1), which leads to a situation where two adjacent nodes end up with the same color. This violates the bipartite condition, which requires that no two connected nodes share the same color.

Input: V = 4, edges[][] = [[0, 1], [1, 2], [2, 3]]

Output: Yes, the given graph is Bipartite

Explanation:

The given graph can be colored in two colors so, it is a bipartite graph.

Input Format

The first line contains two space-separated integers V and E, representing the number of vertices and edges respectively.

The next E lines each contain two space-separated integers u and v, indicating an undirected edge between vertices u and v.

Output Format

The output print "Yes, the given graph is Bipartite" if the graph is bipartite.

Otherwise, print "No, the given graph is not Bipartite".

Refer to the sample output for the formatting specifications.

Sample Test Case

vtu29890

Input: 4 3
0 1
1 2
2 3
Output: Yes, the given graph is Bipartite

Answer

```
//
#include <stdio.h>

#define MAX_VERTICES 100

struct Graph {
   int visited[MAX_VERTICES];
   int adj[MAX_VERTICES][MAX_VERTICES];
};

void addEdge(struct Graph* g, int v, int w)
{
   g->adj[v][w] = 1;
}

void DFS(struct Graph* g, int v)
{
   g->visited[v] = 1;
   printf("%d ", v);

   for (int i = 0; i < MAX_VERTICES; ++i) {
      if (g->adj[v][i] == 1 && !g->visited[i]) {
         DFS(g, i);
      }
   }
}
int main()
{
   int n, e;
   scanf("%d", &n);
   scanf("%d", &e);

   struct Graph g;

   int v, w;
```

```
    for (int i = 0; i < e; i++) {
        scanf("%d %d", &v, &w);
        addEdge(&g, v, w);
    }

    int startVertex;
    scanf("%d", &startVertex);

    DFS(&g, startVertex);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

Sanjay is writing a program to manage an undirected graph using an adjacency list representation. Users will initially input the vertices to create the edges. The program will display the adjacency list of the graph.

Assist Sanjay in creating the program.

Input Format

The input consists of 5 nodes.

Edge information: ab ae bc bd be cd de.

Output Format

The output prints the adjacency list representation of the graph.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 2 3 4

Output: vertex 0: head -> 4 -> 1
vertex 1: head -> 4 -> 3 -> 2 -> 0
vertex 2: head -> 3 -> 1
vertex 3: head -> 4 -> 2 -> 1
vertex 4: head -> 3 -> 1 -> 0

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Structure for adjacency list node
typedef struct Node {
    int dest;
    struct Node* next;
} Node;

// Graph structure
typedef struct {
    int V;
    Node** adj;
} Graph;

// Create a new adjacency list node
Node* newNode(int dest) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->dest = dest;
    node->next = NULL;
    return node;
}

// Create a graph with V vertices
Graph* createGraph(int V) {
    Graph* g = (Graph*)malloc(sizeof(Graph));
    g->V = V;
    g->adj = (Node**)malloc(V * sizeof(Node*));
    for (int i = 0; i < V; i++)
        g->adj[i] = NULL;
    return g;
}
```

```
// Add an undirected edge
void addEdge(Graph* g, int u, int v) {
    Node* nodeV = newNode(v);
    nodeV->next = g->adj[u];
    g->adj[u] = nodeV;

    Node* nodeU = newNode(u);
    nodeU->next = g->adj[v];
    g->adj[v] = nodeU;
}

// Queue for BFS
typedef struct {
    int* arr;
    int front;
    int rear;
} Queue;

Queue* createQueue(int size) {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->arr = (int*)malloc(size * sizeof(int));
    q->front = 0;
    q->rear = 0;
    return q;
}

void enqueue(Queue* q, int val) {
    q->arr[q->rear++] = val;
}

int dequeue(Queue* q) {
    return q->arr[q->front++];
}

int isEmpty(Queue* q) {
    return q->front == q->rear;
}

// Check if the graph is bipartite using BFS
bool isBipartite(Graph* g) {
    int V = g->V;
    int* color = (int*)malloc(V * sizeof(int));
```

```
    for (int i = 0; i < V; i++) color[i] = -1;

    Queue* q = createQueue(V);

    for (int i = 0; i < V; i++) {
        if (color[i] == -1) {
            color[i] = 0;
            enqueue(q, i);

            while (!isEmpty(q)) {
                int u = dequeue(q);
                Node* p = g->adj[u];
                while (p) {
                    int v = p->dest;
                    if (color[v] == -1) {
                        color[v] = 1 - color[u];
                        enqueue(q, v);
                    } else if (color[v] == color[u]) {
                        free(color);
                        free(q->arr);
                        free(q);
                        return false;
                    }
                    p = p->next;
                }
            }
        }
    }

    free(color);
    free(q->arr);
    free(q);
    return true;
}

// Free graph memory
void freeGraph(Graph* g) {
    for (int i = 0; i < g->V; i++) {
        Node* p = g->adj[i];
        while (p) {
            Node* tmp = p;
            p = p->next;
```

```
        free(tmp);
    }
  }
  free(g->adj);
  free(g);
}

int main() {
  int V, E;
  scanf("%d %d", &V, &E);

  Graph* g = createGraph(V);

  for (int i = 0; i < E; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    addEdge(g, u, v);
  }

  if (isBipartite(g))
    printf("Yes, the given graph is Bipartite\n");
  else
    printf("No, the given graph is not Bipartite\n");

  freeGraph(g);
  return 0;
}
```

Status : Wrong **Marks : 0/10**

5. Problem Statement

Sophia is designing a transportation network for a city with multiple interconnected districts. Each district is represented as a vertex, and every road connecting two districts is represented as a directed edge in a graph.

To evaluate connectivity, Sophia needs a program that performs a Breadth-First Search (BFS) traversal of the city's network starting from a specified

source district, and outputs the order in which districts are visited.

Write a program to ensure that Sophia can perform the BFS traversal correctly and view the order of visited districts.

Input Format

The first line of input consists of two space-separated integers, v, and e, representing the total number of cities and the number of connections between cities.

The next e lines each contain two space-separated integers a and b, representing a directed edge from city a to city b.

The last line consists of a single integer s, representing the source city from which BFS traversal should start.

Output Format

The output prints the order in which the cities are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 7
0 1
0 2
1 3
1 4
2 4
3 5
4 5
0

Output: 0 1 2 3 4 5

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define MAX_VERTICES 100

void dfs(int graph[MAX_VERTICES][MAX_VERTICES], int visited[MAX_VERTICES],
int vertex, int N) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    for (int i = 0; i < N; i++) {
        if (graph[vertex][i] == 1 && visited[i] == 0) {
            dfs(graph, visited, i, N);
        }
    }
}

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int N, E;
        scanf("%d %d", &N, &E);

        int graph[MAX_VERTICES][MAX_VERTICES] = {0};

        for (int i = 0; i < E; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            graph[u][v] = 1;
            graph[v][u] = 1;
        }

        int visited[MAX_VERTICES] = {0};

        dfs(graph, visited, 0, N);
        printf("\n");
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Emma is developing a graph traversal feature for a social networking platform. Each user is represented by a lowercase English letter (a to z), and connections between users are represented as directed edges.

Emma wants to perform a Breadth-First Search (BFS) starting from a given user, visiting friends in lexicographical order whenever multiple connections are available.

Write a program to help Emma implement this lexicographically sorted BFS traversal.

Input:

N = 10, M = 10, S = ‘a’,

edges[][2] = { { ‘a’, ‘y’ }, { ‘a’, ‘z’ }, { ‘a’, ‘p’ }, { ‘p’, ‘c’ }, { ‘p’, ‘b’ }, { ‘y’, ‘m’ }, { ‘y’, ‘l’ }, { ‘z’, ‘h’ }, { ‘z’, ‘g’ }, { ‘z’, ‘i’ } }

Output:

a p y z b c l m g h i

Input Format

The first line contains an integer N, representing the number of nodes (users) in the graph.

The second line contains an integer M, representing the number of edges (friendships) in the graph.

The next M lines each contain two space-separated lowercase characters u and v, indicating a directed edge from user u to user v.

The last line contains an integer S, the starting node (user) for the BFS traversal.

Output Format

The output prints the users in the order they are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
10
a y
a z
a p
p c
p b
y m
y l
z h
z g

vtu29890

z i
a
Output: a p y z b c l m g h i

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_SIZE 100

// Function prototypes
bool isValid(int x, int y, int rows, int cols);
bool isAdjacent(char c1, char c2);
int DFS(char matrix[MAX_SIZE][MAX_SIZE], int rows, int cols, int x, int y);

// Main function
int main() {
    // Read input
    int rows, cols;
    scanf("%d%d", &rows, &cols);
    char matrix[MAX_SIZE][MAX_SIZE];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf(" %c", &matrix[i][j]);
        }
    }
    char startChar;
    scanf(" %c", &startChar);

    // Find the longest path starting from startChar
    int longestPath = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] == startChar) {
                int pathLength = DFS(matrix, rows, cols, i, j);
                if (pathLength > longestPath) {
                    longestPath = pathLength;
                }
            }
        }
    }
```

```
    // Print the result
    printf("The length of the longest path with consecutive characters starting
from character %c is %d\n", startChar, longestPath);

    return 0;
}

// Check if the given position is within the bounds of the matrix
bool isValid(int x, int y, int rows, int cols) {
    return (x >= 0 && x < rows && y >= 0 && y < cols);
}

// Check if c2 is an adjacent character of c1 (c2 should come after c1
alphabetically)
bool isAdjacent(char c1, char c2) {
    return (c2 == c1 + 1);
}

// DFS function to find the longest path starting from the given position
int DFS(char matrix[MAX_SIZE][MAX_SIZE], int rows, int cols, int x, int y) {
    // Mark the current position as visited
    bool visited[MAX_SIZE][MAX_SIZE] = {false};
    visited[x][y] = true;

    // Check all 8 adjacent positions
    int maxPathLength = 1;
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            if (isValid(x+i, y+j, rows, cols) && !visited[x+i][y+j] && isAdjacent(matrix[x]
[y], matrix[x+i][y+j])) {
                int pathLength = 1 + DFS(matrix, rows, cols, x+i, y+j);
                if (pathLength > maxPathLength) {
                    maxPathLength = pathLength;
                }
            }
        }
    }

    return maxPathLength;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Aayush is building a network verifier that must detect the presence of cycles in an undirected graph. Given a graph with a set number of vertices and edges, write a program to determine whether the graph contains at least one cycle using depth-first traversal.

Example

Input

N = 4, E = 4

Output

Yes

Explanation

The diagram clearly shows a cycle 0 to 2 to 1 to 0

Input

N = 4, E = 3

Output

No

Explanation

There is no cycle in the given graph.

Input Format

The first line of input consists of two integers, V and E, representing the number of vertices and edges separated by space, respectively.

The next E lines contain two space-separated integers u and v, describing an

edge between vertices u and v.

Output Format

The output prints "Graph contains a cycle" if a cycle exists, otherwise prints "Graph does not contain a cycle".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 5
1 0
0 2
2 1
0 3
3 4

Output: Graph contains a cycle

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_SIZE 100
struct Queue {
    int front, rear, size;
    unsigned capacity;
    int* array;
};

struct Queue* createQueue(unsigned capacity) {
    struct Queue* queue = (struct Queue*)malloc(sizeof(struct Queue));
    queue->capacity = capacity;
    queue->front = queue->size = 0;
    queue->rear = capacity - 1;
    queue->array = (int*)malloc(queue->capacity * sizeof(int));
    return queue;
}
```

```
int isFull(struct Queue* queue) {
    return (queue->size == queue->capacity);
}

int isEmpty(struct Queue* queue) {
    return (queue->size == 0);
}

void enqueue(struct Queue* queue, int item) {
    if (isFull(queue))
        return;
    queue->rear = (queue->rear + 1) % queue->capacity;
    queue->array[queue->rear] = item;
    queue->size = queue->size + 1;
}

int dequeue(struct Queue* queue) {
    if (isEmpty(queue))
        return -1;
    int item = queue->array[queue->front];
    queue->front = (queue->front + 1) % queue->capacity;
    queue->size = queue->size - 1;
    return item;
}

void freeQueue(struct Queue* queue) {
    free(queue->array);
    free(queue);
}

void bfs(int s, int list[MAX_SIZE][MAX_SIZE], int vis[MAX_SIZE], int nov) {
    struct Queue* q = createQueue(nov);

    enqueue(q, s);

    while (!isEmpty(q)) {
        int curr = dequeue(q);

        if (vis[curr] == 0) {
            vis[curr] = 1;
            printf("%d ", curr);
```

```
        }

        for (int i = 0; i < nov; ++i) {
            if (list[curr][i] == 1 && vis[i] == 0) {
                enqueue(q, i);
            }
        }
    }

    freeQueue(q);
}
int main() {
    int t;
    scanf("%d", &t);

    while (t-- > 0) {
        int nov, edg;
        scanf("%d %d", &nov, &edg);

        int list[MAX_SIZE][MAX_SIZE] = {0};
        int vis[MAX_SIZE] = {0};

        for (int i = 0; i < edg; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            list[u][v] = 1;
        }

        bfs(0, list, vis, nov);
        printf("\n");
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

Given an M × N matrix of characters, find the length of the longest path in the matrix starting from a given character.

In alphabetical order, all characters in the longest path should be increasing and consecutive. We are allowed to search the string in all eight possible directions, i.e., North, West, South, East, North-East, North-West, South-East, and South-West.

Use depth-first search (DFS) to solve this problem.

For example, consider the following matrix of characters:

The length of the longest path with consecutive characters starting from character C is 6.

The longest path is:

Input Format

The first line of input is an integer n representing the row size.

The second line of input is an integer m representing the column size.

The following lines of input are the characters of the matrix.

The last line of the input is the character for which the length of the longest path is to be found.

Output Format

The output prints "The length of the longest path with consecutive characters starting from character <startChar> is <length>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

5
d e h x b
a o g p e
d d c f d
e b e a s
c d y e n
c

Output: The length of the longest path with consecutive characters starting from character c is 6

Answer

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int V;
    int **adj;
} Graph;

Graph* createGraph(int vertices) {
    Graph* g = (Graph*)malloc(sizeof(Graph));
    g->V = vertices;
    g->adj = (int**)malloc(vertices * sizeof(int*));
    for (int i = 0; i < vertices; i++) {
        g->adj[i] = (int*)calloc(vertices, sizeof(int));
    }
    return g;
}

void addEdge(Graph* g, int src, int dest) {
    g->adj[src][dest] = 1;
    g->adj[dest][src] = 1;
}

int isCyclicUtil(int v, int visited[], int parent, Graph* g) {
    visited[v] = 1;
    for (int i = 0; i < g->V; i++) {
        if (g->adj[v][i]) {
            if (!visited[i]) {
                if (isCyclicUtil(i, visited, v, g))
                    return 1;
            } else if (i != parent) {
```

```
            return 1;
        }
    }
  }
  return 0;
}

int isCyclic(Graph* g) {
  int *visited = (int*)calloc(g->V, sizeof(int));
  for (int i = 0; i < g->V; i++) {
    if (!visited[i]) {
      if (isCyclicUtil(i, visited, -1, g)) {
        free(visited);
        return 1;
      }
    }
  }
  free(visited);
  return 0;
}
int main() {
  int V, E, u, v;
  scanf("%d %d", &V, &E);
  Graph* g = createGraph(V);
  for (int i = 0; i < E; i++) {
    scanf("%d %d", &u, &v);
    addEdge(g, u, v);
  }
  isCyclic(g) ? printf("Graph contains a cycle") : printf("Graph does not contain a
cycle");
  return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

Priya is developing a learning module to simulate breadth-first traversal of directed graphs. Given multiple test cases where each describes a directed

graph with vertices and edges, write a program to perform BFS starting from vertex 0 and print the traversal order for each test case.

Example

Input:

1

5 4

0 1 0 2 0 3 2 4

Output:

0 1 2 3 4

Explanation:

For the given graph, the BFS is 0 1 2 3 4.

Input Format

The first line of input contains an integer T, denoting the number of test cases.

Then T test cases follow. Each test case consists of two lines.

The descriptions of the test cases are as follows:

1. The first line of each test case contains two space-separated integers, N and E, which denote the number of vertices and edges, respectively.
2. The second line of each test case contains E space-separated pairs u and v, denoting that there is an edge from u to v.

Output Format

For each test case, the output prints the BFS order of the graph vertices.

Each vertex should appear once in the order it is visited, separated by a space.

vtu29890

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 2
5 4
0 1 0 2 0 3 2 4
3 2
0 1 0 2

Output: 0 1 2 3 4
0 1 2

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define MAX 26

void LexiBFS(char G[MAX][MAX], char S, bool vis[MAX], int N) {
    char queue[MAX];
    int front = 0, rear = 0;
    queue[rear++] = S;
    vis[S - 'a'] = true;

    while (front != rear) {
        char top = queue[front++];
        printf("%c ", top);

        char adj[MAX];
        int adj_count = 0;

        for (int i = 0; i < MAX; i++) {
            if (G[top - 'a'][i] && !vis[i]) {
                adj[adj_count++] = 'a' + i;
            }
        }

        if (adj_count > 0) {
            for (int i = 0; i < adj_count - 1; i++) {
                for (int j = i + 1; j < adj_count; j++) {
                    if (adj[i] > adj[j]) {
```

```
                char temp = adj[i];
                adj[i] = adj[j];
                adj[j] = temp;
            }
        }
    }

    for (int i = 0; i < adj_count; i++) {
        vis[adj[i] - 'a'] = true;
        queue[rear++] = adj[i];
    }
  }
}
}

void CreateGraph(int N, int M, char S, char Edges[MAX][2]) {
    char G[MAX][MAX] = {0};
    bool vis[MAX] = {0};

    for (int i = 0; i < M; i++) {
        G[Edges[i][0] - 'a'][Edges[i][1] - 'a'] = 1;
    }

    LexiBFS(G, S, vis, N);
}

int main() {
    int N, M;
    char S;

    scanf("%d %d", &N, &M);

    char Edges[MAX][2];
    for (int i = 0; i < M; i++) {
        scanf(" %c %c", &Edges[i][0], &Edges[i][1]);
    }

    scanf(" %c", &S);
    CreateGraph(N, M, S, Edges);

    return 0;
}
```

Status : Wrong

Marks : 0/10

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 8_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 0

Section 1 : MCQ

1. The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one of the following is a possible order of visiting the nodes in the graph below?

Answer

Status : Skipped ***Marks : 0/1***

2. Consider the following graph.

Apply breadth-first search traversal of the above graph. Which of the following traversal is possible if the start vertex is G? (Assume lexicographic ordering)

vtu29890

Answer

-

Status : - *Marks : 0/1*

3. Which of the following statements is true about DFS?

Answer

-

Status : - *Marks : 0/1*

4. In the _____ traversal, we process all of a vertex's descendants before we move to an adjacent vertex.

Answer

-

Status : - *Marks : 0/1*

5. Which of the following conditions is sufficient to detect a cycle in a directed graph?

Answer

-

Status : - *Marks : 0/1*

6. In an adjacency list representation of a graph, what does each node in the list represent?

Answer

-

Status : - *Marks : 0/1*

vtu29890

7. Regarding the implementation of Breadth First Search using queues, what is the maximum distance between two nodes present in the queue? (considering each edge length 1)

Answer

-

Status : - ***Marks : 0/1***

8. Which of the following problems can be solved by using both BFS & DFS?

Answer

-

Status : - ***Marks : 0/1***

9. The number of elements in the adjacency matrix of a graph having 7 vertices is _____.

Answer

-

Status : - ***Marks : 0/1***

10. Given the following adjacency matrix of a graph(G) determine the number of components in the G.

[0 1 1 0 0 0],

[1 0 1 0 0 0],

[1 1 0 0 0 0],

[0 0 0 0 1 0],

[0 0 0 1 0 0],

[0 0 0 0 0 0].

vtu29890

Answer

-

Status : - *Marks : 0/1*

11. What will be the result of depth-first traversal in the following tree?

Answer

-

Status : - *Marks : 0/1*

12. Which of the following is a possible result of depth-first traversal of the given graph(consider 1 to be the source element)?

Answer

-

Status : - *Marks : 0/1*

13. Given two vertices in a graph s and t, which of the two traversals (BFS and DFS) can be used to find if there is path from s to t?

Answer

-

Status : - *Marks : 0/1*

14. In BFS, what happens when a node is visited for the second time?

Answer

-

Status : - *Marks : 0/1*

vtu29890

15. Which data structure is used in Breadth First Search?

Answer

-

Status : ***-*** ***Marks : 0/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 8_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Daniel is working on a navigation system for a city, where each intersection is represented as a vertex, and the roads between them are edges. He needs to calculate the shortest path between two intersections using the Breadth-First Search (BFS) algorithm.

Given the adjacency matrix of the city's road network, Daniel needs to find the shortest path from a source intersection to a destination intersection. If there are multiple shortest paths, the path with the smaller indexed intersections should be preferred.

Can you assist Daniel in implementing the solution for this problem?

Example

Input:

5

0 1 1 0 0

1 0 1 0 0

1 1 0 1 0

0 0 1 0 1

0 0 0 1 0

0 4

Output:

3

0 2 3 4

Explanation:

The graph has 5 vertices (0 to 4).

The adjacency matrix represents an undirected graph where 1 indicates an edge between two vertices.

The starting vertex is 0, and the ending vertex is 4.

The BFS starts from vertex 0, exploring its neighbors 1 and 2.

BFS continues to vertex 2, discovering vertex 3.

From vertex 3, it reaches the destination vertex 4.

The shortest path found is 0 2 3 4, with a length of 3.

The program prints the path length (3) and the path itself (0 2 3 4).

Input Format

The first line contains an integer N, the number of intersections (vertices) in the city.

The next N lines each contain N integers, representing the adjacency matrix. If there is a road between intersection i and intersection j, the matrix entry will be 1, otherwise 0.

The last line contains two integers, source and goal, representing the source and destination intersections (indexed from 0).

Output Format

The first line should print the length of the shortest path from the source to the goal intersection.

The second line should print the path from the source to the goal, including both the source and the goal.

If the source or goal index is invalid, print "Not found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0 1 1 0 0
1 0 1 0 0
1 1 0 1 0
0 0 1 0 1
0 0 0 1 0
0 4

Output: 3
0 2 3 4

Answer

```
#include <stdio.h>

int graph[105][105], q[105], prev[105], visited[105], front = 0, rear = 0;

int bfs(int vertex, int totalVertices, int endVertex){
   while (front <= rear){
      visited[vertex] = 1;
      for (int i = 1; i <= totalVertices; i++){
         if (graph[vertex][i] == 1 && visited[i] == 0){
            visited[i] = 1;
```

```
            prev[i] = vertex;
            q[rear] = i;
            rear++;
        }
    }
    vertex = q[front];
    front++;
    if (vertex == endVertex){
        return 1;
    }
  }
  return 0;
}

int main(){
    int totalVertices, startingVertex, endingVertex;
    scanf("%d", &totalVertices);
    for (int i = 1; i <= totalVertices; i++){ // O(totalVertices)
        prev[i] = -1;
        visited[i] = 0;
    }
    for (int i = 1; i <= totalVertices; i++){ // O(totalVertices^2)
        for (int j = 1; j <= totalVertices; j++){
            scanf("%d", &graph[i][j]);
        }
    }
    scanf("%d %d", &startingVertex, &endingVertex);
    startingVertex += 1;
    endingVertex += 1;
    int flag = bfs(startingVertex, totalVertices, endingVertex); //
O(totalVertices^2)
    int path[105] = {0};
    int ctr = 0;
    if (flag == 1){
        while (prev[endingVertex] != -1){
            path[ctr] = endingVertex;
            endingVertex = prev[endingVertex];
            ctr++;
        }
        printf("%d\n", ctr);
        printf("%d", startingVertex - 1);
        for (int i = ctr - 1; i >= 0; i--){
```

```
            printf(" %d", path[i] - 1);
        }
        puts("");
    }
    else {
        printf("Not found\n");
    }


    return 0;
}


#include <stdio.h>

int graph[105][105], q[105], prev[105], visited[105], front = 0, rear = 0;

int bfs(int vertex, int totalVertices, int endVertex) {
    while (front <= rear) {
        visited[vertex] = 1;
        for (int i = 1; i <= totalVertices; i++) {
            if (graph[vertex][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                prev[i] = vertex;
                q[rear++] = i;
            }
        }
        vertex = q[front++];
        if (vertex == endVertex) {
            return 1;
        }
    }
    return 0;
}

// Divide and Conquer approach to print path
void printPath(int start, int current) {
    if (start == current) {
        printf("%d", start - 1); // adjust for 0-based
        return;
    }
    printPath(start, prev[current]); // divide
```

```
    printf(" %d", current - 1);      // conquer
}

int main() {
    int totalVertices, startingVertex, endingVertex;
    scanf("%d", &totalVertices);

    for (int i = 1; i <= totalVertices; i++) {
        prev[i] = -1;
        visited[i] = 0;
    }

    for (int i = 1; i <= totalVertices; i++) {
        for (int j = 1; j <= totalVertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    scanf("%d %d", &startingVertex, &endingVertex);
    startingVertex += 1;
    endingVertex += 1;

    q[rear++] = startingVertex;
    int found = bfs(startingVertex, totalVertices, endingVertex);

    if (found) {
        // Count path length
        int count = 0, temp = endingVertex;
        while (temp != startingVertex) {
            count++;
            temp = prev[temp];
        }
        printf("%d\n", count);
        printPath(startingVertex, endingVertex); // uses divide & conquer
        printf("\n");
    } else {
        printf("Not found\n");
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Ravi is developing a network simulator that checks if a given undirected graph contains a cycle. His task is to build a program that determines whether any loop exists in the graph and also counts the number of nodes visited during the cycle detection process.

Write a program to ensure that the task is done by printing whether a cycle exists (1 or 0) and reporting the total number of recursive visits during traversal.

Input Format

The first line of the input consists of an integer V, representing the number of vertices.

The second line of the input consists of an integer E, representing the number of edges.

The next E lines each contain two space-separated integers u and v, representing an edge between vertices u and v.

Output Format

The first line prints 1 if the graph contains a cycle, else it prints 0

The second line prints "Number of nodes visited: X" where X is the number of nodes visited during the cycle detection function.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
3

0 1
1 2
2 0
Output: 1
Number of nodes visited: 3

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct LinkedList {
    struct Node* head;
};

struct Graph {
    int V;
    struct LinkedList* adj;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int V) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->adj = (struct LinkedList*)malloc(V * sizeof(struct LinkedList));

    for (int i = 0; i < V; ++i)
        graph->adj[i].head = NULL;

    return graph;
```

```
}

void addEdge(struct Graph* graph, int v, int w) {
    struct Node* newNode = createNode(w);
    newNode->next = graph->adj[v].head;
    graph->adj[v].head = newNode;

    newNode = createNode(v);
    newNode->next = graph->adj[w].head;
    graph->adj[w].head = newNode;
}

int visitCount = 0;

int isCyclicUtil(int v, int visited[], int parent, struct Graph* graph) {
    visitCount++;
    visited[v] = 1;

    struct Node* temp = graph->adj[v].head;
    while (temp != NULL) {
        int neighbor = temp->data;

        if (!visited[neighbor]) {
            if (isCyclicUtil(neighbor, visited, v, graph))
                return 1;
        } else if (neighbor != parent) {
            return 1;
        }

        temp = temp->next;
    }

    return 0;
}

int isCyclic(struct Graph* graph) {
    int* visited = (int*)malloc(graph->V * sizeof(int));
    for (int i = 0; i < graph->V; ++i)
        visited[i] = 0;

    for (int u = 0; u < graph->V; ++u) {
        if (!visited[u]) {
```

```
            if (isCyclicUtil(u, visited, -1, graph))
                return 1;
        }
    }

    free(visited);
    return 0;
}

int main() {
    int vert, edges, i, ele1, ele2;
    scanf("%d %d", &vert, &edges);

    struct Graph* g1 = createGraph(vert);

    for (i = 0; i < edges; i++) {
        scanf("%d %d", &ele1, &ele2);
        addEdge(g1, ele1, ele2);
    }

    int hasCycle = isCyclic(g1);

    if (hasCycle)
        printf("1\n");
    else
        printf("0\n");

    printf("Number of nodes visited: %d\n", visitCount);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Sarah is working on a traffic management system for a city's road network. She needs to find the shortest cycle in the road network, where each intersection is represented as a vertex and each road between intersections is a directed edge with a specific travel time (weight). The

goal is to identify the shortest cycle in the road network, where a cycle is defined as a path that starts and ends at the same intersection, and the length of the cycle is the total travel time of all roads in the cycle.

Can you assist Sarah by writing a program to find the length of the shortest cycle in the city's road network?

Example1

Input:

4

4

0 1 2

1 2 3

2 0 4

3 1 5

Output:

Length of the shortest cycle: 9

Explanation:

After exploring all starting vertices, the code checks for cycles by examining the distances. If an edge leads back to the starting vertex, it means there is a cycle. In this case, edge 1 (vertex 1) leads back to the starting vertex 0. The code calculates the length of the cycle by adding the distances of the vertices along the cycle: 2 (0 -> 1) + 3 (1 -> 2) + 4 (2 -> 0) = 9.

Therefore, the output is "Length of the shortest cycle: 9".

Example2

Input:

3

2

0 1 2

1 2 3

Output:

No cycle was found in the graph.

Explanation:

After exploring all starting vertices, the code checks for cycles by examining the distances. If an edge leads back to the starting vertex, it means there is a cycle. However, in this case, there are no edges that lead back to the starting vertex. Hence, there is no cycle in the graph.

Therefore, the output is "No cycle found in the graph."

Input Format

The first line contains an integer v, representing the number of intersections (vertices) in the city's road network.

The second line contains an integer e, representing the number of roads (edges) in the city.

The next e lines each contain three integers: source, destination, and weight, representing a directed road from intersection source to intersection destination with a travel time (weight).

Output Format

If a cycle exists, the output prints "Length of the shortest cycle: X", where X is the length of the shortest cycle.

If no cycle exists, then the output prints "No cycle found in the graph."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
4

vtu29890

0 1 2
1 2 3
2 0 4
3 1 5

Output: Length of the shortest cycle: 9

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define INF INT_MAX

struct Edge {
    int vertex;
    int weight;
};

int shortestCycleLength(struct Edge** graph, int vertices) {
    int minCycleLength = INF;

    for (int start = 0; start < vertices; start++) {
        int* distance = (int*)malloc(vertices * sizeof(int));
        for (int i = 0; i < vertices; i++) {
            distance[i] = INF;
        }

        int* visited = (int*)malloc(vertices * sizeof(int));
        for (int i = 0; i < vertices; i++) {
            visited[i] = 0;
        }

        int* queue = (int*)malloc(vertices * sizeof(int));
        int front = 0, rear = 0;

        distance[start] = 0;
        visited[start] = 1;
        queue[rear++] = start;

        while (front != rear) {
```

```
        int curr = queue[front++];

        for (int i = 0; i < vertices; i++) {
            if (graph[curr][i].weight != INF) {
                int neighbor = graph[curr][i].vertex;
                int weight = graph[curr][i].weight;

                if (!visited[neighbor]) {
                    visited[neighbor] = 1;
                    distance[neighbor] = distance[curr] + weight;
                    queue[rear++] = neighbor;
                } else if (neighbor == start) {
                    minCycleLength = minCycleLength < distance[curr] + weight ?
minCycleLength : distance[curr] + weight;
                }
            }
        }
    }

    free(distance);
    free(visited);
    free(queue);
  }

  return minCycleLength;
}

int main() {
  int vertices, edges;
  scanf("%d", &vertices);
  scanf("%d", &edges);

  struct Edge** graph = (struct Edge**)malloc(vertices * sizeof(struct Edge*));
  for (int i = 0; i < vertices; i++) {
    graph[i] = (struct Edge*)malloc(vertices * sizeof(struct Edge));
    for (int j = 0; j < vertices; j++) {
        graph[i][j].vertex = j;
        graph[i][j].weight = INF;
    }
  }

  for (int i = 0; i < edges; i++) {
```

```
        int source, destination, weight;
        scanf("%d %d %d", &source, &destination, &weight);
        graph[source][destination].weight = weight;
    }

    int shortestCycle = shortestCycleLength(graph, vertices);

    if (shortestCycle == INF) {
        printf("No cycle found in the graph.\n");
    } else {
        printf("Length of the shortest cycle: %d\n", shortestCycle);
    }

    // Free allocated memory
    for (int i = 0; i < vertices; i++) {
        free(graph[i]);
    }
    free(graph);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Maya is developing a route planning system for a large theme park. Each attraction in the park is represented as a vertex in a directed graph, and each pathway connecting attractions is a directed edge. To help visitors explore all possible paths between two attractions, Maya needs a function that finds all possible routes from a given starting attraction to a destination attraction using Depth-First Search (DFS) traversal.

Write a program to help Maya list all such possible routes from source to destination.

Input Format

The first line contains two space-separated integers V and E, representing the number of attractions and roads in the city.

Each of the next E lines contains two integers, u and v, separated by a space, representing a road from landmark u to attractions v.

The last line contains two integers, s, and d, separated by a space, representing the source attraction and the destination attraction. for which routes need to be found.

Output Format

The output prints all the possible routes from the source attraction to the destination attraction, each route on a separate line.

Each route should be represented as a space-separated sequence of attractions visited in the order of traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4 6
0 1
0 2
0 3
2 0
2 1
1 3
2 3

Output: 2 0 1 3
2 0 3
2 1 3

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_VERTICES 100

struct Graph {
```

```
    int V;
    int adj[MAX_VERTICES][MAX_VERTICES];
};

void addEdge(struct Graph* graph, int u, int v) {
    graph->adj[u][v] = 1;  // Directed edge
}

void printAllPathsUtil(struct Graph* graph, int u, int d, bool visited[], int path[], int
pathIndex) {
    visited[u] = true;
    path[pathIndex] = u;
    pathIndex++;

    if (u == d) {
        for (int i = 0; i < pathIndex; i++) {
            printf("%d", path[i]);
            if (i < pathIndex - 1) printf(" ");
        }
        printf("\n");
    } else {
        for (int i = 0; i < graph->V; i++) {
            if (graph->adj[u][i] == 1 && !visited[i]) {
                printAllPathsUtil(graph, i, d, visited, path, pathIndex);
            }
        }
    }

    visited[u] = false;  // Backtrack
}

void printAllPaths(struct Graph* graph, int s, int d) {
    bool visited[MAX_VERTICES] = {false};
    int path[MAX_VERTICES];
    printAllPathsUtil(graph, s, d, visited, path, 0);
}

int main() {
    int V, E;
    scanf("%d %d", &V, &E);

    struct Graph g;
```

```
    g.V = V;

    // Initialize adjacency matrix
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            g.adj[i][j] = 0;

    // Read edges
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(&g, u, v);
    }

    int s, d;
    scanf("%d %d", &s, &d);

    printAllPaths(&g, s, d);

    return 0;
}

#include <stdio.h>
#include <stdlib.h>

// Function to perform DFS and find all paths from source to destination
void findAllPaths(int **adjMatrix, int *visited, int *path, int pathIndex,
                  int current, int destination, int V) {

    // Add current vertex to the path
    path[pathIndex] = current;
    pathIndex++;

    // If we reached the destination, print the path
    if (current == destination) {
        for (int i = 0; i < pathIndex; i++) {
            printf("%d", path[i]);
            if (i < pathIndex - 1) {
                printf(" ");
            }
        }
        printf("\n");
```

```
        return;
    }

    // Mark current vertex as visited
    visited[current] = 1;

    // Explore all adjacent vertices
    for (int i = 0; i < V; i++) {
        if (adjMatrix[current][i] == 1 && !visited[i]) {
            findAllPaths(adjMatrix, visited, path, pathIndex, i, destination, V);
        }
    }

    // Backtrack: unmark current vertex as visited for other paths
    visited[current] = 0;
}

int main() {
    int V, E;

    // Read number of vertices (attractions) and edges (roads)
    scanf("%d %d", &V, &E);

    // Dynamically allocate memory for adjacency matrix
    int **adjMatrix = (int **)malloc(V * sizeof(int *));
    for (int i = 0; i < V; i++) {
        adjMatrix[i] = (int *)calloc(V, sizeof(int));
    }

    // Read edges and build adjacency matrix for directed graph
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adjMatrix[u][v] = 1; // Only one direction for directed graph
    }

    // Read source and destination
    int source, destination;
    scanf("%d %d", &source, &destination);

    // Initialize arrays for DFS
    int *visited = (int *)calloc(V, sizeof(int));
```

```
    int *path = (int *)malloc(V * sizeof(int));

    // Find all paths from source to destination
    findAllPaths(adjMatrix, visited, path, 0, source, destination, V);

    // Free allocated memory
    for (int i = 0; i < V; i++) {
        free(adjMatrix[i]);
    }
    free(adjMatrix);
    free(visited);
    free(path);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 9_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Mythili has a list of item prices and a customer's gift card balance. She needs to find a combination of two or more items whose total price exactly matches the gift card balance. If such a combination exists, she should print the item prices in the order they appear. If multiple valid combinations exist, she must print the first one found. If no valid combination is possible, print "No combination of items"

Write a function that takes the list of item prices, the number of items, and the gift card balance as input, and prints the result based on the above rules.

Input Format

The first line of input consists of an integer, representing the number of items in

the store.

The second line consists of an array of integers separated by a space, representing the prices of the items.

The last line consists of an integer representing the customer's gift card balance.

Output Format

If there exists a combination of items whose prices sum up to the gift card balance, then print this combination.

If no such combination exists, print "No combination of items".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
50
Output: 20 30

Answer

```
#include <stdio.h>
#include <string.h>

// Swap function
void swap(char *a, char *b) {
    char temp = *a;
    *a = *b;
    *b = temp;
}

// Global variable to store the maximum number found
char maxNum[31];

// Function to compare and update maximum number
void updateMax(char str[]) {
    if (strcmp(str, maxNum) > 0) {
```

```
        strcpy(maxNum, str);
    }
}

// Backtracking function
void findMax(char str[], int k, int n, int index) {
    // Base case: no swaps left or reached end
    if (k == 0 || index == n) {
        updateMax(str);
        return;
    }

    char maxDigit = str[index];
    // Find the maximum digit from index to end
    for (int i = index + 1; i < n; i++) {
        if (str[i] > maxDigit) {
            maxDigit = str[i];
        }
    }

    // If current digit is already the max, just move forward
    if (maxDigit != str[index]) k--;

    for (int i = n - 1; i >= index; i--) {
        if (str[i] == maxDigit) {
            // Swap to bring max digit forward
            swap(&str[index], &str[i]);

            // Recurse for next index
            findMax(str, k, n, index + 1);

            // Backtrack
            swap(&str[index], &str[i]);
        }
    }

    // If no swap done at this index, just move forward
    if (maxDigit == str[index]) {
        findMax(str, k, n, index + 1);
    }
}
```

```
int main() {
    int K;
    char str[31];

    scanf("%d", &K);
    scanf("%s", str);

    strcpy(maxNum, str);  // Initialize maxNum as the original number
    int n = strlen(str);

    findMax(str, K, n, 0);

    printf("%s\n", maxNum);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Imagine you're helping a friend plan gift packages with a total budget of N. Your task is to find all unique ways to spend the entire budget on different combinations of gift prices, ensuring no combination is repeated by rearranging the prices.

Note:

By unique, it means that no matter that no other composition can be expressed as a permutation of the generated composition. For eg. [1,2,1] and [1, 1, 2] are not unique.You need to print all combinations in non-decreasing order, for eg. [1, 2, 1] or [1,1,2] will be printed as [1,1,2].The order of printing all the combinations should be in lexicographical order.

Use backtracking to solve the given problem.

Input Format

The input consists of a single integer, n, which represents the number to be broken down.

Output Format

The output consists of a list of lists, where each inner list represents a possible breakdown of the given number.

Each breakdown is represented as a list of integers that add up to the given number.

The breakdowns are printed in lexicographic order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
Output: [[1]]

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 25

typedef struct {
   int** combinations;
   int* sizes;
   int count;
   int capacity;
} Result;

int* g_sizes;

Result* createResult() {
   Result* result = (Result*)malloc(sizeof(Result));
   result->capacity = 100;
   result->count = 0;
   result->combinations = (int**)malloc(result->capacity * sizeof(int*));
   result->sizes = (int*)malloc(result->capacity * sizeof(int));
   return result;
}
```

```
void addCombination(Result* result, int* temp, int size) {
    if (result->count >= result->capacity) {
        result->capacity *= 2;
        result->combinations = (int**)realloc(result->combinations,
                                    result->capacity * sizeof(int*));
        result->sizes = (int*)realloc(result->sizes,
                                result->capacity * sizeof(int));
    }

    result->combinations[result->count] = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        result->combinations[result->count][i] = temp[i];
    }
    result->sizes[result->count] = size;
    result->count++;
}

void findCombinations(int n, int start, int* temp, int size, Result* result) {
    if (n == 0) {
        addCombination(result, temp, size);
        return;
    }

    for (int i = start; i <= n; i++) {
        temp[size] = i;
        findCombinations(n - i, i, temp, size + 1, result);
    }
}

int compareCombinations(const void* a, const void* b) {
    int* arr1 = *(int**)a;
    int* arr2 = *(int**)b;

    int size1 = g_sizes[arr1 - *(int**)a];
    int size2 = g_sizes[arr2 - *(int**)b];

    int min_size = size1 < size2 ? size1 : size2;

    for (int i = 0; i < min_size; i++) {
        if (arr1[i] != arr2[i]) {
            return arr1[i] - arr2[i];
        }
```

```
    }
    return size1 - size2;
}

void sortCombinations(Result* result) {
    g_sizes = result->sizes;
    qsort(result->combinations, result->count, sizeof(int*),
compareCombinations);
}

void printResult(Result* result) {
    printf("[");
    for (int i = 0; i < result->count; i++) {
        printf("[");
        for (int j = 0; j < result->sizes[i]; j++) {
            printf("%d", result->combinations[i][j]);
            if (j < result->sizes[i] - 1) {
                printf(", ");
            }
        }
        printf("]");
        if (i < result->count - 1) {
            printf(", ");
        }
    }
    printf("]\n");
}

void freeResult(Result* result) {
    for (int i = 0; i < result->count; i++) {
        free(result->combinations[i]);
    }
    free(result->combinations);
    free(result->sizes);
    free(result);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        printf("Error reading input\n");
```

```
        return 1;
    }

    int* temp = (int*)malloc(MAX_SIZE * sizeof(int));
    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    Result* result = createResult();
    if (result == NULL) {
        free(temp);
        printf("Memory allocation failed\n");
        return 1;
    }

    findCombinations(n, 1, temp, 0, result);
    sortCombinations(result);
    printResult(result);

    free(temp);
    freeResult(result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a number K and string str of digits denoting a positive integer, build the largest number possible by performing swap operations on the digits of str at most K times.

Input Format

The first line of input consists of a positive integer K.

The second line of input consists of a string of digits of positive integers.

Output Format

The output displays the largest number possible by performing swap operations on the digits of str at most K times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
254

Output: 524

Answer

```
#include <stdio.h>
#include <string.h>

// Swap function
void swap(char *a, char *b) {
   char temp = *a;
   *a = *b;
   *b = temp;
}

// Global variable to store the maximum number found
char maxNum[31];

// Function to compare and update maximum number
void updateMax(char str[]) {
   if (strcmp(str, maxNum) > 0) {
      strcpy(maxNum, str);
   }
}

// Backtracking function
void findMax(char str[], int k, int n, int index) {
   // Base case: no swaps left or reached end
   if (k == 0 || index == n) {
      updateMax(str);
```

```
        return;
    }

    char maxDigit = str[index];
    // Find the maximum digit from index to end
    for (int i = index + 1; i < n; i++) {
        if (str[i] > maxDigit) {
            maxDigit = str[i];
        }
    }

    // If current digit is already the max, just move forward
    if (maxDigit != str[index]) k--;

    for (int i = n - 1; i >= index; i--) {
        if (str[i] == maxDigit) {
            // Swap to bring max digit forward
            swap(&str[index], &str[i]);

            // Recurse for next index
            findMax(str, k, n, index + 1);

            // Backtrack
            swap(&str[index], &str[i]);
        }
    }

    // If no swap done at this index, just move forward
    if (maxDigit == str[index]) {
        findMax(str, k, n, index + 1);
    }
}

int main() {
    int K;
    char str[31];

    scanf("%d", &K);
    scanf("%s", str);

    strcpy(maxNum, str);  // Initialize maxNum as the original number
    int n = strlen(str);
```

```
    findMax(str, K, n, 0);

    printf("%s\n", maxNum);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given an array of integers and a sum B, find all unique combinations in the array where the sum is equal to B. The same number may be chosen from the array any number of times to make B.

1. All numbers will be positive integers.
2. Elements in a combination (a1, a2, …, ak) must be in non-descending order. (ie, a1 ≤ a2 ≤ … ≤ ak).
3. The combinations themselves must be sorted in ascending order.

Examples:

Input:

arr[] = {7,2,6,5}

B = 16

Output:

(2 2 2 2 2 2 2 2)

(2 2 2 2 2 6)

(2 2 2 5 5)

(2 2 5 7)

(2 2 6 6)

(2 7 7)

(5 5 6)

Input:

arr[] = {6,5,7,1,8,2,9,9,7,7,9}

B = 6

Output:

(1 1 1 1 1 1)

(1 1 1 1 2)

(1 1 2 2)

(1 5)

(2 2 2)

(6)

Input Format

The first line of input consists of space-separated positive integers representing the elements of the array.

The second line consists of a single positive integer representing the target sum B.

Output Format

The output prints "Combinations that sum to [B] are:" followed by the combinations in parentheses, each combination separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7 2 6 5
16
Output: Combinations that sum to 16 are:
(2 2 2 2 2 2 2 2) (2 2 2 2 2 6) (2 2 2 5 5) (2 2 5 7) (2 2 6 6) (2 7 7) (5 5 6)

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_COMB_SIZE 25
#define MAX_ARR_SIZE 100
typedef struct {
    int** combinations;
    int* combSizes;
    int count;
    int capacity;
} Result;

Result* createResult() {
    Result* result = (Result*)malloc(sizeof(Result));
    result->capacity = 100;
    result->count = 0;
    result->combinations = (int**)malloc(result->capacity * sizeof(int*));
    result->combSizes = (int*)malloc(result->capacity * sizeof(int));
    return result;
}

int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

void addCombination(Result* result, int* tempList, int tempSize) {
    if (result->count >= result->capacity) {
        result->capacity *= 2;
        result->combinations = (int**)realloc(result->combinations,
                                    result->capacity * sizeof(int*));
        result->combSizes = (int*)realloc(result->combSizes,
                                  result->capacity * sizeof(int));
    }
    result->combinations[result->count] = (int*)malloc(tempSize * sizeof(int));
    memcpy(result->combinations[result->count], tempList, tempSize *
sizeof(int));
    result->combSizes[result->count] = tempSize;
    result->count++;
}
```

```
void findCombinations(Result* result, int* arr, int n, int sum,
                      int* tempList, int tempSize, int start) {
    if (sum < 0) return;
    if (sum == 0) {
        addCombination(result, tempList, tempSize);
        return;
    }
    for (int i = start; i < n; i++) {
        if (i > start && arr[i] == arr[i-1]) continue;
        tempList[tempSize] = arr[i];
        findCombinations(result, arr, n, sum - arr[i],
                         tempList, tempSize + 1, i);
    }
}

void freeResult(Result* result) {
    for (int i = 0; i < result->count; i++) {
        free(result->combinations[i]);
    }
    free(result->combinations);
    free(result->combSizes);
    free(result);
}
int main() {
    char input[1000];
    int arr[MAX_ARR_SIZE], n = 0;
    int target;

    fgets(input, sizeof(input), stdin);
    char* token = strtok(input, " \n");
    while (token != NULL && n < MAX_ARR_SIZE) {
        arr[n++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    scanf("%d", &target);
    qsort(arr, n, sizeof(int), compare);

    int* tempList = (int*)malloc(MAX_COMB_SIZE * sizeof(int));
    Result* result = createResult();

    findCombinations(result, arr, n, target, tempList, 0, 0);
```

```
    printf("Combinations that sum to %d are:\n", target);
    for (int i = 0; i < result->count; i++) {
        printf("( ");
        for (int j = 0; j < result->combSizes[i]; j++) {
            printf("%d ", result->combinations[i][j]);
        }
        printf(") ");
    }
    printf("\n");

    free(tempList);
    freeResult(result);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Rohan is designing a security system that generates all possible rearrangements of a given access code made up of distinct characters. He inputs a string representing the access code into the system. The system must display every possible permutation of the characters without repetition.

Write a program to ensure that all permutations of the given string are printed in separate lines.

Input Format

The input consists of a single line containing a string S, consisting of lowercase or upper-case alphabets.

Output Format

The output prints each permutation of the string on a new line.

The permutations should be printed in the order generated by swapping characters starting from the first position

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: ABC

Output: ABC
ACB
BAC
BCA
CBA
CAB

Answer

```
#include <stdio.h>
#include <string.h>

// Function to swap two characters
void swap(char *a, char *b) {
    char temp = *a;
    *a = *b;
    *b = temp;
}

// Recursive function to generate permutations
void permute(char str[], int left, int right) {
    if (left == right) {
        printf("%s\n", str);
        return;
    }

    for (int i = left; i <= right; i++) {
        // Swap current index with left
        swap(&str[left], &str[i]);

        // Recur for remaining substring
        permute(str, left + 1, right);

        // Backtrack (restore original order)
```

```
        swap(&str[left], &str[i]);
    }
}

int main() {
    char str[6];  // Max length is 5 + 1 for null character
    scanf("%s", str);

    int n = strlen(str);

    permute(str, 0, n - 1);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Given an undirected graph and a number m, determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

Note: Here, the coloring of a graph means the assignment of colors to all vertices.

Example

Input:

0 1 1 1

1 0 1 0

vtu29890

1 1 0 1

1 0 1 0

3

Output:

Solution Exists:

1 2 3 2

Explanation:

A minimum of 3 colors is required for the above graph.

Input Format

The input consists of four lines each containing four space-separated integers (1 if connected, 0 if not), representing the adjacency matrix of a graph where each element represents the connection between vertices.

The last line consists of an integer m, representing the number of colors available.

Output Format

If a solution exists:

1. The first line prints "Solution Exists:".
2. The second line prints the assigned colors for each vertex, separated by a space.

If no solution exists, the output prints "Solution does not exist".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 1 1 1

1 0 1 0
1 1 0 1
1 0 1 0
3
Output: Solution Exists:
1 2 3 2

Answer

```
#include <stdio.h>

#define MAX 4

struct MColoringProblem {
    int V;
    int color[MAX];
};

void printSolution(struct MColoringProblem* coloring) {
    printf("Solution Exists:\n");
    for (int i = 0; i < coloring->V; i++) {
        printf("%d ", coloring->color[i]);
    }
    printf("\n");
}

int isSafe(int v, int graph[MAX][MAX], int color[MAX], int c, int V) {
    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1 && c == color[i]) {
            return 0;
        }
    }
    return 1;
}

int graphColoringUtil(int graph[MAX][MAX], int m, int color[MAX], int v, int V) {
    if (v == V) {
        return 1;
    }
    for (int c = 1; c <= m; c++) {
        if (isSafe(v, graph, color, c, V)) {
            color[v] = c;
```

```
            if (graphColoringUtil(graph, m, color, v + 1, V)) {
                return 1;
            }
            color[v] = 0;
        }
    }
    return 0;
}

void graphColoring(struct MColoringProblem* coloring, int graph[MAX][MAX], int
m) {
    coloring->V = MAX;
    if (!graphColoringUtil(graph, m, coloring->color, 0, coloring->V)) {
        printf("Solution does not exist\n");
        return;
    }
    printSolution(coloring);
}

int main() {
    struct MColoringProblem coloring;
    int graph[MAX][MAX];
    int m;

    for (int i = 0; i < MAX; i++) {
        for (int j = 0; j < MAX; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    scanf("%d", &m);

    graphColoring(&coloring, graph, m);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a partially filled 9x9 2D array, the goal is to assign digits (from 1 to 9) to the empty cells so that every row, column, and sub-grid of size 3x3 contains exactly one instance of the digits from 1 to 9.

Write a program for the same.

Example

Input:

3 0 6 5 0 8 4 0 0

5 2 0 0 0 0 0 0 0

0 8 7 0 0 0 0 3 1

0 0 3 0 1 0 0 8 0

9 0 0 8 6 3 0 0 5

0 5 0 0 9 0 6 0 0

1 3 0 0 0 0 2 5 0

0 0 0 0 0 0 0 7 4

0 0 5 2 0 6 3 0 0

Output:

3 1 6 5 7 8 4 9 2

5 2 9 1 3 4 7 6 8

4 8 7 6 2 9 5 3 1

2 6 3 4 1 5 9 8 7

9 7 4 8 6 3 1 2 5

8 5 1 7 9 2 6 4 3

1 3 8 9 4 7 2 5 6

6 9 2 3 5 1 8 7 4

7 4 5 2 8 6 3 1 9

Explanation

To solve the given Sudoku puzzle, we apply the fundamental rules of Sudoku: each row, column, and 3x3 subgrid must contain the digits 1 through 9 without any repetition. Starting with the given partially filled grid, we systematically fill in the empty cells. For each empty cell, we check the numbers already present in the corresponding row, column, and 3x3 subgrid to determine which number can legally go in that position. This process involves logical deduction and elimination. In the case of the provided puzzle, the empty cells are filled one by one, ensuring that each row, column, and subgrid adheres to the Sudoku rules. For example, in Row 1, the missing numbers (1, 7, 9, 2) are placed in the available spots based on their placement in the columns and subgrids. This continues for each row, progressively narrowing down the possibilities for the empty cells until the entire grid is complete. The final solution, after applying this reasoning, looks like the output grid:

Input Format

The input consists of a 9x9 matrix in 9 rows and 9 columns separated by space.

Output Format

The output displays the 9x9 matrix with the updated values.

If the conditions are not satisfied, then display "No Solution exists".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 0 6 5 0 8 4 0 0
5 2 0 0 0 0 0 0 0
0 8 7 0 0 0 0 3 1
0 0 3 0 1 0 0 8 0
9 0 0 8 6 3 0 0 5
0 5 0 0 9 0 6 0 0
1 3 0 0 0 0 2 5 0
0 0 0 0 0 0 0 7 4
0 0 5 2 0 6 3 0 0

Output: 3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define SIZE 9

// Function to print the Sudoku grid
void printBoard(int board[SIZE][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

// Function to check if placing num in board[row][col] is valid
bool isValid(int board[SIZE][SIZE], int row, int col, int num) {
    // Check the row
    for (int x = 0; x < SIZE; x++) {
        if (board[row][x] == num) {
            return false;
        }
    }

    // Check the column
    for (int x = 0; x < SIZE; x++) {
        if (board[x][col] == num) {
            return false;
        }
    }
```

```
    // Check the 3x3 sub-grid
    int startRow = row - row % 3;
    int startCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i + startRow][j + startCol] == num) {
                return false;
            }
        }
    }

    return true;
}

// Function to solve the Sudoku using backtracking
bool solveSudoku(int board[SIZE][SIZE]) {
    int row, col;
    bool empty = false;

    // Find the next empty cell
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++) {
            if (board[row][col] == 0) {
                empty = true;
                break;
            }
        }
        if (empty) {
            break;
        }
    }

    // If no empty cell is found, the Sudoku is solved
    if (!empty) {
        return true;
    }

    // Try all numbers from 1 to 9 for the empty cell
    for (int num = 1; num <= SIZE; num++) {
        if (isValid(board, row, col, num)) {
            board[row][col] = num;
```

```
            // Recursively attempt to solve the Sudoku
            if (solveSudoku(board)) {
                return true;
            }

            // If placing num doesn't lead to a solution, reset and try the next number
            board[row][col] = 0;
        }
    }

    return false; // Trigger backtracking if no valid number is found
}

int main() {
    int board[SIZE][SIZE];

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            scanf("%d", &board[i][j]);
        }
    }

    // Solve the Sudoku puzzle
    if (solveSudoku(board)) {
        printBoard(board);
    } else {
        printf("No Solution exists\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be

moved.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

If solutions exist: Print each valid board configuration, where "Q" represents a queen and "." represents an empty cell.

Each row of the board should be space-separated.

Each configuration should be followed by a blank line.

If no configuration is possible, print "No solution".

Refer to the sample output for the formatting specification.

Sample Test Case

Input: 4
0

vtu29890

1
Output: . Q . .
. . . Q
Q . . .
. . Q .

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

#define MAX 15

typedef struct {
    int n;                // Size of the board
    int startRow, startCol; // Fixed position for the first queen
    int board[MAX][MAX];
    bool hasSolution;
} NQueens;

// Check if it's safe to place a queen at (row, col)
bool isSafe(NQueens* nq, int row, int col) {
    for (int r = 0; r < row; r++) {
        if (nq->board[r][col] == 1)
            return false;
    }

    for (int r = row - 1, c = col - 1; r >= 0 && c >= 0; r--, c--) {
        if (nq->board[r][c] == 1)
            return false;
    }

    for (int r = row - 1, c = col + 1; r >= 0 && c < nq->n; r--, c++) {
        if (nq->board[r][c] == 1)
            return false;
    }

    return true;
}
```

```
// Print current board configuration
void printBoard(NQueens* nq) {
    nq->hasSolution = true;
    for (int i = 0; i < nq->n; i++) {
        for (int j = 0; j < nq->n; j++) {
            printf("%s ", nq->board[i][j] ? "Q" : ".");
        }
        printf("\n");
    }
    printf("\n");
}

// Recursive utility to solve N-Queens
void solve(NQueens* nq, int row) {
    if (row == nq->n) {
        printBoard(nq);
        return;
    }

    for (int col = 0; col < nq->n; col++) {
        if (isSafe(nq, row, col)) {
            if (row == nq->startRow && col != nq->startCol)
                continue;

            nq->board[row][col] = 1;
            solve(nq, row + 1);
            nq->board[row][col] = 0;
        }
    }
}

void solveWithFixedQueen(NQueens* nq) {
    if (nq->startRow < 0 || nq->startRow >= nq->n || nq->startCol < 0 || nq->startCol
>= nq->n) {
        printf("Invalid queen position!\n");
        return;
    }

    nq->board[nq->startRow][nq->startCol] = 1;
    solve(nq, 0);

    if (!nq->hasSolution)
```

```
    printf("No solution\n");
}

int main() {
    int n, i, j;
    scanf("%d %d %d", &n, &i, &j);

    if (n > MAX) {
        printf("Board size too large (max %d)\n", MAX);
        return 1;
    }

    NQueens nq = {n, i, j, {0}, false};

    solveWithFixedQueen(&nq);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Consider a rat placed at (0, 0) in a square matrix of order N*N. It has to reach the destination at (N – 1, N – 1). Find all possible paths that the rat can take to reach from the source to the destination. The directions in which the rat can move are 'U'(up), 'D'(down), 'L' (left), and 'R' (right). Value 0 at a cell in the matrix represents that it is blocked and the rat cannot move to it while value 1 at a cell in the matrix represents that the rat can travel through it. Return the list of paths in lexicographically increasing order.

Note: In a path, no cell can be visited more than once. If the source cell is 0, the rat cannot move to any other cell.

Example 1

Input:

N = 4

m[][] =

{{1, 0, 0, 0},

{1, 1, 0, 1},

{1, 1, 0, 0},

{0, 1, 1, 1}}

Output:

DDRDRR DRDDRR

Explanation:

The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

Example 2

Input:

N = 2

m[][] =

{{1, 0},

{1, 0}}

Output:

-1

Explanation:

No path exists and the destination cell is blocked.

Input Format

The first line of input contains an integer n, representing the size of the maze.

The next n lines each contain n space-separated integers, representing the maze.

Output Format

vtu29890

The output consists of

If at least one path is found, print all valid paths as space-separated strings.

If no path is found, prints -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
1 0 0 0
1 1 0 1
1 1 0 0
0 1 1 1
Output: DDRDRR DRDDRR

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define SIZE 9

// Function to print the Sudoku grid
void printBoard(int board[SIZE][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

// Function to check if placing num in board[row][col] is valid
bool isValid(int board[SIZE][SIZE], int row, int col, int num) {
    // Check the row
    for (int x = 0; x < SIZE; x++) {
        if (board[row][x] == num) {
            return false;
```

```
        }
    }

    // Check the column
    for (int x = 0; x < SIZE; x++) {
        if (board[x][col] == num) {
            return false;
        }
    }

    // Check the 3x3 sub-grid
    int startRow = row - row % 3;
    int startCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i + startRow][j + startCol] == num) {
                return false;
            }
        }
    }

    return true;
}

// Function to solve the Sudoku using backtracking
bool solveSudoku(int board[SIZE][SIZE]) {
    int row, col;
    bool empty = false;

    // Find the next empty cell
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++) {
            if (board[row][col] == 0) {
                empty = true;
                break;
            }
        }
        if (empty) {
            break;
        }
    }
```

```
    // If no empty cell is found, the Sudoku is solved
    if (!empty) {
        return true;
    }

    // Try all numbers from 1 to 9 for the empty cell
    for (int num = 1; num <= SIZE; num++) {
        if (isValid(board, row, col, num)) {
            board[row][col] = num;

            // Recursively attempt to solve the Sudoku
            if (solveSudoku(board)) {
                return true;
            }

            // If placing num doesn't lead to a solution, reset and try the next number
            board[row][col] = 0;
        }
    }

    return false; // Trigger backtracking if no valid number is found
}

int main() {
    int board[SIZE][SIZE];

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            scanf("%d", &board[i][j]);
        }
    }

    // Solve the Sudoku puzzle
    if (solveSudoku(board)) {
        printBoard(board);
    } else {
        printf("No Solution exists\n");
    }

    return 0;
}
```

Status : Wrong

Marks : 0/10

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. In a Sudoku solver, when does the backtracking algorithm backtrack?

Answer

When no valid number can be placed in a cell

Status : Correct ***Marks : 1/1***

2. Backtracking may lead to a solution that is _____.

Answer

optimal

Status : Wrong ***Marks : 0/1***

vtu29890

3. In general, backtracking can be used to solve?

Answer

Combinatorial problems

Status : Correct ***Marks : 1/1***

4. Which of the following problems cannot be solved using backtracking?

Answer

Iteration with queues

Status : Correct ***Marks : 1/1***

5. What is the main idea behind Backtracking to check if a graph is connected?

Answer

Trying all possible paths from a starting vertex and marking visited vertices

Status : Correct ***Marks : 1/1***

6. Which data structure is commonly used in backtracking to store partial solutions?

Answer

Stack or List

Status : Correct ***Marks : 1/1***

7. Which of the following is essential for backtracking to function correctly?

Answer

A recursive call with a rollback (backtrack step)

Status : Correct ***Marks : 1/1***

vtu29890

8. Which of the following constraints are typically used in Sudoku backtracking?

Answer

All the mentioned options

Status : Correct ***Marks : 1/1***

9. In backtracking solutions for maze problems, when do we backtrack from a cell?

Answer

When all directions from the current cell are either visited or blocked

Status : Correct ***Marks : 1/1***

10. What is the primary technique used to solve the Rat in a Maze problem?

Answer

Backtracking

Status : Correct ***Marks : 1/1***

11. Which of the following best describes the backtracking approach to generating permutations of a string?

Answer

Fix one character, recursively permute the rest, and backtrack

Status : Correct ***Marks : 1/1***

12. What is a base case in a backtracking algorithm?

Answer

A condition when a solution is found or invalid

vtu29890

Status : Correct ***Marks : 1/1***

13. In the Hamiltonian Cycle problem using backtracking, what must be true for a path to be valid?

Answer

All vertices are included once, and the last connects to the first

Status : Correct ***Marks : 1/1***

14. In Backtracking, what is the base case for checking connectivity in an undirected graph?

Answer

All vertices are visited

Status : Correct ***Marks : 1/1***

15. In Backtracking, what is the purpose of marking vertices as visited during graph traversal?

Answer

To avoid revisiting the same vertex

Status : Correct ***Marks : 1/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 9_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

You are given a chessboard of size N x N. The objective is to place N queens on the board such that no two queens threaten each other (i.e., no two queens share the same row, column, or diagonal).

Additionally, the board may already have K queens pre-placed at valid positions. You must:

Verify that the initial placements are valid (i.e., not attacking each other).

Try to place the remaining N - K queens.

Print one valid arrangement if it exists.

Input Format

vtu29890

The first line of input contains two integers N and K, separated by a space:

N: Size of the chessboard (1 ≤ N ≤ 20)

K: Number of pre-placed queens (0 ≤ K ≤ N)

The next K lines each contain two integers r and c separated by a space:

These represent the row and column indices (0-based) of the pre-placed queens.

Output Format

If a valid solution exists, print the board as N lines, each containing N integers (0 or 1), where 1 represents a queen and 0 is an empty square.

If no solution is possible, print: "No solution found."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
1
0 1
Output: 0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

int board[MAX][MAX];
int n;
```

```
// Check if placing a queen at (row, col) is safe
int isSafe(int row, int col) {
    int i, j;

    // Check row
    for (i = 0; i < n; i++) {
        if (board[row][i] == 1 && i != col)
            return 0;
    }

    // Check column
    for (i = 0; i < n; i++) {
        if (board[i][col] == 1 && i != row)
            return 0;
    }

    // Check diagonal (top-left to bottom-right)
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1 && !(i == row && j == col))
            return 0;
    }
    for (i = row, j = col; i < n && j < n; i++, j++) {
        if (board[i][j] == 1 && !(i == row && j == col))
            return 0;
    }

    // Check diagonal (top-right to bottom-left)
    for (i = row, j = col; i >= 0 && j < n; i--, j++) {
        if (board[i][j] == 1 && !(i == row && j == col))
            return 0;
    }
    for (i = row, j = col; i < n && j >= 0; i++, j--) {
        if (board[i][j] == 1 && !(i == row && j == col))
            return 0;
    }

    return 1;
}

// Verify if initial placements are valid
int verifyInitialPlacements() {
```

```
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (board[i][j] == 1 && !isSafe(i, j))
                return 0;
        }
    }
    return 1;
}

// Backtracking function to solve N-Queens
int solveNQueens(int queensPlaced) {
    if (queensPlaced == n)
        return 1;

    int i, j, placedCount = 0;

    // Count already placed queens
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (board[i][j] == 1)
                placedCount++;
        }
    }

    if (placedCount == n)
        return 1;

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (board[i][j] == 0 && isSafe(i, j)) {
                board[i][j] = 1;
                if (solveNQueens(queensPlaced + 1))
                    return 1;
                board[i][j] = 0; // Backtrack
            }
        }
    }

    return 0;
}
```

```
void printBoard() {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int k, i, r, c;

    scanf("%d %d", &n, &k);

    // Initialize board with 0s
    for (i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            board[i][j] = 0;
        }
    }

    // Take initial placements
    for (i = 0; i < k; i++) {
        scanf("%d %d", &r, &c);
        board[r][c] = 1;
    }

    if (!verifyInitialPlacements()) {
        printf("No solution found.\n");
        return 0;
    }

    int placedQueens = 0;
    for (i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j] == 1)
                placedQueens++;
        }
    }

    if (solveNQueens(placedQueens)) {
```

```
        printBoard();
    } else {
        printf("No solution found.\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statment

You are tasked with placing N queens on an N×N chessboard such that no two queens can attack each other. Specifically:

No two queens can be on the same row.

No two queens can be on the same column.

No two queens can be on the same diagonal.

You need to find all possible valid placements of the queens and print them.

Example Scenario: N = 4

For n = 4, we need to find all the valid placements for 4 queens on a 4x4 chessboard.

Return only the number of distinct solutions to the n-queens puzzle.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example

Input:
4

Output:

2

..Q.

Q...

...Q

.Q..

.Q..

...Q

Q...

..Q.

Explanation:

There exist two distinct solutions to the 4-queens puzzle as shown.

Input Format

The input consists of a single integer n, representing the size of the chessboard.

Output Format

The first line contains an integer representing the number of distinct solutions to the N-Queens problem for the given value of n.

For each solution:

The board configuration is printed with n lines.

Each line contains n characters:

'Q' represents a queen.

'.' represents an empty space.

The configuration is printed in top-to-bottom row order, i.e., the first line corresponds to the top row of the board.

Solutions are printed in the order they are found by the backtracking algorithm,

vtu29890

which places queens column by column from left to right, and in each column, tries rows from top to bottom (row 0 to row n−1).

A blank line is printed after each solution to separate different board configurations.

Refer to the sample output format for the formatting specifications.

Sample Test Case

Input: 4

Output: 2

```
..Q.
Q...
...Q
.Q..

.Q..
...Q
Q...
..Q.
```

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 15

char solutions[1000][MAX][MAX];  // To store all solutions
int solution_count = 0;
char board[MAX][MAX];
int rows[MAX], diag1[2 * MAX], diag2[2 * MAX];

void saveSolution(int n) {
    for (int i = 0; i < n; i++) {
        strcpy(solutions[solution_count][i], board[i]);
    }
```

```
    solution_count++;
}

void solve(int col, int n) {
    if (col == n) {
        saveSolution(n);
        return;
    }

    for (int row = 0; row < n; row++) {
        if (rows[row] || diag1[col - row + n - 1] || diag2[col + row])
            continue;

        board[row][col] = 'Q';
        rows[row] = 1;
        diag1[col - row + n - 1] = 1;
        diag2[col + row] = 1;

        solve(col + 1, n);

        board[row][col] = '.';
        rows[row] = 0;
        diag1[col - row + n - 1] = 0;
        diag2[col + row] = 0;
    }
}

int main() {
    int n;
    scanf("%d", &n);

    // Initialize board
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            board[i][j] = '.';

    memset(rows, 0, sizeof(rows));
    memset(diag1, 0, sizeof(diag1));
    memset(diag2, 0, sizeof(diag2));

    solve(0, n);
```

```
    // Print the count first
    printf("%d\n", solution_count);

    // Then print all saved solutions
    for (int k = 0; k < solution_count; k++) {
        for (int i = 0; i < n; i++) {
            printf("%s\n", solutions[k][i]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected graph represented as an adjacency matrix. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle. If no cycle is found, the program should output "No solution".

Your task is to help Bob in implementing the same.

Input Format

The first line of input consists of a single integer, V, indicating the number of vertices in the graph.

The following V lines each contain V space separated integers, representing the adjacency matrix of the graph, where a '1' means an edge exists between two vertices, and a '0' means it does not.

Output Format

If a Hamiltonian cycle is found, print "Solution found" followed by the cycle as a series of vertices starting and ending at the starting vertex, space-separated on a new line.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8
0 1 0 1 1 0 0 0
1 0 1 0 0 1 0 0
0 1 0 1 0 0 1 0
1 0 1 0 0 0 0 1
1 0 0 0 0 1 0 1
0 1 0 0 1 0 1 0
0 0 1 0 0 1 0 1
0 0 0 1 1 0 1 0
Output: Solution found
0 1 2 3 7 6 5 4 0

Answer

```
#include <stdio.h>
#include <stdlib.h>

int V;
int graph[8][8];   // Max V = 8
int path[8];
int path_count;

// Check if vertex v is already in path
int is_present(int v) {
   for (int i = 0; i < path_count - 1; i++)
      if (path[i] == v) return 1;
   return 0;
}

// Recursive function to find Hamiltonian cycle
int solve(int vertex) {
   // Base case: If all vertices included and last connects to first
   if (graph[vertex][0] == 1 && path_count == V)
      return 1; // Solution found
```

```
    if (path_count == V)
        return 0;

    for (int v = 0; v < V; v++) {
        if (graph[vertex][v] == 1) {
            path[path_count] = v;
            path_count++;

            // Remove edge to avoid revisiting
            graph[vertex][v] = 0;
            graph[v][vertex] = 0;

            if (!is_present(v)) {
                if (solve(v)) return 1; // Solution found
            }

            // Backtrack: Restore edge and path
            graph[vertex][v] = 1;
            graph[v][vertex] = 1;
            path_count--;
            path[path_count] = -1;
        }
    }
    return 0;
}

void display() {
    for (int i = 0; i <= V; i++)
        printf("%d ", path[i % V]);
    printf("\n");
}

int main() {
    scanf("%d", &V);

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    // Initialize path
    for (int i = 0; i < V; i++) path[i] = -1;
```

```
    path[0] = 0;
    path_count = 1;

    if (solve(0)) {
        printf("Solution found\n");
        display();
    } else {
        printf("No solution\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In chess, each step of a knight comprises stepping by two squares horizontally and one square vertically, or by one square horizontally and two squares vertically. A knight making one step from location (0,0) of an infinite chess board would finish up at one of the following eight locations: (1,2), (−1,2), (1,−2), (−1,−2), (2,1), (−2,1), (2,−1), (−2,−1).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

Input Format

The first line of input contains two integers x and y, representing the location of a knight on a chess board.

Output Format

The output displays a single integer m, representing the minimum number of steps needed for a knight to move from (0,0) to (x,y).

Sample Test Case

Input: 1 2

Output: 1

Answer

```
// You are using GCC

#include <stdio.h>
#include <limits.h>

#define RNG 100
#define ZERO 50

int dir[8][2] = {
    {-2, -1}, {-2, 1}, {-1, -2}, {-1, 2},
    {1, -2}, {1, 2}, {2, -1}, {2, 1}
};

int d[RNG][RNG];

int moveKnight(int x, int y, int depth) {
    if (x < 0 || y < 0 || x >= RNG || y >= RNG) {
        return INT_MAX;
    }
    if (d[x][y] != -1 && d[x][y] <= depth) {
        return INT_MAX;
    }
    d[x][y] = depth;
    if (x == ZERO && y == ZERO) {
        return depth;
    }

    int res = INT_MAX;
    for (int i = 0; i < 8; ++i) {
        int nextX = x + dir[i][0];
        int nextY = y + dir[i][1];
        int val = moveKnight(nextX, nextY, depth + 1);
        if (val < res) {
            res = val;
        }
    }
    return res;
}
```

```
int main() {
    for (int i = 0; i < RNG; ++i) {
        for (int j = 0; j < RNG; ++j) {
            d[i][j] = -1;
        }
    }

    int x, y;
    scanf("%d %d", &x, &y);
    int result = moveKnight(x + ZERO, y + ZERO, 0);
    printf("%d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 10_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Jovika is learning about heaps and wants to implement a program that constructs a max heap and inserts elements into it.

She wants to write a program that implements max heap insertion. The program defines functions to swap elements and perform maximum heapification. She wants to extend this program to take a list of integers, build a max heap, and insert a new element into the heap.

Help her in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements to be inserted into the max heap.

vtu29890

The second line consists of N space-separated integers, representing the elements to be inserted into the max heap.

The third line consists of an integer representing the new element to be inserted into the max heap.

Output Format

The output prints the space-separated integers, representing the elements in the max heap after inserting the new element in the order of insertion.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
3 1 4 2 5
6
Output: 6 3 5 2 1 4

Answer

```
// You are using GCC
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify function (max heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Function to build max heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

// Insert new element into heap
void insert(int arr[], int *n, int key) {
    (*n)++;
    arr[*n - 1] = key;

    int i = *n - 1;

    // Bubble up
    while (i != 0 && arr[(i - 1) / 2] < arr[i]) {
        swap(&arr[i], &arr[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int newElement;
    scanf("%d", &newElement);
```

```
    // Build max heap
    buildMaxHeap(arr, n);

    // Insert new element
    insert(arr, &n, newElement);

    // Print heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Bob is fascinated by numbers and loves arranging them in various ways. His recent exploration led him to Max Heaps.

Bob likes odd numbers more than even numbers. After inserting elements into a Max Heap, he wants to remove all even numbers from it while keeping the Max Heap property intact.

Your task is to help Bob build a Max Heap from the given elements, display it, delete all the even numbers from the resultant array and then rearrange the elements in the form of Max Heap.

Input Format

The first line of input consists of an integer N, representing the number of elements initially to be inserted into the Max Heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the heap.

Output Format

The first line of output prints the space-separated elements of the Max Heap

after building it from the given N elements.

The second line prints the space-separated elements of the Max Heap after removing even elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3 2 1
Output: 3 2 1
3 1

Answer

```
// You are using GCC
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify function
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
```

```
    }
}

// Build max heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build max heap
    buildMaxHeap(arr, n);

    // Step 2: Print initial heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Step 3: Remove even numbers
    int newArr[100];
    int newSize = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 != 0) {  // keep odd numbers only
            newArr[newSize++] = arr[i];
        }
    }

    // Step 4: Rebuild heap with remaining elements
    buildMaxHeap(newArr, newSize);

    // Step 5: Print new heap
```

```
    for (int i = 0; i < newSize; i++) {
        printf("%d ", newArr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Iniya is studying data structures and algorithms, and she is currently learning about heaps.

She wants to practice implementing Min-Heap and perform a deletion operation on it. A min-heap is a binary tree in which the value of each node is less than or equal to the values of its children.

Write a program to assist Iniya in building a min-heap and delete the root element.

Input Format

The first line of input consists of an integer N, representing the number of elements in the initial Min-Heap.

The second line consists of N space-separated integers, where each integer represents an element in the Min-Heap.

Output Format

The output prints the space-separated elements of the Min-Heap after performing the deletion operation of the root.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

vtu29890

80 40 30 10 50 33 66

Output: 30 40 33 80 50 66

Answer

```
// You are using GCC
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify function for Min Heap
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        heapify(arr, n, smallest);
    }
}

// Build Min Heap
void buildMinHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build Min Heap
    buildMinHeap(arr, n);

    // Step 2: Delete root
    arr[0] = arr[n - 1];  // Replace root with last element
    n--;                  // Reduce size

    // Step 3: Heapify root
    heapify(arr, n, 0);

    // Step 4: Print heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

A trie (pronounced as "try") or prefix tree is a tree data structure used to store and retrieve keys in a dataset of strings efficiently. This data structure has various applications, such as autocomplete and spellchecker.

Implement the Trie class:

Trie() Initializes the trie object.void insert(String word): Inserts the string word into the trie.boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.boolean startsWith(String prefix) Returns true if a previously inserted string word has the prefix, and false otherwise.

Example

Input:

4

apple

cat

orange

mat

cat

ora

Output:

The word is present in the trie.

There is a word with the given prefix in the trie.

Input Format

The first line of the input consists of the number of words to insert, N.

The next N lines of input consist of the words in each line.

The next line of input consists of the word to search.

The last line of the input consists of the prefix to search.

Output Format

When searching for a word using the search method, the program will output one of the following messages:

- If the word is present in the trie, the output prints "The word is present in the trie."
- If the word is not present in the trie, the output prints "The word is not present in the trie."

vtu29890

When checking for a prefix using the startsWith method, the program will output one of the following messages:

- If there is a word in the trie with the given prefix, the output prints "There is a word with the given prefix in the trie."
- If there is no word in the trie with the given prefix, the output prints "There is no word with the given prefix in the trie."

Sample Test Case

Input: 4
apple
cat
orange
mat
catlyn
ora

Output: The word is not present in the trie.
There is a word with the given prefix in the trie.

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALPHABET_SIZE 26

// Trie Node
struct TrieNode {
    struct TrieNode *children[ALPHABET_SIZE];
    int isEndOfWord;
};

// Create a new node
struct TrieNode* createNode() {
    struct TrieNode *node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
    node->isEndOfWord = 0;
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;
    return node;
}
```

```
// Insert word
void insert(struct TrieNode *root, char *word) {
    struct TrieNode *curr = root;
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';
        if (curr->children[index] == NULL)
            curr->children[index] = createNode();
        curr = curr->children[index];
    }
    curr->isEndOfWord = 1;
}

// Search full word
int search(struct TrieNode *root, char *word) {
    struct TrieNode *curr = root;
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';
        if (curr->children[index] == NULL)
            return 0;
        curr = curr->children[index];
    }
    return curr->isEndOfWord;
}

// Check prefix
int startsWith(struct TrieNode *root, char *prefix) {
    struct TrieNode *curr = root;
    for (int i = 0; prefix[i]; i++) {
        int index = prefix[i] - 'a';
        if (curr->children[index] == NULL)
            return 0;
        curr = curr->children[index];
    }
    return 1;
}

int main() {
    int n;
    scanf("%d", &n);

    struct TrieNode *root = createNode();
```

```
    char word[100];

    // Insert words
    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    char searchWord[100], prefix[100];
    scanf("%s", searchWord);
    scanf("%s", prefix);

    // Search result
    if (search(root, searchWord))
        printf("The word is present in the trie. ");
    else
        printf("The word is not present in the trie. ");

    // Prefix result
    if (startsWith(root, prefix))
        printf("There is a word with the given prefix in the trie.");
    else
        printf("There is no word with the given prefix in the trie.");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Given an array of integers representing the combat power of N players.

You need to efficiently perform the following operations:

Query (Q L R): Find the maximum combat power between ranks L and R (inclusive).Update (U i val): Update the power of the player at index i to a new value val.

To ensure efficiency, implement these operations using a Segment Tree.

Input Format

The first line contains two space-separated integers, N and Q, representing the number of players and the number of operations.

The second line contains N space-separated integers representing the initial combat power scores of all players, where index 0 is the top-ranked player.

The next Q lines describe the operations:

- "Q L R" Query represents the maximum combat power from ranks L to R (inclusive).
- "U i val" Updates the power score of the player at rank i to val.

All indices are 0-based.

Output Format

For each query Q L R, print the maximum combat power in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
4 7 2 9 5 1
Q 1 4
U 3 6
Q 1 4
U 0 10
Q 0 2

Output: 9
7
10

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 100
```

```
int arr[MAX];
int seg[4 * MAX];

// Build Segment Tree
void build(int node, int start, int end) {
   if (start == end) {
      seg[node] = arr[start];
   } else {
      int mid = (start + end) / 2;
      build(2 * node, start, mid);
      build(2 * node + 1, mid + 1, end);
      seg[node] = (seg[2 * node] > seg[2 * node + 1]) ? seg[2 * node] : seg[2 * node
+ 1];
   }
}

// Update value
void update(int node, int start, int end, int idx, int val) {
   if (start == end) {
      arr[idx] = val;
      seg[node] = val;
   } else {
      int mid = (start + end) / 2;
      if (idx <= mid)
         update(2 * node, start, mid, idx, val);
      else
         update(2 * node + 1, mid + 1, end, idx, val);

      seg[node] = (seg[2 * node] > seg[2 * node + 1]) ? seg[2 * node] : seg[2 * node
+ 1];
   }
}

// Range Maximum Query
int query(int node, int start, int end, int L, int R) {
   if (R < start || end < L)
      return -1; // outside range

   if (L <= start && end <= R)
      return seg[node];
```

```
    int mid = (start + end) / 2;
    int left = query(2 * node, start, mid, L, R);
    int right = query(2 * node + 1, mid + 1, end, L, R);

    return (left > right) ? left : right;
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build tree
    build(1, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'Q') {
            int L, R;
            scanf("%d %d", &L, &R);
            printf("%d ", query(1, 0, n - 1, L, R));
        } else if (type == 'U') {
            int idx, val;
            scanf("%d %d", &idx, &val);
            update(1, 0, n - 1, idx, val);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sai has been studying various algorithms related to heaps. Recently, he came across a problem where he needed to filter elements from a given array based on certain criteria related to heap properties.

He wants to build a max heap from a given array of integers, delete elements that fall outside the specified range, and then rearrange the elements in the form of a Max Heap.

Write a program to help Sai in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the elements of the array.

The third line consists of two integers, a and b, separated by a space, representing the range to filter the elements.

Output Format

The output prints the space-separated integers, representing the elements that remain in the max heap after removing elements outside the given range.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 7 8 3 12
1 6

Output: 5 3

Answer

```
// You are using GCC
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Build Max Heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int a, b;
    scanf("%d %d", &a, &b);

    // Step 1: Build Max Heap
    buildMaxHeap(arr, n);

    // Step 2: Filter elements within range [a, b]
    int filtered[100];
    int size = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] >= a && arr[i] <= b) {
            filtered[size++] = arr[i];
        }
    }

    // Step 3: Rebuild Max Heap
```

```
    buildMaxHeap(filtered, size);

    // Step 4: Print result
    for (int i = 0; i < size; i++) {
        printf("%d ", filtered[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Maxwell, an enthusiastic computer science student, is exploring the fascinating world of data structures. His task is to create a program that constructs a min heap from a list of integers and identifies negative numbers within it.

Help Maxwell in completing the program.

Input Format

The first line of input consists of an integer N, representing the number of integers in the array.

The second line consists of N space-separated integers, representing the elements of the list.

Output Format

The first line of output prints the min-heap as a space-separated list of integers.

The second line prints the negative numbers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

15 -5 20 -19 23 42 29

Output: -19 -5 20 15 23 42 29
-19 -5

Answer

```
// You are using GCC
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Min Heap)
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        heapify(arr, n, smallest);
    }
}

// Build Min Heap
void buildMinHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
```

```
    scanf("%d", &n);

    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build Min Heap
    buildMinHeap(arr, n);

    // Step 2: Print Min Heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Step 3: Print negative numbers
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            printf("%d ", arr[i]);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are given an array of n integers. You need to perform q queries of two types:

Q l r — Find the minimum value in the subarray from index l to r (inclusive).U idx val — Update the element at index idx to val.

Implement an efficient solution using a Segment Tree to handle both query and update operations.

Input Format

The first line contains two space-separated integers n and q, representing the

size of the array and the number of queries

The second line contains n space-separated integers representing the elements of the array.

Next q lines: Each line contains a query in one of the following formats:

- Q l r
- U idx val

Output Format

For each query of type Q, print the minimum value in the specified range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3 2
4 2 5
Q 0 2
Q 1 1

Output: 2
2

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int arr[MAX];
int seg[4 * MAX];

// Build Segment Tree (Min)
void build(int node, int start, int end) {
   if (start == end) {
      seg[node] = arr[start];
   } else {
      int mid = (start + end) / 2;
```

```
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        seg[node] = (seg[2 * node] < seg[2 * node + 1]) ? seg[2 * node] : seg[2 * node
+ 1];
    }
}

// Update value
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        arr[idx] = val;
        seg[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid)
            update(2 * node, start, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, end, idx, val);

        seg[node] = (seg[2 * node] < seg[2 * node + 1]) ? seg[2 * node] : seg[2 * node
+ 1];
    }
}

// Range Minimum Query
int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L)
        return 100000; // large value (acts like infinity)

    if (L <= start && end <= R)
        return seg[node];

    int mid = (start + end) / 2;
    int left = query(2 * node, start, mid, L, R);
    int right = query(2 * node + 1, mid + 1, end, L, R);

    return (left < right) ? left : right;
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build tree
    build(1, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'Q') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d ", query(1, 0, n - 1, l, r));
        } else if (type == 'U') {
            int idx, val;
            scanf("%d %d", &idx, &val);
            update(1, 0, n - 1, idx, val);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Meera is working on a text processing task where she wants to identify the longest common prefix among a given set of words. She decides to use a Trie structure to efficiently compute this prefix.

Write a program to help Meera insert the given words into a Trie and then determine the longest common prefix among them.

Input Format

The first line contains an integer n, representing the number of words.

The next n lines each contain a single word.

Output Format

The output prints "The longest common prefix is: <prefix>" if a common prefix exists.

Otherwise, print "There is no common prefix"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
aeroplane
aerospace
aerial
aesthetic
Output: The longest common prefix is: ae

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALPHABET_SIZE 26

// Trie Node
struct TrieNode {
   struct TrieNode *children[ALPHABET_SIZE];
   int isEndOfWord;
};

// Create new node
struct TrieNode* createNode() {
   struct TrieNode *node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
   node->isEndOfWord = 0;
   for (int i = 0; i < ALPHABET_SIZE; i++)
      node->children[i] = NULL;
   return node;
```

```
}

// Insert word into Trie
void insert(struct TrieNode *root, char *word) {
    struct TrieNode *curr = root;
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';
        if (curr->children[index] == NULL)
            curr->children[index] = createNode();
        curr = curr->children[index];
    }
    curr->isEndOfWord = 1;
}

// Count children of a node
int countChildren(struct TrieNode *node, int *index) {
    int count = 0;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (node->children[i] != NULL) {
            count++;
            *index = i;
        }
    }
    return count;
}

// Find Longest Common Prefix
void findLCP(struct TrieNode *root, char *prefix) {
    struct TrieNode *curr = root;
    int index;
    int i = 0;

    while (countChildren(curr, &index) == 1 && curr->isEndOfWord == 0) {
        prefix[i++] = index + 'a';
        curr = curr->children[index];
    }
    prefix[i] = '\0';
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    struct TrieNode *root = createNode();
    char word[100];

    // Insert words
    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    char prefix[100];
    findLCP(root, prefix);

    if (strlen(prefix) > 0)
        printf("The longest common prefix is: %s ", prefix);
    else
        printf("There is no common prefix ");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 10_MCQ

Attempt : 2
Total Mark : 15
Marks Obtained : 9

Section 1 : MCQ

1. Which of the following sequences of array elements forms a heap?

Answer

{23, 17, 14, 6 13, 10, 1, 5, 7, 12}

Status : Wrong ***Marks : 0/1***

2. Consider a max heap, represented by the array: 23, 17, 14, 6, 13, 10, 1, 5. If a value 20 is inserted into this heap, then what will be the new max heap?

Answer

23, 20, 17, 14, 13, 10, 6, 5, 1

Status : Wrong ***Marks : 0/1***

vtu29890

3. Given a binary max heap. The elements are sorted in an array as 25, 14, 16, 13, 10, 8, 12. What is the content of the array after two delete operations?

Answer

14, 13, 12, 8, 10

Status : Correct ***Marks : 1/1***

4. In a Min Heap, what is the condition that must be satisfied between a parent and its children?

Answer

The parent must have a lower value than its children

Status : Correct ***Marks : 1/1***

5. When inserting an element with a value of 8 into a Min Heap, where would it be placed if the current heap contains the elements [10, 12, 15, 20, 17]?

Answer

As the right child of the root

Status : Wrong ***Marks : 0/1***

6. If you're inserting an element with a value of 5 into a Min Heap, and the current heap contains the elements [11, 16, 10, 13, 18], what will the Min Heap look like after the insertion?

Answer

5, 10, 11, 13, 16, 18

Status : Wrong ***Marks : 0/1***

7. When deleting the root element from a Min Heap, which of the following replaces it as the new root?

Answer

the last element from the tree

Status : Correct ***Marks : 1/1***

8. Tries are mainly used for:

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

9. In a trie, how is the end of a valid word usually represented?

Answer

By using an end-of-word (terminal) marker at a node

Status : Correct ***Marks : 1/1***

10. Which of the following is true about tries?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

11. What is stored in a segment tree node for a Range Minimum Query (RMQ)?

Answer

Maximum element in range

Status : Wrong ***Marks : 0/1***

12. In a Range Max Query tree, what is the internal node value?

Answer

Maximum of child values

Status : Correct ***Marks : 1/1***

13. Which segment tree operation is used for range max queries?

Answer

max() merge

Status : Correct ***Marks : 1/1***

14. What data structure is often used in segment tree nodes to count distinct elements?

Answer

Set

Status : Correct ***Marks : 1/1***

15. In range distinct count queries, what does each segment tree node contain?

Answer

Frequency table

Status : Wrong ***Marks : 0/1***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 10_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Aarav, a computer science student, is working on a heap-based data processing system. He needs to find the second-largest element from a collection of integers represented as a max-heap. Aarav first constructs the max-heap, then removes the largest element to extract the next largest value.

Write a program to help Aarav determine and print the second largest element, or display a message if it does not exist.

Input Format

The first line contains an integer n, representing the number of elements in the heap.

vtu29890

The second line contains n space-separated integers representing the elements of the heap.

Output Format

The output prints the second-largest element in the heap.

If there is no second-largest element, print the message:

"There is no second largest element in the heap."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 70 10 30
Output: 70

Answer

```
// You are using GCC
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;
```

```
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Build Max Heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // If less than 2 elements
    if (n < 2) {
        printf("There is no second largest element in the heap.");
        return 0;
    }

    // Step 1: Build Max Heap
    buildMaxHeap(arr, n);

    // Step 2: Remove root (largest)
    arr[0] = arr[n - 1];
    n--;

    // Step 3: Heapify again
    heapify(arr, n, 0);

    // Step 4: Print new root (second largest)
    printf("%d", arr[0]);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Elijah wants to verify if an array represents a valid min-heap. Implement a program to check whether a given array satisfies the min-heap property.

In a min-heap, for every node i, the value at that node must be less than or equal to the values of its child nodes (if they exist). Write a program to read an array and determine whether it maintains the min-heap structure.

Input Format

The first line contains an integer n, representing the number of elements.

The next line contains n space-separated integers representing the elements to be inserted into the min heap.

Output Format

The first line prints "The array represents a valid min-heap" if the array satisfies the min-heap property. Otherwise, print "The array does not represent a valid min-heap".

The second line prints the space-separated elements of the array as given in the input.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: The array represents a valid min-heap
1 2 3

Answer

```
// You are using GCC
#include <stdio.h>

int isMinHeap(int arr[], int n) {
    for (int i = 0; i <= (n / 2) - 1; i++) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[i] > arr[left])
            return 0;

        if (right < n && arr[i] > arr[right])
            return 0;
    }
    return 1;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (isMinHeap(arr, n))
        printf("The array represents a valid min-heap ");
    else
        printf("The array does not represent a valid min-heap ");

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a list of daily transaction amounts over N days. Each transaction can be positive (deposit) or negative (withdrawal).

You must efficiently:

Find the sum of transactions between two given days.Update the transaction amount for any specific day.

To perform these operations efficiently, implement a Segment Tree.

Input Format

The first line contains two integers, N and Q, representing the number of days and number of operations.

The second line contains N space-separated integers representing the transaction amount for each day.

The next Q lines contain operations in one of the following formats:

- S L R represents Query sum from index L to R
- U i val Updates index i with new value val

All indices are 0-based.

Output Format

For each query S L R, print the sum of transactions in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
10 -5 20 15 -10 5
S 0 2
U 1 7
S 0 2
S 3 5

vtu29890

S 0 5
Output: 25
37
10
47

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int arr[MAX];
int seg[4 * MAX];

// Build Segment Tree (Sum)
void build(int node, int start, int end) {
    if (start == end) {
        seg[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        seg[node] = seg[2 * node] + seg[2 * node + 1];
    }
}

// Update value
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        arr[idx] = val;
        seg[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid)
            update(2 * node, start, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, end, idx, val);

        seg[node] = seg[2 * node] + seg[2 * node + 1];
    }
}
```

```
// Range Sum Query
int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L)
        return 0;

    if (L <= start && end <= R)
        return seg[node];

    int mid = (start + end) / 2;
    int left = query(2 * node, start, mid, L, R);
    int right = query(2 * node + 1, mid + 1, end, L, R);

    return left + right;
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    build(1, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'S') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d ", query(1, 0, n - 1, l, r));
        } else if (type == 'U') {
            int i, val;
            scanf("%d %d", &i, &val);
            update(1, 0, n - 1, i, val);
        }
    }

    return 0;
```

}

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given a string s and a string array dictionary, return the longest string in the dictionary that can be formed by deleting some of the given string characters.

If there is more than one possible result, return the longest word with the smallest lexicographical order.

Example

Input:

s = "abpcplea"

dictionary = ["ale", "apple", "monkey", "plea"]

Output:

"apple"

Input Format

The first line of input consists of the string.

The second line of input consists of the array size.

The following lines of input consist of the array elements, each in a separate line.

Output Format

The output prints the longest string in the dictionary that can be formed by deleting some of the given string characters.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: abpcplea
4
ale
apple
monkey
plea

Output: apple

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

// Check if word is a subsequence of s
int isSubsequence(char *s, char *word) {
    int i = 0, j = 0;

    while (s[i] && word[j]) {
        if (s[i] == word[j]) {
            j++;
        }
        i++;
    }

    return word[j] == '\0';
}

// Compare lexicographically
int lexicographicallySmaller(char *a, char *b) {
    return strcmp(a, b) < 0;
}

int main() {
    char s[1005];
    scanf("%s", s);

    int n;
    scanf("%d", &n);

    char dict[1005][1005];
```

```
    char result[1005] = "";

    for (int i = 0; i < n; i++) {
        scanf("%s", dict[i]);
    }

    for (int i = 0; i < n; i++) {
        if (isSubsequence(s, dict[i])) {

            // Choose better word
            if (strlen(dict[i]) > strlen(result) ||
               (strlen(dict[i]) == strlen(result) &&
                strcmp(dict[i], result) < 0)) {
                strcpy(result, dict[i]);
            }
        }
    }

    printf("%s ", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 11_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an integer n, representing the total number of stairs, the task is to determine the number of distinct ways to reach the top when at each move you can climb 1, 2, or 3 steps.

Write a program using the Dynamic Programming (Bottom-Up Approach) algorithm to ensure that the total number of possible ways to climb the stairs is correctly calculated and displayed.

Examples:

Input: 4

Output: 7

Explanation: There are seven ways: {1, 1, 1, 1}, {1, 2, 1}, {2, 1, 1}, {1, 1, 2}, {2,

2}, {3, 1}, {1, 3}.

Input: 3

Output: 4

Explanation: There are four ways: {1, 1, 1}, {1, 2}, {2, 1}, {3}.

Input Format

The input contains a single integer n, representing the total number of stairs.

Output Format

The output prints a single integer representing the total number of distinct ways to reach the top.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
Output: 7

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    // Base cases
    int dp[25];  // since n ≤ 20, array size 25 is safe
    dp[0] = 1;   // 1 way to stay at the ground (no steps)
    dp[1] = 1;   // 1 way to climb 1 step
    dp[2] = 2;   // {1+1, 2}

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2] + dp[i-3];
    }
```

```
    printf("%d", dp[n]);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a non-negative integer n, the task is to compute the n-th Lucas number using the Dynamic Programming (Bottom-Up Approach) algorithm. The Lucas sequence follows the recurrence relation:

L(0) = 2, L(1) = 1, and L(n) = L(n−1) + L(n−2) for n ≥ 2.

Write a program to ensure that the n-th Lucas number is correctly calculated and displayed.

Input Format

The input contains a single integer n, representing the position in the Lucas sequence.

Output Format

The output prints a single integer representing the n-th Lucas number.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
Output: 4

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
```

```
    scanf("%d", &n);

    // Base cases
    if (n == 0) {
        printf("2");
        return 0;
    }
    if (n == 1) {
        printf("1");
        return 0;
    }

    int dp[25];  // safe size since n ≤ 20
    dp[0] = 2;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }

    printf("%d", dp[n]);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement:

Dr. Ananya is a computational biologist working on analyzing DNA sequences. She needs to identify the longest palindromic subsequence within a DNA string, which may represent certain symmetric patterns important for genetic research.

Given a string S representing a DNA sequence (composed of letters A, C, G, T), find the length of the longest subsequence of S that reads the same forwards and backwards.

Note:

A subsequence is formed by deleting zero or more characters without

changing the order of the remaining characters.The DNA sequences can be long, and an efficient solution using dynamic programming is required to handle them within computational limits.

Input Format

The input displays a single line containing the DNA string S.

Output Format

The output displays a single integer representing the length of the longest palindromic subsequence in S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam

Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int dp[n][n];

    // Initialize DP table
    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;  // single character is a palindrome of length 1
    }
```

```
    // Build the table bottom-up
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            if (s[i] == s[j]) {
                if (len == 2)
                    dp[i][j] = 2;
                else
                    dp[i][j] = 2 + dp[i+1][j-1];
            } else {
                dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
            }
        }
    }

    printf("%d", dp[0][n-1]);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Arjun is tracking the stock prices for the last N days. He wants to invest in a way that he buys and sells stocks alternately to maximize his profit. He can either buy or skip on a day, but after buying he must sell before buying again. Find the maximum profit he can achieve from the stock prices. Help him to implement the task.

Formula

Let dp[i][0] = maximum profit on day i if not holding a stock

Let dp[i][1] = maximum profit on day i if holding a stock

Transition:

dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]) either skip or sell today

dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]) either keep holding or buy today

Base case:

dp[0][0] = 0

dp[0][1] = -prices[0]

Result:

max_profit = dp[N-1][0]

Input Format

The first line of input consists of Integer N representing the number of days.

The second line of input consists of N Integers prices[i] representing the stock price on day i.

Output Format

The output prints: "Maximum profit:" max_profit

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3

Output: Maximum profit: 2

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int N;
    scanf("%d", &N);

    int prices[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &prices[i]);
```

```
    }
    int dp[N][2];

    // Base case
    dp[0][0] = 0;           // not holding stock on day 0
    dp[0][1] = -prices[0];   // holding stock on day 0 (buy)

    // Fill DP table
    for (int i = 1; i < N; i++) {
        dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]);   // skip or sell
        dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]);   // hold or buy
    }

    int max_profit = dp[N-1][0];
    printf("Maximum profit: %d", max_profit);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statment

To celebrate Diwali, Dhoni wants to buy a variety of crackers. He noticed four different types of crackers on the menu list, each with a price. He's now trying to figure out how many different ways he can get the crackers for Rs.N.

Rockets - 1 RsSparklers - 2 RsChakkars - 3 RsFlower pots - 5 Rs

Example

Input

4

Output

The total number of ways to get crackers is 4

Explanation

1st way -> 4 rockets

2nd way -> 2 Sparklers

3rd way -> 1 Sparkler, 2 rockets

4th way -> 1 Chakkar, 1 rocket

Input Format

The input consists of a single integer N, representing the total amount (in rupees) Dhoni wants to spend on crackers.

Output Format

The output prints "The total number of ways to get crackers is X" Where X representing the total number of different ways Dhoni can buy crackers such that the total cost equals N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
Output: The total number of ways to get crackers is 6

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int coins[] = {1, 2, 3, 5};
    int m = sizeof(coins) / sizeof(coins[0]);

    long long dp[N+1];
    for (int i = 0; i <= N; i++) {
        dp[i] = 0;
    }
    dp[0] = 1;  // one way to make 0 (choose nothing)

    // Bottom-up DP: count ways
```

```
    for (int i = 0; i < m; i++) {
        for (int j = coins[i]; j <= N; j++) {
            dp[j] += dp[j - coins[i]];
        }
    }

    printf("The total number of ways to get crackers is %lld", dp[N]);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 11_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 24.5

Section 1 : Coding

1. Problem Statement

Given a set of coin denominations and a target value V, determine the minimum number of coins required to make the value V.

If it's not possible to make the value with the given coins, return -1.

Input Format

The first line contains an integer n, representing the number of coin denominations.

The second line contains n space-separated integers, representing the coin denominations.

The third line contains an integer amount, representing the target value V.

Output Format

vtu29890

The output prints the minimum number of coins required to make the target value.

Print -1 if the value cannot be formed using the given denominations.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 5
11
Output: 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int coins[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    int amount;
    scanf("%d", &amount);

    int dp[amount + 1];
    for (int i = 0; i <= amount; i++) {
        dp[i] = INT_MAX;  // initialize with large value
    }
    dp[0] = 0;  // base case: 0 coins needed to make amount 0
```

```
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }

    if (dp[amount] == INT_MAX) {
        printf("-1");
    } else {
        printf("%d", dp[amount]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Rajesh runs a delivery company and needs to deliver n packages to different cities. Each package i has:

A weight wt[i]A profit val[i] if delivered on timeA delivery deadline deadline[i] (in hours from now)Rajesh has a truck with maximum weight capacity W and can only deliver one package at a time. He can choose the order of deliveries, but if a package is delivered after its deadline, he earns zero profit for it.

Task: Determine the maximum total profit Rajesh can earn by selecting which packages to deliver and in what order, without exceeding the truck's capacity at any point.

Input Format

First line: integer n — number of packages

Second line: n integers — values of each package (profit)

Third line: n integers — weights of each package

Fourth line: n integers — deadlines of each package

Fifth line: integer W — truck capacity

Output Format

The output prints the single integer maximum total profit.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 200 150 120
10 20 15 15
5 3 4 2
30

Output: 470

Answer

```
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

typedef struct {
    int val;
    int wt;
    int deadline;
} Package;

int main() {
    int n;
    scanf("%d", &n);

    Package p[n];
    for (int i = 0; i < n; i++) scanf("%d", &p[i].val);
```

```
for (int i = 0; i < n; i++) scanf("%d", &p[i].wt);
for (int i = 0; i < n; i++) scanf("%d", &p[i].deadline);
int W;
scanf("%d", &W);

// Maximum deadline among packages
int maxDeadline = 0;
for (int i = 0; i < n; i++) {
   if (p[i].deadline > maxDeadline) maxDeadline = p[i].deadline;
}

// dp[mask][time] = max profit achievable with subset mask up to time
// n ≤ 20, deadlines ≤ 50
static int dp[1<<20][51];
for (int i = 0; i < (1<<n); i++) {
   for (int t = 0; t <= maxDeadline; t++) {
      dp[i][t] = -1;
   }
}
dp[0][0] = 0;

int ans = 0;

for (int mask = 0; mask < (1<<n); mask++) {
   for (int t = 0; t <= maxDeadline; t++) {
      if (dp[mask][t] == -1) continue;
      // try adding a new package
      for (int i = 0; i < n; i++) {
         if (!(mask & (1<<i))) {
            if (p[i].wt <= W && t+1 <= p[i].deadline) {
               int newMask = mask | (1<<i);
               int newTime = t+1;
               int newProfit = dp[mask][t] + p[i].val;
               if (dp[newMask][newTime] < newProfit) {
                  dp[newMask][newTime] = newProfit;
               }
               ans = max(ans, newProfit);
            }
         }
      }
      ans = max(ans, dp[mask][t]);
   }
```

```
    }
    printf("%d", ans);
    return 0;
}
```

Status : Partially correct ***Marks : 4.5/10***

3. Problem Statement

You are given n items, each with a certain weight and value. Your task is to select items to put in a knapsack of capacity W so that the total value is maximized. You cannot break an item; you must either take the whole item or leave it.

Formally, given two integer arrays, val[0..n-1] and wt[0..n-1], representing the values and weights of n items respectively, and an integer W representing the knapsack capacity, determine the maximum value subset of items such that the total weight does not exceed W.

Solve this problem using dynamic programming.

Input Format

The first line contains an integer n, the number of items.

The second line contains n integers, the values of each item separated by spaces.

The third line contains n integers, the weights of each item separated by spaces.

The fourth line contains an integer W, the maximum capacity of the knapsack.

Output Format

Print a single integer representing the maximum value that can be carried in the knapsack.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2
50 60
5 10
10

Output: 60

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    scanf("%d", &n);

    int val[n], wt[n];
    for (int i = 0; i < n; i++) scanf("%d", &val[i]);
    for (int i = 0; i < n; i++) scanf("%d", &wt[i]);
    scanf("%d", &W);

    int dp[n+1][W+1];

    // Build DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    printf("%d", dp[n][W]);
    return 0;
```

}

Status : Correct ***Marks : 10/10***

4. Problem Statement

Imagine you are a tourist in Moscow and have booked a bus tour to see some amazing attractions just outside of town. The bus first drives around town for a while (a long while, since Moscow is a big city) picking up people at their respective hotels. It then proceeds to the amazing attraction, and after a few hours go back into the city, again driving to each hotel, this time to drop people off.

For some reason, whenever you do this, your hotel is always the first to be visited for pickup and the last to be visited for drop-off, meaning that you have to suffer through two not-so-amazing sightseeing tours of all the local hotels.

This is clearly not what you want to do (unless for some reason you are really into hotels), so let's fix it. We will develop some software to enable the sightseeing company to route its bus tours more fairly—though it may sometimes mean a longer total distance for everyone, fair is fair, right?

For this problem, there is a starting location (the sightseeing company headquarters), h hotels that need to be visited for pickups and dropoffs, and a destination location (the amazing attraction). We need to find a route that goes from the headquarters, through all the hotels, to the attraction, then back through all the hotels again (possibly in a different order), and finally back to the headquarters.

To guarantee that none of the tourists (and, in particular, you) is forced to suffer through two full tours of the hotels, we require that every hotel that is visited among the first $h/2$ hotels on the way to the attraction is also visited among the first $h/2$ hotels on the way back.

Subject to these restrictions, we would like to make the complete bus tour as short as possible. Note that these restrictions may force the bus to drive past a hotel without stopping there (this is not considered visiting)

and then visit it later, as illustrated in the sample input.

Input Format

The first line of each test case consists of two integers n and m, where n is the number of locations (hotels, headquarters, attraction) and m is the number of pairs of locations between which the bus can travel.

The n different locations are numbered from 0 to n−1, where 0 is the headquarters, 1 through n−2 are the hotels, and n−1 is the attraction. Assume that there is at most one direct connection between any pair of locations and it is possible to travel from any location to any other location (but not necessarily directly).

Following the first line are m lines, each containing three integers u, v, and t, indicating that the bus can go directly between locations u and v in t seconds (in either direction).

Output Format

For each test case, print the case number and the minimum total travel time of the complete tour.

The output should follow this format: "Case X: Y"

where:

X is the test case number starting from 1.

Y is the shortest possible total time (in seconds) for the bus tour.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 4
0 1 10
1 2 20
2 3 30
3 4 40
4 6
0 1 1

0 2 1
0 3 1
1 2 1
1 3 1
2 3 1

Output: Case 1: 300
Case 2: 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define INF 1000000000

int min(int a, int b) { return a < b ? a : b; }

int main() {
    int n, m;
    int caseNo = 1;
    while (scanf("%d %d", &n, &m) == 2) {
        int dist[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                dist[i][j] = (i == j ? 0 : INF);

        for (int i = 0; i < m; i++) {
            int u, v, t;
            scanf("%d %d %d", &u, &v, &t);
            dist[u][v] = min(dist[u][v], t);
            dist[v][u] = min(dist[v][u], t);
        }

        // Floyd–Warshall for all-pairs shortest paths
        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    if (dist[i][k] + dist[k][j] < dist[i][j])
                        dist[i][j] = dist[i][k] + dist[k][j];

        int h = n - 2; // number of hotels
        int half = h / 2;
```

```
// Bitmask DP: state = (mask of visited hotels, current node, phase)
// phase 0 = going to attraction, phase 1 = returning to HQ
int maxMask = 1 << h;
int dp[maxMask][n][2];
for (int mask = 0; mask < maxMask; mask++)
    for (int i = 0; i < n; i++)
        for (int p = 0; p < 2; p++)
            dp[mask][i][p] = INF;

dp[0][0][0] = 0; // start at HQ, no hotels visited, phase 0

for (int mask = 0; mask < maxMask; mask++) {
    for (int u = 0; u < n; u++) {
        // phase 0: heading to attraction
        if (dp[mask][u][0] < INF) {
            // try visiting hotels
            for (int hIdx = 0; hIdx < h; hIdx++) {
                if (!(mask & (1 << hIdx))) {
                    int v = hIdx + 1; // hotel index
                    int newMask = mask | (1 << hIdx);
                    dp[newMask][v][0] = min(dp[newMask][v][0],
                        dp[mask][u][0] + dist[u][v]);
                }
            }
            // try going to attraction
            if (u != n-1) {
                dp[mask][n-1][1] = min(dp[mask][n-1][1],
                    dp[mask][u][0] + dist[u][n-1]);
            }
        }
        // phase 1: returning to HQ
        if (dp[mask][u][1] < INF) {
            // try visiting hotels
            for (int hIdx = 0; hIdx < h; hIdx++) {
                if (!(mask & (1 << hIdx))) {
                    int v = hIdx + 1;
                    int newMask = mask | (1 << hIdx);
                    dp[newMask][v][1] = min(dp[newMask][v][1],
                        dp[mask][u][1] + dist[u][v]);
                }
            }
```

```
                // try going back to HQ
                if (u != 0) {
                    dp[mask][0][1] = min(dp[mask][0][1],
                        dp[mask][u][1] + dist[u][0]);
                }
            }
        }
    }

    int ans = INF;
    for (int mask = 0; mask < maxMask; mask++) {
        if (__builtin_popcount(mask) == h) {
            ans = min(ans, dp[mask][0][1]);
        }
    }

    printf("Case %d: %d\n", caseNo++, ans);
  }
  return 0;
}
```

Status : Wrong **Marks : 0/10**

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. Consider the following two sequences:

X = < B, C, D, C, A, B, C >Y = < C, A, D, B, C, B >

The length of longest common subsequence of X and Y is:

Answer

5

Status : Wrong ***Marks : 0/1***

2. Which of the following problems should be solved using dynamic programming?

Answer

Longest common subsequence

Status : Correct ***Marks : 1/1***

3. Which of the following problems is closely related to the Subsets Sum problem and can also be solved using dynamic programming?

Answer

Coin Change Problem

Status : Correct ***Marks : 1/1***

4. Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B.

Then x + 10y = _____.

Answer

33

Status : Wrong ***Marks : 0/1***

5. Consider the strings "PQRSTPQRS" and "PRATPBRQRPS". What is the length of the longest common subsequence?

Answer

7

Status : Correct ***Marks : 1/1***

6. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct ***Marks : 1/1***

7. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct ***Marks : 1/1***

8. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct ***Marks : 1/1***

9. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Computing the edit distance between two strings

Status : Correct ***Marks : 1/1***

10. When a top-down approach to dynamic programming is applied to a problem, it usually _____.

Answer

decreases the time complexity and increases the space complexity

Status : Correct ***Marks : 1/1***

11. Which of the following problems is NOT solved using dynamic programming?

Answer

vtu29890

Fractional knapsack problem

Status : Correct ***Marks : 1/1***

12. Which of the following is NOT a characteristic of dynamic programming?

Answer

Solving problems in a sequential manner.

Status : Correct ***Marks : 1/1***

13. What happens when a top-down approach of dynamic programming is applied to any problem?

Answer

It increases the space complexity and decreases the time complexity.

Status : Correct ***Marks : 1/1***

14. The Fibonacci sequence is often used to illustrate dynamic programming concepts. What is the time complexity of a naive recursive implementation of Fibonacci numbers?

Answer

O(2^n)

Status : Correct ***Marks : 1/1***

15. What are the different techniques to solve dynamic programming problems:

Answer

Memoization and Bottom-Up

Status : Correct ***Marks : 1/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 11_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 37.5

Section 1 : Coding

1. Problem Statement

Write a program to implement an optimal binary search tree using dynamic programming.

Given a sorted array key [0... n-1] and an array freq [0... n-1] of frequency counts, where freq [i] is the number of searches for key [i], Construct a binary search tree of all keys such that the total cost of all the searches is as small as possible.

Note: Cost of a BST node: level of that node * frequency. The level of the root is 1.

Input Format

The first line of the input is the value of n.

vtu29890

The second line of input is the keys separated by a single space.

The last line of input is the frequency separated by a single space.

Output Format

The output prints the cost of the optimal binary search tree.

Sample Test Case

Input: 3
10 12 20
34 8 50

Output: Cost of Optimal BST is 142

Answer

```
#include <stdio.h>
#include <limits.h>

int sumFreq(int freq[], int i, int j) {
   int s = 0;
   for (int k = i; k <= j; k++) s += freq[k];
   return s;
}

int main() {
   int n;
   scanf("%d", &n);

   int keys[n], freq[n];
   for (int i = 0; i < n; i++) scanf("%d", &keys[i]);
   for (int i = 0; i < n; i++) scanf("%d", &freq[i]);

   int cost[n][n];

   // cost[i][i] = freq[i] (only one key, root at level 1)
   for (int i = 0; i < n; i++) cost[i][i] = freq[i];

   // chain length from 2 to n
   for (int L = 2; L <= n; L++) {
      for (int i = 0; i <= n - L; i++) {
         int j = i + L - 1;
         cost[i][j] = INT_MAX;
```

```
        int sum = sumFreq(freq, i, j);

        // try making each key in interval [i..j] the root
        for (int r = i; r <= j; r++) {
            int c = (r > i ? cost[i][r - 1] : 0) +
                    (r < j ? cost[r + 1][j] : 0) +
                    sum;
            if (c < cost[i][j]) cost[i][j] = c;
        }
    }
}

printf("Cost of Optimal BST is %d", cost[0][n - 1]);
return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, val[0..n-1] and wt[0..n-1] which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of val[] such that the sum of the weights of this subset is smaller than or equal to W. You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50
Output: 220

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    scanf("%d", &n);

    int val[n], wt[n];
    for (int i = 0; i < n; i++) scanf("%d", &val[i]);
    for (int i = 0; i < n; i++) scanf("%d", &wt[i]);
    scanf("%d", &W);

    int dp[n+1][W+1];

    // Build DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
```

```
    }
    printf("%d", dp[n][W]);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given an array, the task is to find the sum of the maximum sum by alternating subsequences starting with the first element. Here alternating sequence means first decreasing, then increasing, then decreasing,... For example, 10, 5, 14, and 3 are alternating sequences.

Note: The reverse sequence type (increasing—decreasing—increasing—...) is not considered alternating here.

Example

Input:

6

4 3 8 5 3 8

Output:

28

Explanation:

The alternating subsequence (starting with the first element) that has the maximum sum is {4, 3, 8, 5, 8}

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output displays a single integer representing the maximum sum of an alternating subsequence from the given array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
4 3 8 5 3 8

Output: 28

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) { return a > b ? a : b; }

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    // dp[i][0] = max sum ending at i expecting a decrease next
    // dp[i][1] = max sum ending at i expecting an increase next
    int dp[n][2];

    // Initialize with first element
    dp[0][0] = arr[0];  // after first element, next must be decrease
    dp[0][1] = arr[0];  // also track for consistency

    int ans = arr[0];

    for (int i = 1; i < n; i++) {
        dp[i][0] = arr[i]; // start new subsequence at arr[i]
        dp[i][1] = arr[i];

        for (int j = 0; j < i; j++) {
            // If we expect decrease, previous must be larger
```

```
            if (arr[j] > arr[i]) {
                dp[i][0] = max(dp[i][0], dp[j][1] + arr[i]);
            }
            // If we expect increase, previous must be smaller
            if (arr[j] < arr[i]) {
                dp[i][1] = max(dp[i][1], dp[j][0] + arr[i]);
            }
        }
        ans = max(ans, max(dp[i][0], dp[i][1]));
    }

    printf("%d", ans);
    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

4. Problem Statement

A taxi driver wants to maximize the cab fare by choosing the longest possible travel path through a city represented as a square matrix.

Each cell in the matrix represents the kilometers travelled at that location.

The taxi can move only in two directions:

Right (i, j+1)Down (i+1, j)

The driver will move to the next cell only if the difference between the current value and the next value is exactly +1 or -1.

Rules:

Movement is allowed only right or down.The absolute difference between the current cell value and the next cell value must be 1.If both right and down moves satisfy the condition, the driver always prefers the right move.The trip continues until no valid move is possible.

Your task is to determine and print the longest trip path starting from the first cell (0,0).

Input Format

The first line contains an integer N, representing the size of the square matrix.

The next N lines contain N space-separated integers, representing the matrix values.

Output Format

The output prints the longest trip path starting from the first element of the matrix in the format "Longest trip path a->b->c->d" where a, b, c, d are the values visited in the path.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
2 3 1
0 4 5
8 7 6

Output: Longest trip path 2->3->4->5->6

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int absDiff(int a, int b) {
   return (a > b) ? (a - b) : (b - a);
}

int main() {
   int N;
   scanf("%d", &N);

   int matrix[N][N];
   for (int i = 0; i < N; i++)
     for (int j = 0; j < N; j++)
          scanf("%d", &matrix[i][j]);
```

```
    // Path storage
    int path[N * N];
    int len = 0;

    int i = 0, j = 0;
    path[len++] = matrix[i][j];

    while (1) {
        int moved = 0;

        // Prefer right move if valid
        if (j + 1 < N && absDiff(matrix[i][j], matrix[i][j + 1]) == 1) {
            j++;
            path[len++] = matrix[i][j];
            moved = 1;
        }
        // Else try down move
        else if (i + 1 < N && absDiff(matrix[i][j], matrix[i + 1][j]) == 1) {
            i++;
            path[len++] = matrix[i][j];
            moved = 1;
        }

        if (!moved) break; // no valid move
    }

    printf("Longest trip path ");
    for (int k = 0; k < len; k++) {
        if (k > 0) printf("->");
        printf("%d", path[k]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 12_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of non-negative integers, the task is to find the minimum number of elements such that their sum should be greater than the sum of the rest of the elements of the array.

Example:

Input: arr[] = [3 , 1 , 7, 1]

Output: 1

Explanation: Smallest subset is {7}. Sum of this subset is greater than the sum of all other elements left after removing subset {7} from the array

Input: arr[] = [2 , 1 , 2]

Output: 2

Explanation: Smallest subset is {2 , 1}. Sum of this subset is greater than the sum of all other elements left after removing subset {2 , 1} from the array

Input Format

The first line contains an Integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single integer representing the minimum number of elements whose sum is greater than the sum of the remaining elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 1 7 1

Output: 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for sorting in descending order
int cmp(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    int totalSum = 0;
    for(int i = 0; i < n; i++) {
```

```
    scanf("%d", &arr[i]);
    totalSum += arr[i];
  }

  // Sort array in descending order
  qsort(arr, n, sizeof(int), cmp);

  int subsetSum = 0, count = 0;
  for(int i = 0; i < n; i++) {
    subsetSum += arr[i];
    count++;
    if(subsetSum > totalSum - subsetSum) {
      printf("%d", count);
      return 0;
    }
  }

  return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given an array prices[] of n different candies, where prices[i] represents the price of the ith candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to k other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

Note: In both cases, you must take the maximum number of free candies possible during each purchase.

Examples:

Input: prices[] = [3, 2, 1, 4], k = 2

Output: [3, 7]

Explanation: For minimum cost, buy the candies costing 1 and 2. for maximum cost, buy the candies costing 4 and 3.

Input: prices[] = [3, 2, 1, 4, 5], k = 4

Output: [1, 5]

Explanation: For minimum cost, buy the candy costing 1. for maximum cost, buy the candy costing 5.

Input Format

The first line contains an Integer n, representing the number of candies.

The second line contains n space-separated integers, representing the prices of the candies.

The third line contains an integer k, representing the number of free candies per purchase.

Output Format

The output prints two integers separated by a space:

minCost maxCost

representing the minimum and maximum money needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 1 4
2
Output: 3 7

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for sorting in ascending order
int cmpAsc(const void *a, const void *b) {
```

```
    return (*(int*)a - *(int*)b);
}

// Comparator for sorting in descending order
int cmpDesc(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n, k;
    scanf("%d", &n);

    int prices[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &prices[i]);
    }
    scanf("%d", &k);

    // ---------- Minimum Cost ----------
    qsort(prices, n, sizeof(int), cmpAsc);
    int minCost = 0;
    int i = 0, j = n - 1;
    while(i <= j) {
        minCost += prices[i]; // buy cheapest
        i++;
        j -= k; // take k most expensive free
    }

    // ---------- Maximum Cost ----------
    qsort(prices, n, sizeof(int), cmpDesc);
    int maxCost = 0;
    i = 0; j = n - 1;
    while(i <= j) {
        maxCost += prices[i]; // buy most expensive
        i++;
        j -= k; // take k cheapest free
    }

    printf("%d %d", minCost, maxCost);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a fraction represented by two integers — a numerator n and a denominator d — express the fraction as a sum of unique unit fractions (fractions with numerator 1). This representation is known as the Egyptian Fraction.

Write a program to ensure that the given fraction is converted into its Egyptian Fraction form.

Input Format

The input contains two space-separated integers n and d, representing the numerator and denominator of the fraction.

Output Format

The output prints the Egyptian Fraction representation of the given fraction in the form of unit fractions separated by +.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 14
Output: 1/3 + 1/11 + 1/231

Answer

```
// You are using GCC
#include <stdio.h>

// Function to print Egyptian Fraction representation
void printEgyptianFraction(int n, int d) {
   while (n != 0) {
      // Calculate the ceiling of d/n
      int x = (d + n - 1) / n;  // equivalent to ceil(d/n)
```

```
        // Print the unit fraction
        printf("1/%d", x);

        // Reduce the fraction: n/d - 1/x = (n*x - d) / (d*x)
        n = n * x - d;
        d = d * x;

        if (n != 0) {
            printf(" + ");
        }
    }
}

int main() {
    int n, d;
    scanf("%d %d", &n, &d);

    printEgyptianFraction(n, d);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Emily is building a data compression feature in her file organizing system. She wants filenames to have no repeated characters and appear in the smallest possible order so users can locate files easily. For example, if a corrupted filename is "cbacdcbc", she wants it cleaned up as "acdb" — which preserves one instance of each letter, keeps relative order, and is the smallest alphabetically.

This function helps ensure filenames are clean, minimal, and alphabetically organized!

Example 1:

Input: s = "bcabc"

Output: "ABC"

Example 2:

Input: s = "cbacdcbc"

Output: "acdb"

Input Format

The first line of input contains a string num representing a non-negative integer.

The second line contains an integer k representing how many digits to remove.

Output Format

The output prints the string representing the smallest possible number after removing exactly k digits.

Leading zeros should be removed, except if the number is zero itself.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: bcabc

Output: abc

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    char s[10005];
    scanf("%s", s);

    int n = strlen(s);
    int freq[26] = {0};
    bool inStack[26] = {false};

    // Count frequency of each character
```

```
    for (int i = 0; i < n; i++) {
        freq[s[i] - 'a']++;
    }

    char stack[10005];
    int top = -1;

    for (int i = 0; i < n; i++) {
        int c = s[i] - 'a';
        freq[c]--; // one occurrence used

        if (inStack[c]) continue; // skip if already in result

        // Greedy check: pop larger chars if they appear later again
        while (top >= 0 && stack[top] > s[i] && freq[stack[top] - 'a'] > 0) {
            inStack[stack[top] - 'a'] = false;
            top--;
        }

        stack[++top] = s[i];
        inStack[c] = true;
    }

    stack[top + 1] = '\0';
    printf("%s", stack);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Given the arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the station so that no train is kept waiting. Consider that all the trains arrive on the same day and leave on the same day.

Arrival and departure times can never be the same for a train, but we can have the arrival time of one train equal to the departure time of the other.

At any given instance, the same platform cannot be used for both the departure of a train and the arrival of another train. In such cases, we need different platforms.

Example 1

Input:

N=6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}, dep[] = {0910, 1200, 1120, 1130, 1900, 2000}

Output: 3

Explanation: A minimum of three platforms are required to safely arrive and depart all trains.

Input Format

The first line consists of an integer N, which represents the total number of trains.

The second line consists of N space-separated integers representing the arrival time.

The third line consists of N space-separated integers representing the departure time.

Output Format

The output displays an integer which is the minimum number of platforms required to safely arrive and depart the train.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
0900 0940 0950 1100 1500 1800
0910 1200 1120 1130 1900 2000

vtu29890

Output: 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for sorting integers
int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n], dep[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    for(int i = 0; i < n; i++) {
        scanf("%d", &dep[i]);
    }

    // Sort arrival and departure times
    qsort(arr, n, sizeof(int), cmp);
    qsort(dep, n, sizeof(int), cmp);

    int platforms = 1, result = 1;
    int i = 1, j = 0;

    // Traverse arrival and departure arrays
    while(i < n && j < n) {
        if(arr[i] <= dep[j]) {
            platforms++;   // new train arrives before previous departs
            i++;
        } else {
            platforms--;   // train departs, free a platform
            j++;
        }
        if(platforms > result) {
            result = platforms;
```

```
        }
    }

    printf("%d", result);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 12_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

Raju has a hot dog of a certain length that he wishes to divide strategically to maximize the number of friends he can serve. The effectiveness of a given partition is determined by the product of its segment lengths. Raju's goal is to determine the optimal way to cut the hot dog such that this product is maximized, thereby maximizing the number of friends he can serve.

Example

Input:

4

Output:

Maximum friends served 4

Explanation:

For a hot dog of length 4, possible partitions and their corresponding products are:

1, 3 (1 × 3) = 3

2, 2 (2 × 2) = 4 (Maximum)

3, 1 (3 × 1) = 3

1, 1, 2 (1 × 1 × 2) = 2

1, 2, 1 (1 × 2 × 1) = 2

2, 1, 1 (2 × 1 × 1) = 2

1, 1, 1, 1 (1 × 1 × 1 × 1) = 1

Thus, the optimal partition results in a maximum product of 4, serving the highest number of friends.

Input Format

The input consists of a single integer n, representing the length of the hot dog.

Output Format

The output prints the maximum number of friends served.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Output: Maximum friends served 4

Answer

```
// You are using GCC
#include <stdio.h>

// Function to compute maximum product partition
int maxProduct(int n) {
    if (n == 1) return 1;
```

```
    int dp[n+1];
    for (int i = 0; i <= n; i++) dp[i] = 0;

    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        for (int j = 1; j < i; j++) {
            // Either split into j and (i-j), or keep (i-j) as whole
            int product = j * (i - j);
            int productWithSplit = j * dp[i - j];
            if (product > dp[i]) dp[i] = product;
            if (productWithSplit > dp[i]) dp[i] = productWithSplit;
        }
    }
    return dp[n];
}

int main() {
    int n;
    scanf("%d", &n);

    int result = maxProduct(n);
    printf("Maximum friends served %d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are given a set of items, each with a weight, a value, and a penalty value. You need to solve the Fractional Knapsack problem with the objective of maximizing the total value while minimizing penalties. Implement a greedy algorithm to choose items that best meet this objective.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 14.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 1: (7 - 1) / 2 = 3

Item 2: (12 - 4) / 3 = 2

Start filling the knapsack:

Add all of Item 1 (Weight: 2, Value: 7 - 1 = 6)

Add 1 unit of Item 2 (Weight: 3, Value: 12 - 4 = 8)

The total value is 6 + 8 = 14.

So, the correct output for the given input is 14.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

vtu29890

Maximum total value while minimizing penalties: 43.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 2: (30 - 5) / 10 = 2.5

Item 1: (20 - 2) / 5 = 3.6

Start filling the knapsack:

Add all of Item 1 (Weight: 5, Value: 20 - 2 = 18)

Add 1 unit of Item 2 (Weight: 10, Value: 30 - 5 = 25)

Now, let's calculate the total value of the knapsack:

Total Weight = 5 + 10 = 15 (fits within the knapsack's capacity)

Total Value = 18 + 25 = 43

Input Format

The first line contains an integer 'N' representing the number of items.

The next 'N' lines contain information for each item:

Three integers 'weight,' 'value,' and 'penalty,' are separated by spaces, where 'weight' is the weight of the item, 'value' is the value of the item, and 'penalty' is the penalty for taking the item.

The last line contains an integer 'knapsackWeight' representing the weight limit of the knapsack.

Output Format

The output prints "Maximum total value while minimizing penalties: " followed by the maximum total value achievable while minimizing penalties, rounded to two decimal places.

vtu29890

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
2 7 1
3 12 4
5
Output: Maximum total value while minimizing penalties: 14.00

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int weight;
    int value;
    int penalty;
    double ratio;
    double effectiveValue;
} Item;

int compare(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i1->ratio < i2->ratio) return 1;
    if (i1->ratio > i2->ratio) return -1;
    return 0;
}

int main() {
    int N;
    scanf("%d", &N);

    Item items[N];
    for (int i = 0; i < N; i++) {
        scanf("%d %d %d", &items[i].weight, &items[i].value, &items[i].penalty);
        items[i].effectiveValue = items[i].value - items[i].penalty;
        items[i].ratio = (double)items[i].effectiveValue / items[i].weight;
    }

    int knapsackWeight;
```

```
    scanf("%d", &knapsackWeight);

    // Sort items by ratio descending
    qsort(items, N, sizeof(Item), compare);

    double totalValue = 0.0;
    int remainingWeight = knapsackWeight;

    for (int i = 0; i < N && remainingWeight > 0; i++) {
        if (items[i].weight <= remainingWeight) {
            totalValue += items[i].effectiveValue;
            remainingWeight -= items[i].weight;
        } else {
            // Fractional part
            totalValue += items[i].ratio * remainingWeight;
            remainingWeight = 0;
        }
    }

    printf("Maximum total value while minimizing penalties: %.2f", totalValue);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

There are n participants in a race. Each participant is assigned a performance score given in the integer array performance.

You are responsible for awarding medals to these participants, subject to the following conditions:

Each participant must receive at least one medal.

Participants with a higher performance score than their neighbors must receive more medals than their neighbors.

Return the minimum number of medals you need to distribute to the participants.

Example 1:

Input: ratings = [1,0,2]

Output: 5

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 1 medal (since their performance is the lowest).

Participant 3 gets 2 medals (since their performance is higher than Participant 2).

The total number of medals required is 5.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 2 medals (since their performance is better than Participant 1).

Participant 3 gets 1 medal (because their performance is equal to Participant 2, and they don't need more medals).

The total number of medals required is 4.

Input Format

The first line consists of an integer n representing the number of participants.

The second line consists of n space-separated integers representing the performance scores of the participants.

Output Format

An integer representing the minimum number of medals required to satisfy the conditions.

vtu29890

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 0 2
Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    scanf("%d", &n);

    int ratings[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &ratings[i]);
    }

    int medals[n];
    for (int i = 0; i < n; i++) {
        medals[i] = 1;  // each participant gets at least 1 medal
    }

    // Left to right pass
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1]) {
            medals[i] = medals[i - 1] + 1;
        }
    }

    // Right to left pass
    for (int i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1] && medals[i] <= medals[i + 1]) {
            medals[i] = medals[i + 1] + 1;
        }
    }
```

```
    long long total = 0;
    for (int i = 0; i < n; i++) {
        total += medals[i];
    }

    printf("%lld", total);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

You are planning a circular road trip that passes through several gas stations. Each station provides a certain amount of fuel, and traveling from one station to the next consumes a specific amount of fuel.

Your goal is to determine a starting gas station index from which you can begin your journey such that you complete the entire circuit in a clockwise direction without running out of fuel at any point.

Given two integer arrays:

gas[i] represents the fuel available at the i-th station,

cost[i] represents the fuel required to travel from station i to station i+1 (circularly),

Return the starting station index that allows completing the full trip.

If no such station exists, return -1.

Example 1

Input:

5

1 2 3 4 5

3 4 5 1 2

Output:

3

Explanation:

Start at station 3 (index 3) and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 4. Your tank = 4 - 1 + 5 = 8

Travel to station 0. Your tank = 8 - 2 + 1 = 7

Travel to station 1. Your tank = 7 - 3 + 2 = 6

Travel to station 2. Your tank = 6 - 4 + 3 = 5

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

Example 2

Input

3

2 3 4

3 4 3

Output:

-1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 0. Your tank = 4 - 3 + 2 = 3

Travel to station 1. Your tank = 3 - 3 + 3 = 3

You cannot travel back to station 2, as it requires 4 units of gas but you only have 3.

Therefore, you can't travel around the circuit once no matter where you start.

Input Format

The first line of input consists of an integer N, representing the number of gas stations.

The second line of input consists of N space-separated integers representing the amount of gas available at each station.

The third line of input consists of N space-separated integers representing the cost of traveling from each station to the next.

Output Format

The output prints an integer representing the index of the starting gas station that allows you to complete the circuit, or -1 if no such station exists.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
3 4 5 1 2
Output: 3

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int gas[n], cost[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &gas[i]);
    }
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &cost[i]);
    }

    int totalGas = 0, totalCost = 0;
    for (int i = 0; i < n; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
    }

    // If total gas < total cost, impossible to complete circuit
    if (totalGas < totalCost) {
        printf("-1");
        return 0;
    }

    int start = 0, tank = 0;
    for (int i = 0; i < n; i++) {
        tank += gas[i] - cost[i];
        if (tank < 0) {
            // reset starting point
            start = i + 1;
            tank = 0;
        }
    }

    printf("%d", start);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 12

Section 1 : MCQ

1. The first step in the naive greedy algorithm is _____.

Answer

analysing the zero flow

Status : Correct ***Marks : 1/1***

2. Suppose you have coins of denominations 1,3 and 4. You use a greedy algorithm, in which you choose the largest denomination coin which is not greater than the remaining sum. For which of the following sums, will the algorithm doesn't produce an optimal answer?

Answer

10

vtu29890

Status : Correct ***Marks : 1/1***

3. Which data structure is most commonly used to efficiently implement a greedy algorithm that repeatedly selects the minimum (or maximum) element at each step?

Answer

Heap

Status : Correct ***Marks : 1/1***

4. What is a greedy algorithm?

Answer

An algorithm that only works for problems with small input size

Status : Wrong ***Marks : 0/1***

5. Which of the following problems is typically solved using a greedy algorithm?

Answer

Coin change problem.

Status : Wrong ***Marks : 0/1***

6. What is the primary advantage of greedy algorithms over other approaches for certain problems?

Answer

Greedy algorithms are easy to implement

Status : Correct ***Marks : 1/1***

7. Which of the following is a disadvantage of greedy algorithm?

vtu29890

Answer

It may not always find the globally optimal solution

Status : Correct ***Marks : 1/1***

8. Which of the following approaches should be used to find the solution to the activity selection problem?

Answer

Greedy approach

Status : Correct ***Marks : 1/1***

9. Suppose two activities A and B, have start and finish times as SA, FA and SB, FB respectively. Both activities are said to be compatible, under which of the following conditions?

Answer

SA >= FB or SB >= FA

Status : Correct ***Marks : 1/1***

10. In the activity selection problem, if two activities have the same finish time, which one is selected?

Answer

The one with the earlier start time

Status : Correct ***Marks : 1/1***

11. Consider the following number of activities with their start and finish time given below. In which sequence will the activity be selected in order to maximize the number of activities, without any conflicts?

Answer

A1, A3, A5

Status : Wrong ***Marks : 0/1***

12. Which of the following is a characteristic of the greedy algorithm?

Answer

It makes the best choice at each step by considering only the local problem state.

Status : Correct ***Marks : 1/1***

13. What is the primary reason why the greedy algorithm does not work for some optimization problems?

Answer

It does not reconsider previous decisions and makes choices based only on the current state.

Status : Correct ***Marks : 1/1***

14. In the greedy approach using a stack, what condition do we check before popping from the stack

Answer

If the top is greater than current digit and k > 0

Status : Correct ***Marks : 1/1***

15. Why is sorting not used in this greedy solution?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

vtu29890

Vel Tech Group of Institutions

Name: BANNEPALLI SANJANA
Email: vtu29890@veltech.edu.in
Roll no: vtu29890
Phone: null
Branch: VTU
Department: CSE-AIML
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results

2028_NeoColab_Competitive Coding II_L15

VTU_Week 12_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an amount, find the minimum number of notes of different denominations that sum up to the given amount. Starting from the highest denomination note, try to accommodate as many notes as possible for a given amount.

We may assume that we have infinite supply of notes of values {2000, 500, 200, 100, 50, 20, 10, 5, 1}

Examples:

Input: 800

Output: Currency Count

500 : 1

200 : 1

100 : 1

Input: 2456

Output: Currency Count

2000 : 1

200 : 2

50 : 1

5 : 1

1 : 1

Input Format

The input contains an integer representing the amount to be broken down.

Output Format

The output prints "Currency Count" followed by each denomination used in the format:

"note_value : count"

(Only print denominations that are used.)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 800
Output: Currency Count
500 : 1
200 : 1
100 : 1

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int amount;
    scanf("%d", &amount);

    int denominations[] = {2000, 500, 200, 100, 50, 20, 10, 5, 1};
    int n = sizeof(denominations) / sizeof(denominations[0]);

    printf("Currency Count\n");

    for (int i = 0; i < n; i++) {
        if (amount >= denominations[i]) {
            int count = amount / denominations[i];
            amount = amount % denominations[i];
            printf("%d : %d\n", denominations[i], count);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a lock which is made up of n-different circular rings and each ring has 0-9 digit printed serially on it. Initially all n-rings together show a n-digit integer but there is particular code only which can open the lock.

You can rotate each ring any number of time in either direction. You have to find the minimum number of rotation done on rings of lock to open the lock.

Examples:

Input: Input = 2345, Unlock code = 5432

Output: Rotations required = 8

Explanation: 1st ring is rotated thrice as 2->3->4->5

2nd ring is rotated once as 3->4

3rd ring is rotated once as 4->3

4th ring is rotated thrice as 5->4->3->2

Input: Input = 1919, Unlock code = 0000

Output: Rotations required = 4

Explanation: 1st ring is rotated once as 1->0

2nd ring is rotated once as 9->0

3rd ring is rotated once as 1->0

4th ring is rotated once as 9->0

Input Format

The first line contains an integer representing the current input on the lock.

The second line contains an integer representing the unlock code.

Output Format

The output prints "Minimum Rotation = X" where X is the minimum total number of rotations needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 28756
98234

Output: Minimum Rotation = 12

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
  char input[100], unlock[100];
  scanf("%s", input);
```

```
    scanf("%s", unlock);

    int n = strlen(input);
    int totalRotations = 0;

    for (int i = 0; i < n; i++) {
        int currentDigit = input[i] - '0';
        int targetDigit = unlock[i] - '0';

        int diff = abs(currentDigit - targetDigit);
        int rotations = diff < (10 - diff) ? diff : (10 - diff);

        totalRotations += rotations;
    }

    printf("Minimum Rotation = %d", totalRotations);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement:

Sarah is organizing a race in her local community park, where participants must jump from one marker to the next. Each participant starts at the first marker and can make jumps to other markers based on the distance provided for each marker. Sarah wants to calculate the minimum number of jumps it would take for a participant to reach the final marker.

You are tasked with writing a program that helps Sarah determine the minimum number of jumps required for a participant to reach the last marker.

Each element in the array represents the maximum number of steps a participant can jump forward from that marker. For example, if you're at marker i, you can jump to any marker within the range [i+1, i + nums[i]].

Write a program to calculate the minimum number of jumps to reach the last marker.

Example 1:

Input: nums = 2 3 1 1 4

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = 2 3 0 1 4

Output: 2

Input Format

The input consists of space-separated integers representing the jump lengths from each index.

Output Format

The output prints an integer, representing the minimum number of jumps required to reach the last index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2 3 1 1 4
Output: 2

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int nums[10000], n = 0;
    while (scanf("%d", &nums[n]) == 1) {
        n++;
    }

    int jumps = 0;
```

```
    int currentEnd = 0;
    int farthest = 0;

    for (int i = 0; i < n - 1; i++) {
        if (i + nums[i] > farthest) {
            farthest = i + nums[i];
        }
        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }

    printf("%d", jumps);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Shoot the Kuruvi (Balloons)

There are some spherical balloons taped onto a flat wall that represent the XY-plane. The balloons are represented as a 2D integer array points, where points[i] = [xstart, xend] denotes a balloon whose horizontal diameter stretches between xstart and xend. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with xstart and xend is burst by an arrow shot at x if xstart <= x <= xend. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the number of array points, followed by the array points. Return the minimum number of arrows that must be shot to burst all balloons.

Example

Input

4

10 16

2 8

1 6

7 12

Output

2

Explanation

The balloons can be burst by 2 arrows:

Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].

Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].

Input

4

1 2

3 4

5 6

7 8

Output

4

Explanation

One arrow needs to be shot for each balloon for a total of 4 arrows.

Input

4

1 2

vtu29890

2 3

3 4

4 5

Output

2

Explanation

The balloons can be burst by 2 arrows:

Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].

Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].

Input Format

The first line of input consists of a single integer n, representing the Number of balloons

The next n lines contain two integers xstart and xend, representing the start and end positions of each balloon

Output Format

The output prints a single integer, which represents the minimum number of arrows needed to burst all the balloons.

Refer to the output for the formatting specifications.

Sample Test Case

Input: 4
1 2
2 3
3 4
4 5

Output: 2

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>

typedef struct {
    int start;
    int end;
} Balloon;

int compare(const void *a, const void *b) {
    Balloon *b1 = (Balloon *)a;
    Balloon *b2 = (Balloon *)b;
    return b1->end - b2->end;  // sort by end ascending
}

int main() {
    int n;
    scanf("%d", &n);

    Balloon balloons[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &balloons[i].start, &balloons[i].end);
    }

    // Sort balloons by end coordinate
    qsort(balloons, n, sizeof(Balloon), compare);

    int arrows = 1;  // at least one arrow
    int arrowPos = balloons[0].end;

    for (int i = 1; i < n; i++) {
        if (balloons[i].start > arrowPos) {
            arrows++;
            arrowPos = balloons[i].end;
        }
    }

    printf("%d", arrows);
    return 0;
}
```

Status : Correct ***Marks : 10/10***